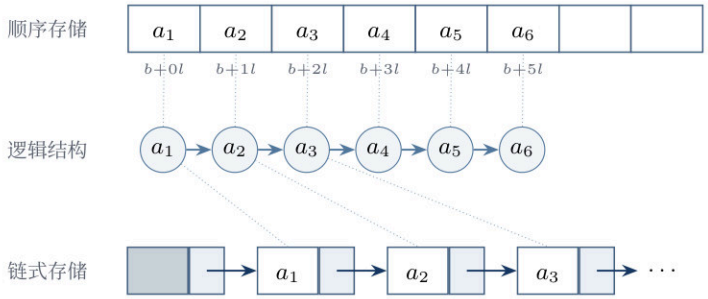
线性表

从逻辑结构到顺序表、链表

可视化基础讲义·含第一章绪论核心概念（数据结构、ADT、算法复杂度）



适用读者　第一次学习线性表、C++ 指针与类模板基础尚不牢固的本科二年级同学

556；

图

配套教材　游洪跃、唐宁九《数据结构与算法（C++ 版）》第 2 版

代码约定　全部代码为 C++（C++17），已用 g++ 13 编译运行验证

2026 年 10 月

阅读说明

这份讲义按“**为什么→直观图→定义→公式/代码→推导→例子→易错→对比→联系**”的顺序展开。每个核心知识点都配有主图、一个“生活中的它”和一个“相似知识对照”。正式练习 38 道集中放在后半部分，答案与解析放在全部练习之后，请先独立做完再看。

视觉元素	含义
定义 / 定理	深蓝标题 + 浅蓝灰底：必须掌握的核心定义、定理、公式
蓝灰左线	为什么需要 / 直观理解 / 知识联系 / 相似知识对照
暖米白	例题、生活中的它、学科史、科普
极浅红	易错提醒、原图纠错（红色只用在这里）
极浅绿	小结、方法总结、学完自检
原图 教学补全	分别表示：原始资料可靠识别的内容；为讲懂而按标准教材补全的内容；当前阶段不要求掌握的延伸
拓展联想	
✓ △ ✗	迁移判断：可直接迁移 / 加条件才成立 / 不能迁移

两个贯穿全书的约定（务必先看）

1. **位序从 1 数，下标从 0 数。** 线性表的“第 i 个元素 a_i”在 C++ 数组中存放在 `elems[i-1]`。课件与教材的 `GetElem(position, e)`、`Insert(position, e)` 中的 `position` 都是位序。
2. **指针图的画法。** 方框左格是数据域 `data`，右格（浅蓝）是指针域 `next`；灰色方框是头结点；∧ 表示空指针 `nullptr`；绿色实线是新建立的指针，红色虚线是被断开或错误的指针，橙色斜体是工作指针名。

目录

阅读说明 1

本专题学什么：三个问题与学习目标 2

本专题在课程体系中的位置与前置知识 3

学科小史：线性表和复杂度是怎样出现的？ 3

从课本到现实：线性表到底用在哪里？ 4

原始资料重编码：本讲义依据了哪些材料 5

原图纠错 5

知识点总览与关系图 7

1　绪论基础：数据结构、ADT 与算法复杂度 7

1.1　数据、数据元素、数据项与逻辑结构 7

1.2 存储结构：把关系“放进”内存 . . . 10
1.3 抽象数据类型（ADT） . . . 12
1.4 算法及其特性 . . . 14
1.5 时间复杂度 . . . 15
1.6 空间复杂度 . . . 20
本章回顾与知识地图 . . . 21

2 线性表的逻辑结构与抽象数据类型 21
2.1 线性表的定义与逻辑特征 . . . 21
2.2 线性表的抽象数据类型 . . . 23
2.3 只用基本操作解决问题：并集、有序归并、去重 . . . 24

3 线性表的顺序存储：顺序表 27
3.1 顺序存储结构与地址公式 . . . 27
3.2 顺序表的 C++ 实现 . . . 29
3.3 插入与删除：为什么要移动，平均移动多少 . . . 31
3.4 顺序表的应用：差集与稀疏多项式相加 . . . 34
本章回顾：顺序表的优缺点 . . . 37

4 线性表的链式存储：链表 37
4.1 链式存储结构与单链表 . . . 37
4.2 单链表基本操作的实现 . . . 40
4.3 循环链表 . . . 45
4.4 双向链表与双向循环链表 . . . 48
4.5 补充：在链表中保存当前位置和元素个数（课件 2 P61–P63） . . . 50
4.6 单链表、循环链表和双向链表的时间效率比较 . . . 51

5 顺序表与链表的比较 52
5.1 从两个维度比较 . . . 52
5.2 怎样选择 . . . 53

正式练习（38 题） 55
知识点—题目对应表 . . . 55
A 基础巩固训练 . . . 55
B 中等偏下提升训练 . . . 59
C 跨学科迁移训练 . . . 61

全部答案速查 62

详细解析：A. 基础巩固 62

详细解析：B. 中等偏下提升 70

详细解析：C. 跨学科迁移 74

总结：高频易错点总表 75

基础解题步骤总结 75

相近概念总对比 76

跨学科 / 相似知识总对比表 76

后续课程连接 77

参考资料 77

本专题学什么：三个问题与学习目标

生活里很多数据天然是“一个接一个”的：排队的人、歌单里的歌、通讯录里按拼音排好的联系人、一个多项式按指数排好的各项。线性表研究的就是这种**“一个挨一个”的数据**。本专题回答三个问题：

问题 1：**“一个挨一个”在数学上怎么严格描述？**——线性表的逻辑结构与抽象数据类型（第 2 章）。

问题 2：**计算机内存里怎么存？**——两种实现：连续存放的**顺序表**（第 3 章）与用指针串起来的**链表**（第 4 章）。

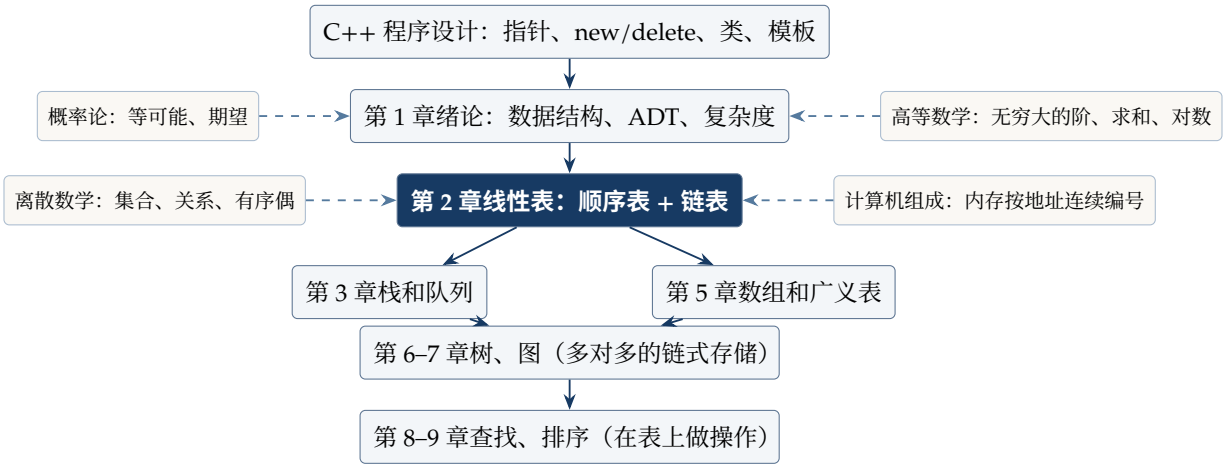
问题 3：**怎样评价、怎样选择？**——用时间复杂度、空间复杂度衡量操作代价（第 1 章），并比较两种实现（第 5 章）。

第 1 章先把第一章绪论中支撑本专题的概念（数据结构、存储结构、ADT、算法与复杂度）补齐，因为你的笔记把它们放在“§3 线性表的类型定义”中作为补充，后面每一节都要用到。

学习目标

1. 能说清数据、数据元素、数据项、数据对象、逻辑结构、存储结构、ADT 的含义与区别；
2. 能用“基本语句频度→最高阶→大 O”的步骤分析简单循环的时间复杂度，并会用加法、乘法规则；
3. 能写出线性表的定义、逻辑特征和基本操作的初始条件与操作结果；
4. 能推导顺序表的地址公式与插入/删除平均移动次数 $n/2$、$(n-1)/2$，并写出 C++ 实现；
5. 能画出单链表、循环链表、双向链表在插入、删除、建立、逆置时的每一步指针变化，并写出正确顺序的 C++ 语句；
6. 能根据“访问方式、插删频率、表长变化”在顺序表与链表之间做选择。

本专题在课程体系中的位置与前置知识



线性表在课程体系中的位置。实线：本课程内部的先后关系；虚线：其他课程提供的工具。

读本讲义前，最好确认自己能看懂下面这些 C++ 语句；看不懂的，正文第一次用到时会再解释一遍。

C++ 写法	本讲义中的用途
`int *p = new int[n]; delete[] p;`	顺序表动态申请/释放一段连续空间
`Node *s = new Node(e, p->next); delete q;`	链表申请/释放一个结点
`p->next` 等价于 `(*p).next`	通过指针访问结点成员
`void f(SqList<int> &L)`	引用传参：不复制整个表，且能修改实参
`bool GetElem(int i, T &e) const`	末尾 `const`：保证这个成员函数不修改表本身
`template <class ElemType>`	类模板：元素类型可以是 int、double、结构体……

数学上只用到：等差数列求和 $1+2+\cdots+n=\frac{n(n+1)}{2}$、等比数列求和、对数 $\log_2 n$ 的含义，以及“当 $n\to\infty$ 时比较两个函数谁增长得快”。

学科小史：线性表和复杂度是怎样出现的？

早期计算机主要做数值计算（解方程、算弹道），数据就是一串串数字，按地址**连续存放**在内存里最自然——这就是“顺序存储”的物质基础：内存单元按地址连续编号，知道起始地址就能算出任一单元的位置。

1950 年代中期，研究者开始让计算机做**非数值**工作：证明逻辑定理、下棋、翻译语言。这些程序要不断地插入、删除、重组符号，而且事先不知道要多少空间。连续存放此时变得笨拙：中间插一个元素就要挪动后面一大片。于是出现了另一种思路：**每个元素自己记住“下一个在哪里”**——链表。



与本专题直接相关的几个节点（只列与线性表、ADT、复杂度有关的事件）。

三个值得记住的转折

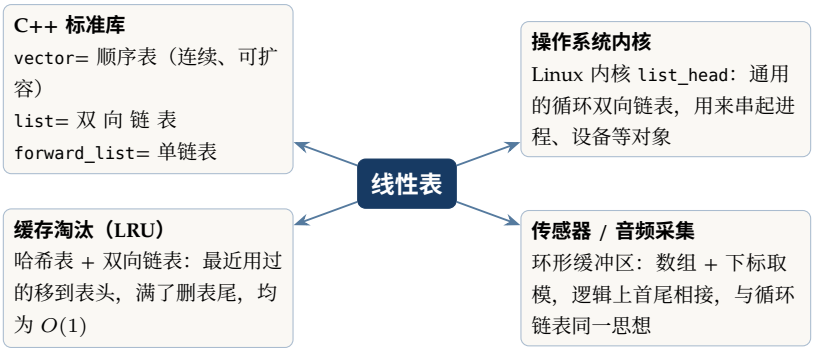
①链表因“非数值计算”而生。1955–1956 年，纽厄尔（A. Newell）、肖（C. Shaw）和西蒙（H. A. Simon）在兰德公司与卡内基理工学院开发信息处理语言 IPL，把链表作为核心数据结构，用来支撑“逻辑理论家”等早期人工智能程序。纽厄尔与西蒙后来获 1975 年图灵奖，颁奖词提到了他们对“表处理”的基础性贡献。

②从“写程序的技巧”到“可以单独研究的学问”。1968 年高德纳（D. E. Knuth）在《计算机程序设计艺术》第 1 卷中把线性表的顺序分配、链接分配、循环链表、双向链表系统化，此后它们成为大学课程的标准内容。1976 年沃思（N. Wirth）的书名《算法 + 数据结构 = 程序》则把这一观点说得很直白。

③把“表”抽象成“一组操作”。1974 年利斯科夫（B. Liskov）与齐勒斯（S. Zilles）发表论文，提出用“对象 + 允许的操作”来定义数据类型，即抽象数据类型。你在课件里看到的 `ADT List { 数据对象；数据关系；基本操作}` 就是这一思想。O 记号本来是数论工具（巴赫曼 1894、朗道 1909），高德纳 1976 年的短文推动了 O、Ω、Θ 在算法分析中的统一用法。

所以今天学线性表，不只是学两种存储方式，而是在学一种至今仍在用的思路：**同一个逻辑结构可以有不同的物理实现，用操作代价来选择实现。**

从课本到现实：线性表到底用在哪里？



课堂上的线性表在真实系统中的四个位置。

为什么工程师要同时会两种实现？

标准库里两种都有。C++ 的 `std::vector` 把元素放在一段连续内存里，按下标访问是常数时间，满了会重新申请更大的空间整体搬过去；`std::list` 是双向链表，任意已知位置插入删除不移动其他元素，但不能像数组那样直接跳到第 i 个。选哪一个，正是第 5 章“顺序表与链表的比较”要回答的问题。

操作系统离不开链表。Linux 内核在 `include/linux/list.h` 中提供了一套通用的循环双向链表 `list_head`，内核里大量对象（例如进程描述符、设备）通过它组织起来。它采用“带头结点的双向循环链表”——正是本讲义 4.4 节的结构。

控制与测量里常见“环形”。你所在的控制专业以后做数据采集时，会经常用到环形缓冲区：固定大小的数组，写指针每次 $w \leftarrow (w+1) \bmod N$，写满后覆盖最旧的数据。它是用顺序存储模拟“首尾相接”，与循环链表“尾结点指回头部”是同一个想法的两种实现。

原始资料重编码：本讲义依据了哪些材料

下表说明每份原始材料被整理到了讲义的哪个位置。原始材料整体清晰，**没有无法辨认的内容**；有若干处笔误、遗漏或表述不严谨，集中列在下一节“原图纠错”中。

原始材料	主要内容	讲义位置
课件 1 P7–P29	为什么学数据结构；数据、数据元素、数据项；四种逻辑结构；存储结构；基本操作五大类	1.1–1.2
课件 1 P33–P35	抽象数据类型 ADT 的两个特性；顺序表类模板	1.3
课件 1 P37–P54	算法 5 特性与基本要求；时间频度；大 O；求和、求积定理；空间复杂度；逆置示例	1.4–1.6
课件 1 P30–P32, P57–P63	单选与填空练习（逻辑结构、复杂度分析）	练习改编、1.5 例题
课件 1 P64–P83	C++ 复习：面向对象、模板、const、异常处理	正文代码中随处说明
课件 2 P3–P13	线性表定义、逻辑特征、ADT、9 个基本操作；例 2.1 并集、例 2.2 有序归并；单选 1	第 2 章
课件 2 P14–P30	顺序存储结构、地址公式、类模板、插入删除及平均移动次数、“五步口诀”、常见错误、差集、作业	第 3 章
课件 2 P31–P58	链式存储、存储映像、单链表术语与类模板、插入删除“先武装自己再融入队伍”、逆置“摆龙门阵”、循环链表、约瑟夫问题	4.1–4.3
课件 2 P59–P67	双向链表、双向循环链表；带当前位置的链表；单选 3–6	4.4–4.5、练习
笔记 Memo 543–548	线性表定义；多项式与稀疏多项式；补充（数据元素、逻辑/存储结构、ADT、算法、复杂度）；9 个基本操作	第 1–2 章、3.5
笔记 Memo 549–556	地址公式；顺序表优缺点；链式存储示例（赵钱孙李…）；头指针/头结点/首元结点；循环链表与尾指针合并；双向链表；效率比较	第 3–5 章
截图（青岛大学 2.6）	顺序表与链表比较表	第 5 章
本人代码 6 个.cpp	顺序表类模板与多项式合并；单链表、循环链表、双向链表类模板；建表与初始化片段	第 3–4 章全部代码

原图纠错

以下问题按“对后续内容影响大小”排序。前 4 条会直接影响做题结果，请重点看。

原图纠错　①课件 2 P11 例 2.2：“不重复”与归并规则矛盾

原始写法：“将 La 和 Lb 归并为新的线性表 Lc，且 Lc 中的数据元素**不重复**……”；随后给出规则“当 $a \le b$ 时 $c = a$，当 $a > b$ 时 $c = b$”，示例 La=(2,5,8,11)，Lb=(2,3,9,11,15,20)，Lc=(2,3,5,8,9,11,15,20)。

存在的问题：按“$a \le b$ 取 a、只移动 i”的规则，2 与 11 都会出现两次，得到 (2,2,3,5,8,9,11,11,15,20)，与“不重复”和示例结果不符。

规范写法：要么去掉“不重复”（经典的严格写法，保留重复）；要么把规则改为三分支：$a < b$ 取 a；$a > b$ 取 b；$a = b$ 只取一个并让 i、j 同时后移。

为什么：有序归并只看“谁小先放谁”，相等时放一个还是两个，必须单独规定。

影响：3.5 节给出两种版本的 C++ 代码并都运行验证；练习中凡涉及归并都会注明规则。

原图纠错　②课件 1 P47：求和下标与代码不一致

原始写法：程序 `for(i=0;i<n;i++) for(j=0;j<i;j++)`，循环体次数写成 $\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}1=\sum_{i=0}^{n-1}(n-i)=\frac{n(n+1)}{2}$。

存在的问题：内层是 `j<i`，即 $j=0,1,\dots,i-1$，应为 $\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}1=\sum_{i=0}^{n-1}i=\frac{n(n-1)}{2}$。用计数程序验证：$n=10$ 时实际执行 45 次，而 $\frac{n(n+1)}{2}=55$。

规范写法：$\frac{n(n-1)}{2}$。　**影响：**最高次项仍为 $\frac{1}{2}n^2$，结论 $O(n^2)$ 不变，但频度数值必须写对。

原图纠错　③笔记 Memo 546：`i=i*2` 循环的推理前后矛盾

原始写法：“设经历 x 次后大于 n：$2^x>n$，$x>\log_2 n$，$\therefore x\le\log_2 n$”。

存在的问题：由 $x>\log_2 n$ 推不出 $x\le\log_2 n$。

规范写法：循环体执行时 i 依次为 $1,2,4,\dots,2^{k-1}$，要求 $2^{k-1}\le n<2^k$，所以执行次数 $k=\lfloor\log_2 n\rfloor+1$，$T(n)=O(\log_2 n)$。详见 1.5 节例 1.4。　**影响：**结论 $O(\log_2 n)$ 正确，只是推理需要改写。

原图纠错　④笔记 Memo 554：尾指针循环链表合并漏了一步

原始写法：`p = Ta->next; Ta->next = Tb->next->next; delete Tb->next;`

存在的问题：B 的表尾结点仍指向已被释放的 B 头结点，环没有闭合，且出现“悬空指针”。

规范写法：再加一句 `Tb->next = p;`，并且合并后新表的尾指针是 `Tb`。　**影响：**见图 4.13（例 4.1）。

原图纠错　⑤本人代码「线性表的创建.cpp」：先读后存的顺序错了

原始写法：循环外 `cin >> x;`，循环内先判满，再 `cin >> x; L.elem[L.length] = x;`，且 `L.length` 从不增加，函数缺少右括号。

存在的问题：第一个输入值被第二次 `cin` 覆盖而丢失；结束标志 -1 反而被存进表里；表长始终为 0，每次都写到同一位置。

规范写法：“读一个→判断是否为 -1 →判满→存入并 `length++`”，见代码 3.2。　**影响：**建表结果完全错误，必须修正。

原图纠错　⑥本人代码「线性表的初始化.cpp」：`if(!L.elem)` 对普通 new 无效

问题：C++ 中普通 `new int[MAXSIZE]` 申请失败时**抛出** `std::bad_alloc` 异常，不会返回空指针，所以 `if(!L.elem)` 永远不会成立。

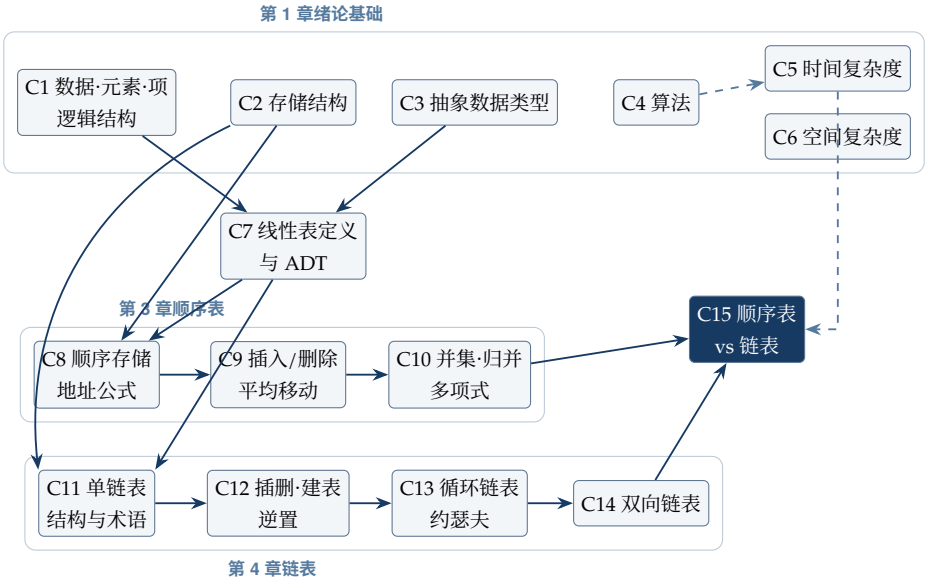
规范写法：用 `new (std::nothrow) int[MAXSIZE]`（需 `#include <new>`）才会在失败时返回 `nullptr`；或用 `try{...} catch(const std::bad_alloc&){...}`。这与课件 1 P81–P83 讲的异常处理正好对应。　**影响：**平时不影响运行，但“判空”这句是无效代码。

原图纠错　⑦其他笔误与表述不严谨（不影响结论）

- 笔记 Memo 552 结点定义 `ElemType date;` 应为 `data`；Memo 547 `DestoryList` 应为 `DestroyList`；课件 2 P6 `LinerList` 应为 `LinearList`；课件 2 P28 `Dispaly` 应为 `Display`。
- 课件 1 P19 称“数据元素是数据处理的最小单位”，同页又称“数据项是不可分割的最小单位”，两句冲突。标准说法：**数据元素是数据的基本单位，数据项是不可分割的最小单位**（你的笔记 Memo 544 写“基本单位”是对的）。
- 课件 1 P44 用“$T(n)/f(n)$ 的极限为非零常数”定义 $O(f(n))$，同页又给出“$\exists C,N_0$，$n>N_0$ 时 $T(n)\le Cf(n)$”。后者才是大 O 的定义；前者是更强的“同阶”条件（见 1.5 节对照）。

- 笔记 Memo 555 写“双向链表插入/删除需同时修改两个方向指针，$T(n) = O(n)$”。修改指针本身只需常数步，即 $O(1)$；$O(n)$ 来自“按位序找到位置”。笔记 Memo 553 写“插入与删除 $T(n) = O(1)$”也应补上前提“已知前驱结点的指针”。

知识点总览与关系图



15 个核心知识点及其关系。箭头表示“学后者要用到前者”；虚线表示“复杂度贯穿全书，用来评价各种实现”。

第 1 章　绪论基础：数据结构、ADT 与算法复杂度

本章回答三个问题：①什么叫“数据结构”，它研究什么？②抽象数据类型（ADT）为什么能把“表是什么”和“表怎么存”分开？③怎样不运行程序就判断一个算法快不快、占不占内存？

本章知识路线：数据→数据元素/数据项→逻辑结构→存储结构→ ADT →算法→时间复杂度→空间复杂度。

线性表是我们学的第一个数据结构。在研究它之前，需要一套“通用语言”：用什么词描述数据之间的关系，用什么尺子衡量操作的快慢。这些词来自第一章绪论，你的笔记 Memo 544–547 把它们作为“§3 补充”记了下来。本章把它们讲完整。

1.1　数据、数据元素、数据项与逻辑结构

> **为什么需要它**　计算机早期只算数（数值计算），后来要处理学生表、通讯录、文件目录、交通网这类**有结构**的数据（课件 1 P7–P17）。处理它们之前，必须先回答：数据由什么组成？组成部分之间是什么关系？

先看一张图。下面这张学生信息表来自课件 2 P3，我们用它来对应四个名词。

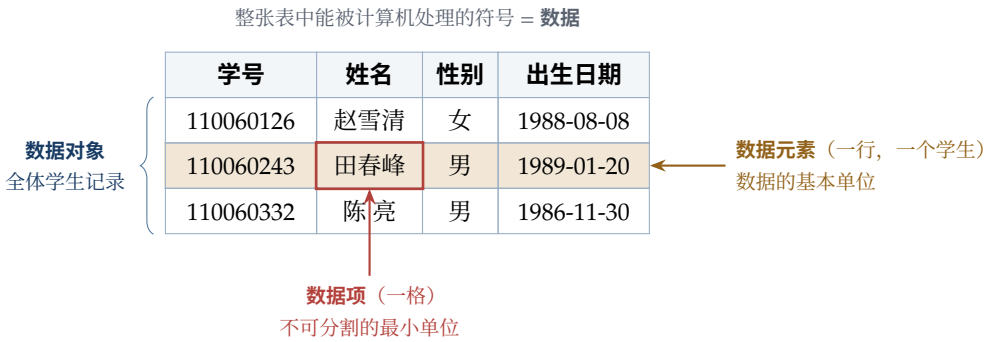


图 1.1: 用一张学生表认识四个名词：表→行→格。

定义（数据的四个层次）

- **数据**：描述客观事物的数、字符以及所有能输入计算机并被程序处理的符号的集合。分数值性数据与非数值性数据。
- **数据元素**：数据的**基本单位**，在程序中通常作为一个整体处理，例如表中的一行。
- **数据项**：组成数据元素的、**不可分割的最小单位**，例如“姓名”“性别”。
- **数据对象**：**性质相同**的数据元素的集合，是数据的一个子集，例如“全体学生记录”。

光有元素还不够：同样是 6 个学生，排成一队、组成一个家族树、互相加好友形成社交网络，是完全不同的数据。于是需要描述**元素之间的关系**。

定义（数据结构与逻辑结构）

数据结构：相互之间存在一种或多种特定关系的数据元素的集合，常写成二元组

$$\text{Data_Structure} = (D,\ R),$$

其中 D 是数据元素的有限集合，R 是 D 上关系的有限集合。关系用有序偶 $\langle u, v\rangle$ 表示：u 是 v 的**直接前驱**，v 是 u 的**直接后继**。

逻辑结构：只描述元素之间的关系，与数据元素本身的形式、内容、相对位置、个数无关，与计算机无关。

例如整数 $1 \sim 10$ 组成的有序集（课件 1 P24）：

$$D = \{x \mid 1 \le x \le 10,\ x \in \mathbb{N}\}, \qquad R = \{\langle x_i, x_{i+1}\rangle \mid x_i, x_{i+1} \in D,\ i = 1, \ldots, 9\}.$$

R 中的每一对都说“x_i 后面紧跟着 x_{i+1}”，这就是线性关系。按 R 的形状，逻辑结构分为四类：

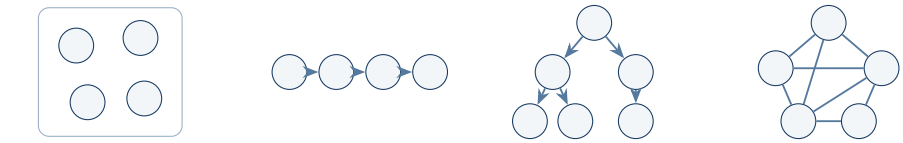


图 1.2: 四种基本逻辑结构。线性表属于 (b)：除首尾外，每个元素恰有一个前驱、一个后继。

你的笔记 Memo 544 记录了两种划分方法，它们是同一件事的粗分与细分：

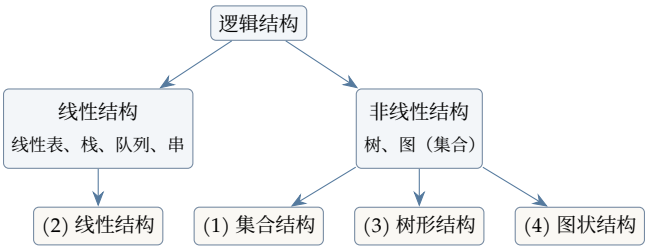


图 1.3: 划分方法一（线性/非线性）与划分方法二（四类）的对应。集合结构元素间没有制约关系，通常归入非线性一侧讨论。

例题 1.1　判断逻辑结构

题：下列数据各是什么逻辑结构？(1) 26 个英文字母表 $(A, B, \ldots, Z)$；(2) 电脑的文件夹目录；(3) 城市之间的燃气管网；(4) 一个班全体同学（不考虑任何次序）。

思路：只看“一个元素有几个前驱、几个后继”。

解：(1) 除 A 外每个字母恰有一个前驱，除 Z 外恰有一个后继→线性；(2) 一个文件夹可含多个子文件夹，但只属于一个父文件夹→树形；(3) 一个小区可连多个小区，且可成环→图状；(4) 元素间无任何关系→集合。

本题想说明：判断逻辑结构不看元素是什么（字母、文件、小区），只看关系的“对数”。这正是定义里“与元素的形式、内容无关”的含义。

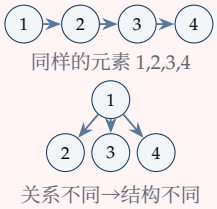
易错提醒

错误想法 1：“数据元素就是数据项。”

错在哪：一个元素（一行）包含多个项（多格）。元素是处理的基本单位，项是不可再分的最小单位。

错误想法 2：“表里的人按学号排列，所以学号决定了逻辑结构。”

错在哪：逻辑结构由关系集合 R 决定。把同样 6 个人放进家族树，就是树形结构；元素内容没变，关系变了。



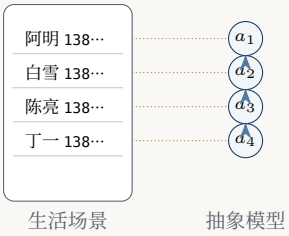
生活中的它　手机通讯录

场景：打开手机通讯录，按拼音排好一串联系人。

对应：整本通讯录 = 数据对象；一个联系人 = 数据元素；姓名、电话、备注 = 数据项；“按拼音一个接一个” = 线性关系 R。

小判断：给“张三”加一个“家庭成员”分组，并让分组下挂多个人，这部分关系就变成了树形（一对多）。

边界：真实通讯录还能多维排序、搜索，用到的结构远不止一张线性表。



对照：数据元素 / 数据项 ↔ 数据库中的记录 / 字段（类比同构型）

一句话定位：数据结构里的“元素—项”与以后《数据库原理》里的“记录（元组）—字段（属性）”是同一种“整体—组成部分”关系，但数据库更关心“集合”，数据结构更关心“关系形状”。

维度	数据结构	关系数据库
整体	数据对象	关系（表）
个体	数据元素	记录（元组）
组成	数据项	字段（属性）
元素间次序	线性表有前驱后继	理论上行无次序

旧结论	迁移	理由
元素由若干项组成	✓	记录由若干字段组成
同一对象中元素类型相同	✓	同一表中每行字段结构相同
线性表元素有先后次序	✗	关系是元组的**集合**，行的先后没有意义



学完这里，你应该能够回答

1. 数据元素与数据项哪个是“基本单位”，哪个是“最小单位”？
2. 逻辑结构为什么说“与计算机无关”？
3. 文件夹目录、地铁线路网、一个人的待办清单分别是什么逻辑结构？

1.2 存储结构：把关系“放进”内存

逻辑结构解决了“关系是什么”，但计算机内存只是一长串按地址编号的单元，没有“前驱”“后继”这种东西。于是下一个问题是：**怎样在内存里既存下元素的值，又存下元素之间的关系？**

定义（存储结构 / 物理结构）

存储结构：数据结构在计算机内存中的表示（映像），包括元素的值和元素之间关系的表示。常见四种：

1. **顺序存储**：用一块**地址连续**的空间依次存放元素，借助“地址上的先后”表示“逻辑上的先后”。
2. **链式存储**：每个元素存放在各自独立的空间中，空间可不连续；在每个元素旁**附加指针**记录相关元素的地址，用指针表示关系。
3. **索引存储**：除元素本身外，另建一张索引表（关键字→地址）。拓展联想
4. **散列存储**：由元素的关键字直接计算出存放地址。拓展联想

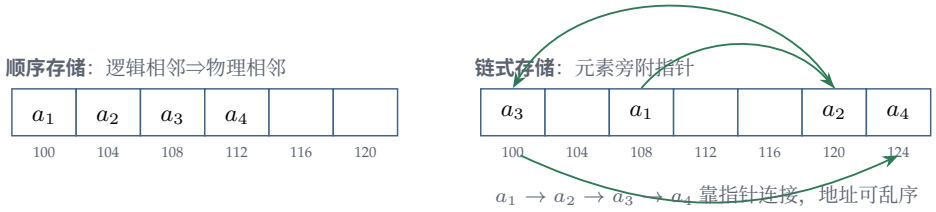


图 1.4: 同一个线性表 (a_1, a_2, a_3, a_4) 的两种存储方式。灰色小字是内存地址（每单元 4 字节）。

图中左边，a_2 就存在 a_1 的下一个单元，“后继”靠**位置**表达；右边，a_1 存在 108，它的后继 a_2 存在 120，“后继”靠**指针**（绿色箭头）表达。课件 1 P27–P29 给出了一个贯穿全课程的原则：

定理 / 结论（逻辑与存储的分工）

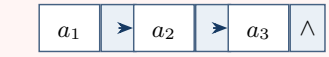
算法的设计取决于逻辑结构，算法的实现依赖于存储结构。 同一种基本操作（如“在第 i 个位置插入”）在顺序存储和链式存储下写法完全不同，但目标都必须符合 ADT 规定的初始条件与操作结果。如果某种存储方式下基本操作很难实现，说明这种存储方式不适合，可以换掉。

无论哪种逻辑结构，基本操作都可分五大类（课件 1 P25–P26）。线性表的 9 个基本操作（第 2 章）正好可以按此归类：

类别	含义	线性表中的例子
构造类	在内存中建立结构	`InitList`、构造函数
属性类	查询，不改变元素与关系（“东瞧瞧、西看看”）	`Length`、`Empty`、`GetElem`、`LocateElem`
数据操纵类	改变元素或关系	`Insert`、`Delete`、`SetElem`、`Clear`
遍历类	每个元素访问且只访问一次	`Traverse`
典型应用类	这种结构特有的应用	多项式相加、约瑟夫问题

易错提醒

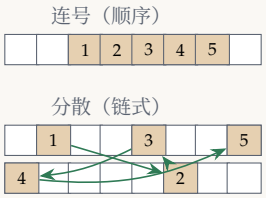
错误想法：“链式存储的元素一定不连续。”
错在哪：链式存储**不要求**连续，但碰巧连续也完全可以（课件 2 P64 单选 3 的答案正是“连续或不连续都可以”）。区分两种存储，看的是**关系靠什么表示**：靠位置还是靠指针。



地址恰好连续，仍是链式（靠指针）

生活中的它　电影院选座

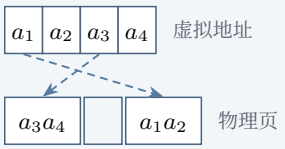
场景：五个同学去看电影。买到连号座位 7 排 5–9 号，就是顺序存储：找第 3 个同学，直接去 7 排 7 号。没买到连号，各自分散坐，每人手机里记下“下一位坐在哪里”，就是链式存储。
对应：座位号 = 地址；人 = 元素；“下一位座位号” = 指针。
小判断：中途又来一位同学要坐在第 2、3 人之间。连号时，后面 3 人都得往右挪；分散坐时，只要改第 2 人手机里的“下一位”。
边界：真实影院座位有限且固定，内存则可动态申请。



对照：逻辑结构 / 存储结构 ↔ 操作系统中的逻辑地址 / 物理地址（同名异义型） 拓展联想

一句话定位：两门课都用“逻辑—物理”分层，但对象不同：数据结构的“逻辑”指**元素间的关系**，操作系统的“逻辑地址”指**程序看到的地址编号**。

数据结构中的说法	迁移	理由或反例
上层只关心逻辑，不关心下层怎么存	✓	两门课的分层思想相同
顺序表逻辑相邻⇒物理相邻	△	在程序的**虚拟地址**中连续；操作系统分页后，实际物理内存页可以不连续
“逻辑”指元素间关系	✗	操作系统的“逻辑地址”是一个数，不是关系



学完这里，你应该能够回答

1. 顺序存储和链式存储分别用什么表示“后继”关系？
2. “算法设计取决于____，算法实现依赖于____。”
3. 判断：链式存储结构中的结点地址一定不连续。（错，为什么？）

1.3 抽象数据类型（ADT）

为什么需要它　线性表有两种实现。如果使用者（写 `main` 的人）必须知道表是数组还是链表才能写代码，那么一换实现，所有调用代码都要改。ADT 的想法是：**只对外公开“能做什么”，把“怎么做”藏起来。**

定义（抽象数据类型 ADT）

抽象数据类型：一个数学模型以及定义在此模型上的一组操作，形式上可写成三元组

$$\text{ADT} = (D,\ S,\ P),$$

D：数据对象；S：D 上的关系集（课件中也记作 R）；P：对 D 的基本操作集。
两个特性：**抽象性**——只描述本质特征、功能和对外接口；**封装性**——把外部特性与内部实现细节分离，并对用户隐藏内部细节。

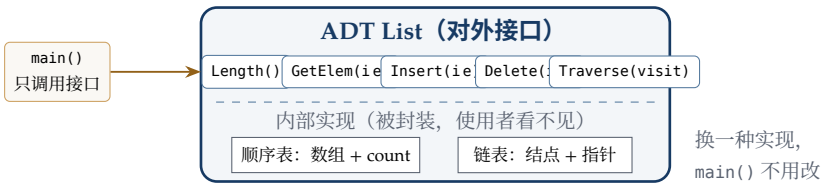


图 1.5: ADT 的“胶囊”模型：接口在外，实现在内；同一接口可以有两种实现。

你的笔记 Memo 545 以复数为例。用 ADT 的格式写出来，再用 C++ 类实现：

```
ADT Complex {
```
数据对象：$D = \{r_1, r_2 \mid r_1, r_2 \text{ 均为实数}\}$
数据关系：$S = \{\langle r_1, r_2\rangle \mid r_1 \text{ 是实部，} r_2 \text{ 是虚部}\}$
基本操作：`Assign(&C, v1, v2)`　初始条件：空复数 C 已存在；操作结果：构造复数 C，r_1, r_2 分别被赋以 v_1, v_2 的值。　……
```
} ADT Complex
```

代码 1.1: 用 C++ 类实现 ADT Complex：数据成员 private（封装），操作 public（接口）

```cpp
class Complex {
private:                        // 内部实现：使用者不能直接访问
```

```
    double r1, r2;              // 实部、虚部
public:                         // 对外接口
    Complex() : r1(0), r2(0) {}                          // 构造一个“空复数”
    void Assign(double v1, double v2) { r1 = v1; r2 = v2; }  // ADT 中的 Assign(&C, v1, v2)
    double Real() const { return r1; }                   // const：只读，不改对象
    double Imag() const { return r2; }
    Complex Add(const Complex &b) const {                // 引用传参，不复制；const 保护实参
        Complex c; c.Assign(r1 + b.r1, r2 + b.r2); return c;
    }
};
```

ADT 写法中的 `&C` 表示“这个参数会被修改”；在 C++ 成员函数里，被修改的就是调用它的对象本身（`c.Assign(1,2)` 修改 `c`），所以不必再写这个参数。课件 1 P76–P80 讲的两种 `const`：参数前的 `const T &x` 保护实参不被改，成员函数末尾的 `const` 保护调用它的对象不被改——后面线性表的 `Length() const`、`GetElem(...) const` 都是这个意思。

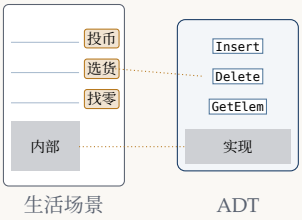
生活中的它　自动售货机

场景：售货机正面只有“投币、选货、找零”几个按钮，你不需要知道里面是弹簧推货还是机械臂取货。

对应：按钮 = 基本操作 P；货道里的商品 = 数据对象 D；货道编号与商品的对应 = 关系 S；机器内部结构 = 被封装的实现。

小判断：厂家把弹簧换成机械臂，顾客的操作步骤变不变？不变——这就是“换实现，接口不变”。

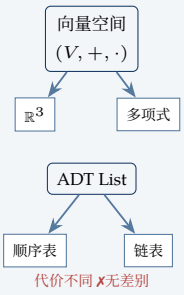
边界：售货机按钮功能固定；软件的 ADT 还要规定每个操作的初始条件和结果。



对照：ADT ↔ 线性代数中的“向量空间”（类比同构型）

一句话定位：向量空间由“集合 + 加法、数乘 + 运算律”刻画，不规定向量必须写成坐标；ADT 由“数据对象 + 关系 + 操作”刻画，不规定必须用数组还是链表。

向量空间中的结论	迁移	理由或反例
只用“允许的运算”刻画对象	✓	ADT 只用基本操作刻画线性表
同一抽象有多种具体模型（$\mathbb{R}^n$、多项式空间）	✓	同一 ADT 有顺序表、链表等实现
运算律用公理精确规定	△	本课程用“初始条件/操作结果”文字说明，不写公理
不同模型“本质一样”，无优劣之分	✗	ADT 的不同实现**代价不同**：`GetElem` 顺序表 $O(1)$，链表 $O(n)$



1.4 算法及其特性

定义（算法）

算法：解决某一具体问题的方法和步骤，是指令的有限序列。它必须具有五个特性：

- **有穷性**　每一步在有限时间内完成，整个算法在有限步之后结束
- **确定性**　每一步有确定的含义，没有二义性；相同输入得到相同输出
- **可行性**　每一步都能通过有限次基本操作实现
- **输入**　有零个或多个输入
- **输出**　有一个或多个输出（课件写“零个或若干个”，标准教材一般要求至少一个输出）

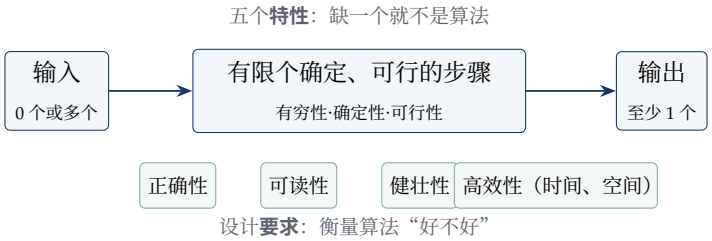


图 1.6: 五个特性回答“是不是算法”；四项设计要求回答“是不是好算法”。

描述算法可以用自然语言、流程图、伪代码或程序设计语言（笔记 Memo 545）。本讲义统一用 C++。

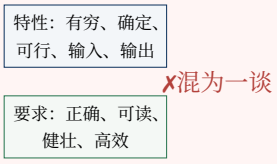
正例与反例。“在 n 个数中找最大值：逐个比较，记下当前最大”是算法。下面的代码**不是**算法，因为它违背有穷性：

```
int i = 1;
while (i > 0) i++;   // 理论上永不结束（实际会整数溢出，属于未定义行为），不满足有穷性
```

易错提醒

错误想法：“可读性、健壮性是算法的特性。”

错在哪：它们属于**设计要求**。一个难读、遇到非法输入就崩溃的程序，只要满足五特性，仍然是算法，只是不好。课件 1 P38 的“基本要求”与 P37 的“特性”要分开记。



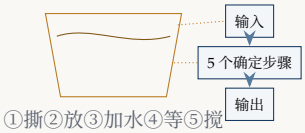
生活中的它　泡一碗方便面

场景：包装上写着：①撕开盖子到虚线处；②放入料包；③加 500 mL 开水；④盖盖 3 分钟；⑤搅拌食用。

对应：面饼、料包、开水 = 输入；一碗面 = 输出；5 步有限 = 有穷性；“500 mL、3 分钟” = 确定性；每步都能做到 = 可行性。

小判断：若第③步写成“加适量水”，违背哪条？——确定性（“适量”有二义）。

边界：生活步骤允许模糊，程序不允许。



对照：算法 ↔ 程序（同名异义型，程序设计 / 操作系统）

一句话定位：程序是算法用某种语言的具体实现，但“程序”不一定满足算法的五特性。

算法的性质	迁移到“程序”	理由或反例
有输入、有输出、每步可行	✓	程序同样如此
有穷性	✗	操作系统、服务器主循环被设计成一直运行，它们是程序，但不是算法
与具体语言无关	✗	程序必须用某种语言写成，并受编译器、机器影响
确定性	△	多线程或调用随机数的程序，同一输入可能输出不同；“随机化算法”需另行定义



图 1.7: 算法必须终止；程序可以被设计成一直运行。

1.5 时间复杂度

为什么需要它　同一问题有多种算法，怎样比较快慢？**事后统计**：写好程序、用同一组数据计时——但受机器主频、编译器、语言、数据分布影响（课件 1 P41）。**事前分析**：不运行，只数“基本语句执行了多少次”，并看它随问题规模 n 怎样增长。本节学的是后者。

1.5.1 时间频度：数一数语句执行了多少次

把一条语句的执行次数称为**频度**；所有语句频度之和记作 $T(n)$，叫做**时间频度**。看课件 1 P42 与笔记 Memo 546 的例子：

```
s = 0;                                  ……  1 次
for (int i = 0;   i < n;   i++)         ……  i=0:1  i<n:n+1  i++:n
    s = s + i;                          ……  n 次
                  注意：i<n 比循环体多判断 1 次（最后一次为假）
```

$$T(n) = 1 + 1 + (n+1) + n + n = 3n + 3$$

图 1.8: 逐条标出频度，求和得到时间频度 $T(n)$。

数到这里你会发现一个问题：$3n+3$ 里的“$+3$”、系数“3”都和机器、写法有关，而且 n 很大时它们几乎不起作用。我们真正关心的是**增长的趋势**：n 翻倍，时间大约翻倍。这就引出了大 O。

1.5.2 大 O 记号：只看增长的阶

定义（渐近时间复杂度，大 O 表示法）

若存在正常数 C 和正整数 N_0，使得当 $n > N_0$ 时恒有

$$T(n) \le C \cdot f(n)$$

则记 $T(n) = O\big(f(n)\big)$，称 $O(f(n))$ 为算法的**渐近时间复杂度**，简称**时间复杂度**。

逐个符号： n 是问题规模（表长、元素个数）；$T(n)$ 是基本语句总频度；$f(n)$ 是一个“标尺函数”，如 n、n^2、$\log_2 n$；C 是允许乘上的常数倍；N_0 表示“只要求从某个规模之后成立”，小规模时怎样都行。

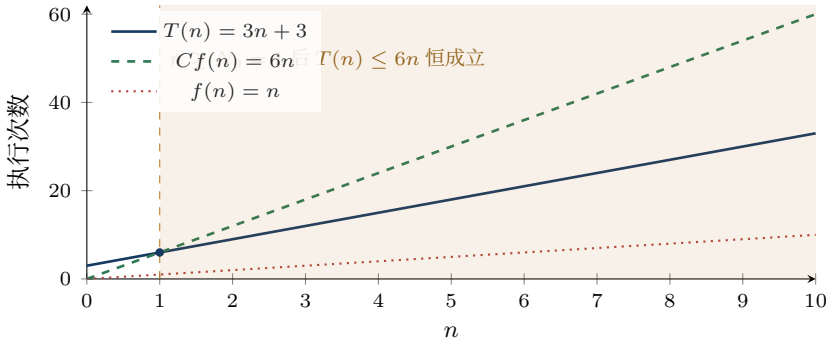


图 1.9: 大 O 的几何意义：从 N_0 往右，$T(n)$ 的曲线始终在 $Cf(n)$ 的下方。所以 $3n+3 = O(n)$。

图中取 $C = 6$：$3n + 3 \le 6n \iff 3 \le 3n \iff n \ge 1$。在 $n = 1$ 处两线相交，之后实线永远在虚线下方。注意 $T(n)$ 本身比 $f(n) = n$ 大（红色点线在最下面），但没关系，大 O 允许乘常数 C。

定理 / 结论 1.1（多项式的阶，笔记 Memo 546）

若 $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots + a_1 n + a_0$ 是 m 次多项式（$a_m > 0$），则 $f(n) = O(n^m)$。

证明

要证：存在 C, N_0，使 $n > N_0$ 时 $f(n) \le C n^m$。　**思路：**把每一项都放大成 n^m 的倍数。

当 $n \ge 1$ 时，对 $0 \le k \le m$ 有 $n^k \le n^m$（$n \ge 1$ 时幂次越高越大），于是

$$\begin{aligned} f(n) &\le |a_m| n^m + |a_{m-1}| n^{m-1} + \cdots + |a_0| && \text{（每项取绝对值，只会变大）} \\ &\le |a_m| n^m + |a_{m-1}| n^m + \cdots + |a_0| n^m && (n^k \le n^m) \\ &= \big(|a_m| + |a_{m-1}| + \cdots + |a_0|\big)\, n^m. \end{aligned}$$

取 $C = |a_m| + \cdots + |a_0|$，$N_0 = 1$ 即得证。■

直观：n 很大时 n^m 远远压过低次项。例如 $T(n) = 2n^3 + 3n^2 + 2n + 1$，$n = 100$ 时 $2n^3 = 2\,000\,000$，其余三项合计只有 30 201，约占 1.5%。所以 $T(n) = O(n^3)$——就像高数里“同阶无穷大只看最高次项”。

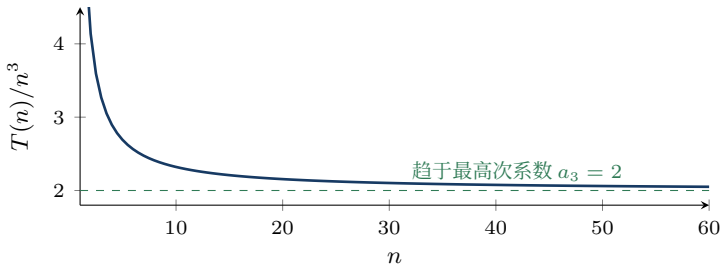


图 1.10: $T(n) = 2n^3 + 3n^2 + 2n + 1$ 与 n^3 的比值迅速趋于常数 2：低次项的影响越来越小。

1.5.3 怎样找“基本语句”，怎样算复杂度

实际分析时不必像图 1.8 那样数每一条语句。笔记 Memo 546 记下了找**基本语句**的三条标准：①执行次数与算法执行时间成正比；②对运行时间贡献最大；③执行次数最多——通常就是**嵌套最深层循环的循环体**。于是：

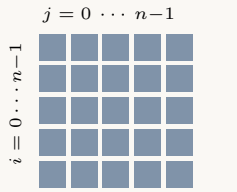
方法：计算时间复杂度的两步（课件 1 P48）

第一步：找出与规模 n 有关的循环，算出最内层循环体的执行次数，得到频度函数；
第二步：只保留关于 n 的最高阶项，去掉系数，就是大 O。若频度与 n 无关（是常数），复杂度为 $O(1)$。

例题 1.2　两重独立循环（课件 1 P46）

```
s = 0;
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        { s = s + i + j; cout << s; }
```

解：基本语句是内层循环体。外层每取一个 i，内层都完整执行 n 次：$\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} 1 = \sum_{i=0}^{n-1} n = n^2$，故 $T(n) = O(n^2)$。



每格 = 执行一次，共 $n \times n$

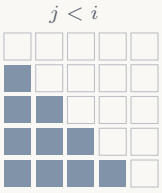
例题 1.3　内层次数依赖外层（课件 1 P47，已更正）

```
for (i = 0; i < n; i++)
    for (j = 0; j < i; j++)
        { s = s + i + j; cout << s; }
```

解：i 取 $0, 1, \ldots, n-1$ 时，内层分别执行 $0, 1, \ldots, n-1$ 次：

$$\sum_{i=0}^{n-1}\sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n,$$

最高阶项 $\frac{1}{2}n^2$，去系数得 $O(n^2)$。
检查：$n = 10$ 时程序实测 45 次，$\frac{10\times 9}{2} = 45$。



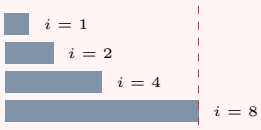
只有阶梯下方执行：约一半

易错：内外层不独立时不能直接“相乘”

例 1.3 中若想当然地说“外层 n 次 × 内层 n 次 $= n^2$”，结论碰巧对，但**频度**错了（实际 $\frac{n(n-1)}{2}$）。更危险的是下面这种情况：

```
for (i = 1; i < n; i *= 2)
    for (j = 0; j < i; j++) s++;
```

“外层 $\log_2 n$ 次 × 内层最多 n 次 $= n\log_2 n$”只是上界，实际是 $1 + 2 + 4 + \cdots < 2n$，即 $O(n)$。**内层次数随外层变化时，要老老实实求和。**



总面积 $1 + 2 + 4 + 8 = 15 < 2n$

例题 1.4　循环变量成倍增长（笔记 Memo 546，已更正推理）

```
i = 1;
while (i <= n)
    i = i * 2;
```

解： 设循环体执行了 k 次。执行时 i 的值依次为 $1, 2, 4, \ldots, 2^{k-1}$，每次都满足 $i \le n$；第 k 次之后 $i = 2^k > n$，循环结束。所以

$$2^{k-1} \le n < 2^k \implies k-1 \le \log_2 n < k$$
$$\implies k = \lfloor \log_2 n \rfloor + 1.$$

故 $T(n) = O(\log_2 n)$。**检查：** $n = 10$：$i = 1, 2, 4, 8$ 共 4 次，$\lfloor \log_2 10 \rfloor + 1 = 3 + 1 = 4$。



每次跳到 2 倍，4 跳越过 n

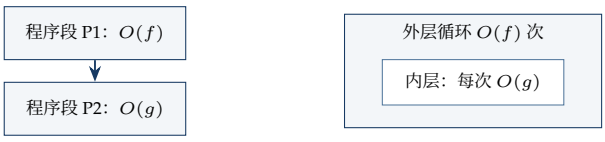
为什么一般不写对数的底？因为换底公式 $\log_a n = \dfrac{\log_b n}{\log_b a}$，不同底只差一个常数倍，而大 O 忽略常数，所以 $O(\log_2 n) = O(\log_3 n) = O(\lg n)$，统一写作 $O(\log n)$（课件 1 P49“认识 $O(\log_2 n)$、$O(\log n)$、$O(\lg n)$”）。

例题 1.5　n 阶矩阵相乘（笔记 Memo 546）

$C = A \times B$, $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$。三重循环，最内层乘加语句执行 $\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n} 1 = n^3$ 次，$T(n) = O(n^3)$。

这印证了笔记中的红字：**时间复杂度通常由嵌套最深层语句的频度决定**。

1.5.4　两条运算规则



先后执行→ **加法**：$O(\max(f, g))$　　嵌套执行→ **乘法**：$O(f \cdot g)$

图 1.11: 加法规则对应“顺序”，乘法规则对应“嵌套”（内层与外层独立时）。

定理 / 结论 1.2（加法规则、乘法规则，课件 1 P50–P51）

若 $T_1(n) = O(f(n))$，$T_2(n) = O(g(n))$，则

$$T_1(n) + T_2(n) = O\big(\max(f(n), g(n))\big), \qquad T_1(n) \times T_2(n) = O\big(f(n) \times g(n)\big).$$

证明

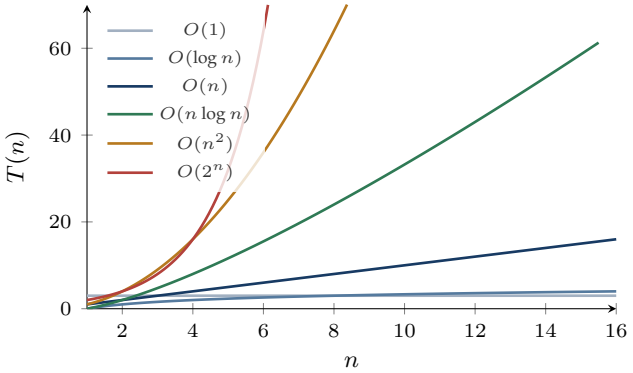
由定义，存在 C_1, N_1 使 $n > N_1$ 时 $T_1 \le C_1 f$；存在 C_2, N_2 使 $n > N_2$ 时 $T_2 \le C_2 g$。取 $N_0 = \max(N_1, N_2)$，则 $n > N_0$ 时两式同时成立。

加法： $T_1 + T_2 \le C_1 f + C_2 g \le (C_1 + C_2)\max(f, g)$，取 $C = C_1 + C_2$。

乘法： $T_1 T_2 \le (C_1 f)(C_2 g) = C_1 C_2\, fg$（此处 T_1, T_2, f, g 均非负），取 $C = C_1 C_2$。■

例： 课件 1 P50 先执行 `for(i..n) a[i]=0;`（$O(n)$），再执行两重循环（$O(n^2)$），总复杂度 $O(\max(n, n^2)) = O(n^2)$。

1.5.5 常见复杂度的比较



	n=10	n=100	n=1000
$\log_2 n$	3.3	6.6	10
n	10	100	1000
$n\log_2 n$	33	664	9966
n^2	100	10^4	10^6
n^3	10^3	10^6	10^9
2^n	1024	$\approx 10^{30}$	$\approx 10^{301}$

图 1.12: 常见复杂度的增长。n 稍大，2^n 就“爆炸”，所以立方阶以上通常称为不实用的算法。

$$O(1) < O(\log_2 n) < O(n) < O(n\log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

从常数阶到平方阶的算法一般称为“易行算法”；指数阶在规模稍大时就无法忍受（课件 1 P49）。

最坏与平均。同一算法对不同输入执行次数可能不同（如在表中查找 x：第一个就找到，或最后才找到）。笔记 Memo 546 的“注”：**一般考虑最坏情况下的时间复杂度**，它保证算法的运行时间不会比这更长；有时也分析等概率下的**平均**情况——顺序表插入删除的 $n/2$ 就是平均值（3.4 节）。

生活中的它　猜数字游戏

场景：朋友心里想一个 1–1000 的整数，你每猜一次他说“大了/小了/对了”。

对应：一个一个猜 = $O(n)$，最坏 1000 次；每次猜中间值 = 每次排除一半 = 例 1.4 中 i 翻倍的反过程，$O(\log_2 n)$。

小估算：$2^9 = 512 < 1000 \le 2^{10} = 1024$，所以“猜中间”最多 10 次就够。

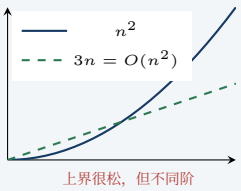
边界：能“猜中间”的前提是候选数有序；线性表无序时只能一个个找。



对照：大 O ↔ 高等数学中“无穷大的阶”（同名异义型）

一句话定位：你的笔记写“类比：同阶无穷小”。更准确的类比是**无穷大**的比较（$n \to \infty$ 时 $T(n) \to \infty$）；而且大 O 只是**上界**，不等于“同阶”。

高数中的结论	迁移	理由或反例
$\lim \frac{T(n)}{f(n)} = c \neq 0$（同阶）$\Rightarrow$ $T = O(f)$	✓	极限存在则 n 足够大时 $\frac{T}{f} < 2c$，取 $C = 2c$
多项式只看最高次项	✓	定理 1.1
$T = O(f) \Rightarrow T$ 与 f 同阶	✗	$3n = O(n^2)$，但 $\lim \frac{3n}{n^2} = 0$，不同阶
自变量连续趋于 x_0	△	这里 n 是正整数且只趋于 $+\infty$



学完这里，你应该能够回答

1. 写出大 O 的定义，并解释 C、N_0 的作用。
2. $T(n) = 3n^2 + 5n\log_2 n + 100$ 的时间复杂度是多少？为什么 log 的底可以不写？

3. 什么时候可以直接用乘法规则？什么时候必须求和？
4. `for(i=1;i<=n;i*=3)` 的循环体执行多少次？

1.6　空间复杂度

时间不是唯一的代价。算法运行时占用的空间包括三部分（课件 1 P52）：程序本身的代码、待处理的输入数据、处理过程中额外需要的**辅助空间**。前两部分不可避免，设计时主要关注第三部分。

定义（渐近空间复杂度）

当问题规模 $n \to \infty$ 时，算法所需**辅助空间**的量级，记作 $S(n) = O(f(n))$。若辅助空间与 n 无关，记为 $O(1)$，称为**原地**（就地）工作。

课件 1 P53–P54 与笔记 Memo 547 用同一个任务比较了两种写法：把数组 a 中 n 个数逆序存放。

代码 1.2: 原地逆置：$S(n) = O(1)$

```
for (int i = 0; i < n / 2; i++) {
    int t = a[i];          // 只借用一个 t
    a[i] = a[n - i - 1];
    a[n - i - 1] = t;
}
```

代码 1.3: 借助数组 b：$S(n) = O(n)$

```
int *b = new int[n];       // 额外 n 个单元
for (int i = 0; i < n; i++) b[i] = a[n - i - 1];
for (int i = 0; i < n; i++) a[i] = b[i];
delete[] b;
```

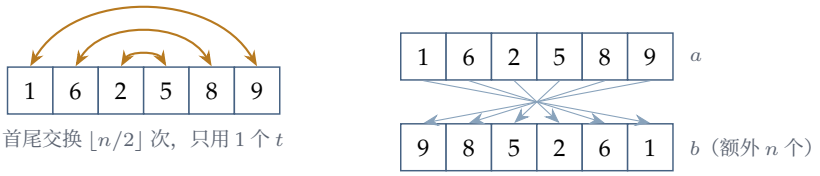


图 1.13: 左：原地交换；右：借用辅助数组。两者时间都是 $O(n)$，空间分别是 $O(1)$ 与 $O(n)$。

课件 1 P54 的说明值得记住：内存足够时，通常更注重时间效率；只有在时间复杂度相同时，才比较空间复杂度的优劣。后面顺序表的插入、删除都只用一个循环变量，空间复杂度是 $O(1)$（笔记 Memo 550）。

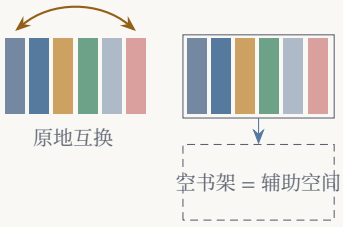
生活中的它　整理书架

场景： 书架上 30 本书要倒过来排。办法一：两手各拿首尾一本互换，一路换到中间；办法二：先把书全搬到地上的空书架，再按倒序搬回来。

对应： 书 = 数组元素；手 = 临时变量 t；空书架 = 辅助数组 b。

小估算： 办法一交换 15 次、只占双手；办法二搬 60 次、多占一个书架——时间同阶，空间 $O(1)$ 对 $O(n)$。

边界： 现实中“搬到地上”可能更省力，计算机中额外空间要申请与释放。



对照：空间复杂度 ↔ 时间复杂度（方法互补型，同课程跨专题）

一句话定位： 两者用同一套大 O 记号与运算规则，但**时间会累加，空间可以重复使用**。

时间复杂度中的结论	迁移到空间	理由或反例
大 O 定义、只看最高阶	✓	定义形式完全相同
顺序执行用加法规则	✓	两段各用的辅助空间取较大者
嵌套循环用乘法规则	✗	循环 n 次、每次用同一个局部变量 t：时间 $O(n)$，空间仍 $O(1)$——空间被**反复使用**，不累加
一般分析最坏情况	✓	空间同样按最坏情况估计



图 1.14: 时间累加、空间复用：循环 n 次的原地逆置，时间 $O(n)$，空间 $O(1)$。

本章回顾：到这里，我们已经解决了什么？

我们有了描述数据的词汇（元素、项、对象、逻辑结构、存储结构），有了把“是什么”与“怎么存”分开的工具（ADT），还有了衡量好坏的尺子（时间、空间复杂度）。**下一步还需要解决什么？**——把这些工具用在第一个具体的数据结构上：给“一个挨一个”的线性表写出严格定义和 ADT（第 2 章），再分别用顺序存储和链式存储实现它，并用复杂度比较它们（第 3–5 章）。

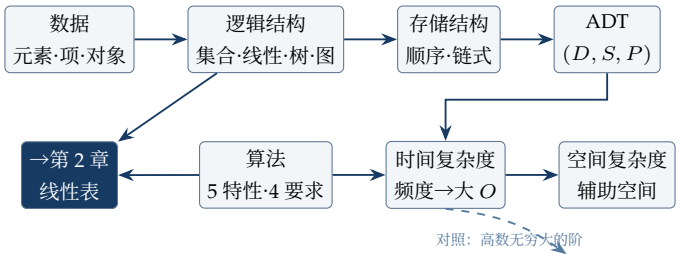


图 1.15: 第 1 章知识地图。虚线指向其他课程中的相似知识。

第 2 章　线性表的逻辑结构与抽象数据类型

上一章解决了什么：我们有了“逻辑结构”“ADT”“复杂度”这些工具。**本章回答三个问题：**①线性表的严格定义是什么，它有哪些逻辑特征？②它的 ADT 规定了哪些基本操作，每个操作的前提和结果是什么？③只用这些基本操作，能解决哪些问题（并集、有序归并、去重）？

本章知识路线：实例→定义与术语→逻辑特征→ ADT（数据对象、数据关系、9 个基本操作）→用基本操作写算法。

2.1　线性表的定义与逻辑特征

为什么需要它　26 个英文字母 $(A, B, \ldots, Z)$、一张学生信息表、一个多项式按指数排好的各项（课件 2 P3，笔记 Memo 543）——内容千差万别，却有共同的形状：**排成一列，每个元素前后最多各有一个邻居**。把这个共同形状抽出来，只研究一次，所有这类问题都能用。

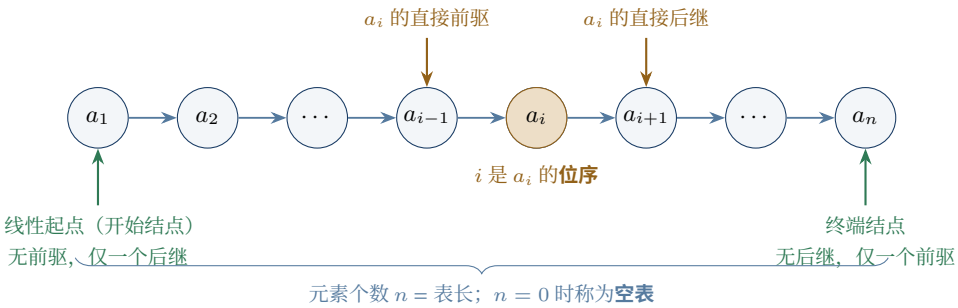


图 2.1: 线性表 $L = (a_1, \ldots, a_n)$ 的标注图（对应笔记 Memo 543 红笔部分）。

定义 （线性表，课件 2 P4）

线性表是由 n $(n \geq 0)$ 个**具有相同类型**的数据元素 $a_1, a_2, \ldots, a_n$ 组成的**有限序列**。元素个数 n 称为表的**长度**；$n = 0$ 时称为**空表**。非空线性表记作

$$L = (a_1, a_2, \ldots, a_{i-1}, a_i, a_{i+1}, \ldots, a_n),$$

其中 a_i 是第 i 个数据元素，i 称为 a_i 在表中的**位序**。a_i 只是一个抽象符号，具体含义随问题而定：可以是一个字母、一个整数，也可以是一整条学生记录。

定义里每个词都有用：“有限”——表长 n 是一个确定的有限数；“序列”——元素有先后次序，(a, b) 与 (b, a) 是不同的表，而且允许重复元素；“相同类型”——表中元素属于同一个数据对象，这样基本操作才能对每个元素一视同仁（这正是 C++ 用类模板 `SqList<ElemType>` 实现的原因）。

定理 / 结论 （非空线性表的逻辑特征，课件 2 P5）

1. 有且仅有一个开始结点 a_1，它没有直接前驱，仅有一个直接后继 a_2；
2. 有且仅有一个终端结点 a_n，它没有直接后继，仅有一个直接前驱 a_{n-1}；
3. 其余内部结点 a_i $(2 \leq i \leq n-1)$ 都有且仅有一个直接前驱 a_{i-1} 和一个直接后继 a_{i+1}。

用第 1 章的语言：线性表的关系集 $R = \{\langle a_{i-1}, a_i \rangle \mid i = 2, 3, \ldots, n\}$，是“一对一”的线性结构。

正例与反例。

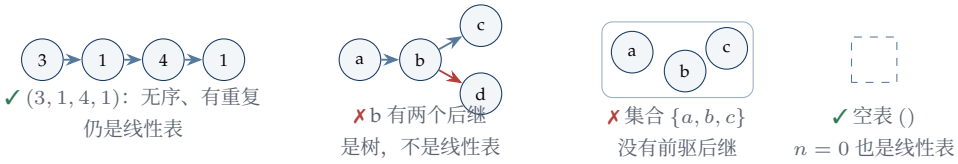


图 2.2: 判断是不是线性表：只看是否满足三条逻辑特征，不看元素是否有序、是否重复。

易错提醒

错误想法 1：“线性表中的元素必须从小到大排列。”——**错。**“序列”指**位置上**有先后，不是**数值上**有大小。有序表只是线性表的一种特殊情况（课件 2 P12 单选 1 的选项 C）。

错误想法 2：“每个元素都有一个前驱和一个后继。”——**错。**a_1 无前驱，a_n 无后继；空表连元素都没有（选项 A、B 均错，正确的是 D）。



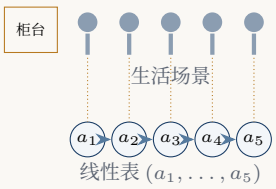
生活中的它　奶茶店排队

场景：奶茶店门口排着 6 个人。

对应：每个人 = 数据元素；“我前面是谁、后面是谁” = 直接前驱/后继；队首（正在点单）= a_1；队尾 = a_n；队伍人数 = 表长 n；店里没人 = 空表；“我排第几个” = 位序 i。

小判断：第 4 个人临时离开，队伍还是线性表吗？是，只是表长变成 5，原来的第 5 人的位序变成 4——这就是后面“删除”操作的效果。

边界：现实中有人“插队”“帮人占位”，线性表只允许通过规定的操作改变。



对照：线性表 ↔ 高等数学中的数列、离散数学中的集合（推广型 / 同名异义型）

一句话定位：线性表像“有限的、元素可以是任意同类对象的数列”，而不是集合。

维度	数列 $\{a_n\}$	集合 $\{a, b, c\}$	线性表 $(a_1, \ldots, a_n)$
元素个数	通常无限	有限或无限	**有限**，可变
次序	有，按下标	**无**	有，按位序
重复	允许	**不允许**	允许
元素类型	数	任意	任意，但**同一类型**
关注点	极限、通项	属于关系	插入、删除、查找的代价

旧结论	迁移	理由或反例
数列用下标 i 指出第 i 项	✓	线性表用位序 i
数列可以无穷	✗	线性表必须有限
集合 $\{a, b\} = \{b, a\}$	✗	线性表 $(a, b) \neq (b, a)$
集合中元素互不相同	✗	线性表允许重复；“纯集合”需另做去重（例 2.3）

数列：$a_1, a_2, a_3, \cdots \to \infty$

线性表：$(a_1, \ldots, a_n)$ ✓有限

集合：$\{a, b\} = \{b, a\}$

线性表：$(a, b) \neq (b, a)$ ✗

2.2　线性表的抽象数据类型

有了定义，下一步要说明“对线性表能做什么”。按第 1 章 ADT 的格式（课件 2 P6，笔记 Memo 547–548）：

```
ADT List {
```

数据对象：$D = \{a_i \mid a_i \in \text{ElemSet},\ i = 1, 2, \ldots, n,\ n \geq 0\}$　　// n 为表长，$n = 0$ 时为空表

数据关系：$R = \{\langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D,\ i = 2, 3, \ldots, n\}$

基本操作：`InitList`、`DestroyList`、`ClearList`、`ListEmpty`、`ListLength`、`GetElem`、`LocateElem`、`PriorElem`、`NextElem`、`ListInsert`、`ListDelete`、`ListTraverse`

```
} ADT List
```

每个基本操作都要写清两件事：**初始条件**（调用前必须满足什么）与**操作结果**（调用后表变成什么样）。下表把笔记中的写法与课件、教材 C++ 类的成员函数对应起来。

笔记 / 课件 ADT 写法	教材 C++ 成员函数	初始条件　→　操作结果
`InitList(&L)`	构造函数	—→构造一个空线性表 L
`DestroyList(&L)`	析构函数	L 已存在→销毁 L，**线性表本身都没有了**
`ClearList(&L)`	`Clear()`	L 已存在→ L 重置为**空表**（表还在）
`ListEmpty(L)`	`Empty() const`	L 已存在→空表返回 true，否则 false
`ListLength(L)`	`Length() const`	L 已存在→返回元素个数
`GetElem(L,i,&e)`	`GetElem(i,e) const`	L 已存在，$1 \le i \le$ Length →用 e 返回第 i 个元素
`LocateElem(L,e, compare())`	（查找函数）	L 已存在→返回第一个与 e 满足 compare 的元素位序，不存在返回 0
`PriorElem(L,cur_e, &pre_e)`	—	若 cur_e 是 L 的元素且非第一个→用 pre_e 返回其前驱，否则 pre_e 无定义
`NextElem(L,cur_e, &next_e)`	—	与上相似，求后继（cur_e 非最后一个）
`ListInsert(&L,i,e)`	`Insert(i,e)`	L 已存在，$1 \le i \le$ Length+1 →在第 i 个位置**之前**插入 e，长度加 1
`ListDelete(&L,i,&e)`	`Delete(i,e)`	L 已存在，$1 \le i \le$ Length →删除第 i 个元素并用 e 返回，长度减 1
`ListTraverse(L,visit())`	`Traverse(visit) const`	L 已存在→依次对每个元素调用 visit

表 2.1: 线性表的基本操作。参数前的 & 表示该参数会被修改（C++ 中用引用实现）。

表中有两处初学者最容易记混的地方，用图说清楚：

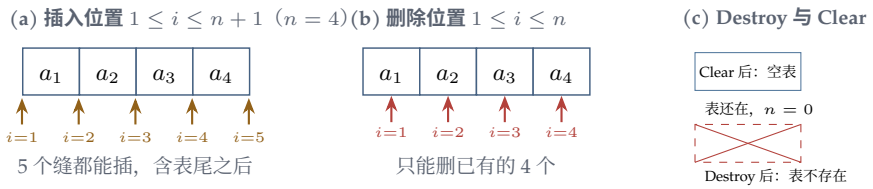


图 2.3: 插入允许 $n+1$ 个位置，删除只有 n 个位置；销毁与清空的区别（笔记 Memo 547 红字）。

ADT 只说“做什么”，不说“怎么做”。在 C++ 中，可以把它写成一个**抽象类**：只有纯虚函数声明，没有实现。第 3、4 章的顺序表、链表都可以看作它的两种实现。 教学补全

代码 2.1: 线性表 ADT 的 C++ 抽象类写法（接口与实现分离）

```cpp
template <class ElemType>
class LinearList {                    // 只规定“能做什么”
public:
    virtual ~LinearList() {}
    virtual int  Length() const = 0;                          // = 0: 纯虚函数，没有实现
    virtual bool Empty() const = 0;
    virtual void Clear() = 0;
    virtual void Traverse(void (*visit)(const ElemType &)) const = 0;
    virtual bool GetElem(int position, ElemType &e) const = 0;      // 1 <= position <= Length()
    virtual bool Insert(int position, const ElemType &e) = 0;       // 1 <= position <= Length()+1
    virtual bool Delete(int position, ElemType &e) = 0;             // 1 <= position <= Length()
};
// 顺序表、链表各自继承它并给出实现: class SqList : public LinearList<ElemType> { ... };
```

为什么用 `bool` 返回值？ 课件的成员函数在位置非法时返回 `false`，而不是直接崩溃——这是第 1 章“健壮性”的体现。调用者应当检查返回值。

2.3　只用基本操作解决问题：并集、有序归并、去重

ADT 的威力在于：下面三个算法只调用基本操作，**不关心表是怎么存的**。代码写成 C++ 函数模板，任何提供了这些成员函数的表类都能用。

2.3.1 例 2.1　集合的并（课件 2 P9–P10）

问题：集合 A、B 分别用线性表 La、Lb 表示，求 $A = A \cup B$。
转化：扩大 La，把“在 Lb 中、不在 La 中”的元素插到 La 表尾。三步：①从 Lb 依次取元素 e（`GetElem`）；②在 La 中查 e（`LocateElem`）；③查不到就插到 La 表尾（`Insert(La_len+1, e)`）。

代码 2.2: 例 2.1 并集：La = La ∪ Lb（只调用基本操作）

```
template <class ElemType>
void Union(SqList<ElemType> &La, const SqList<ElemType> &Lb) {
    ElemType e;
    int laLen = La.Length();
    for (int i = 1; i <= Lb.Length(); i++) {
        Lb.GetElem(i, e);                    // ① 取 Lb 的第 i 个元素
        if (La.LocateElem(e) == 0)           // ② La 中没有它（返回 0 表示查找失败）
            La.Insert(++laLen, e);           // ③ 插到 La 的表尾
    }
}
```

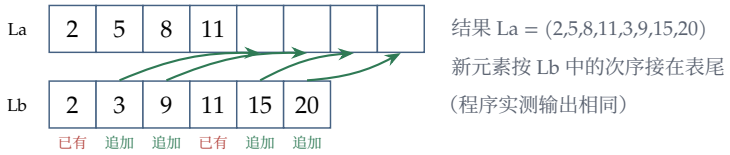


图 2.4: 例 2.1 的执行结果：Lb 中 2、11 已在 La 中，跳过；3、9、15、20 依次追加。

复杂度预告：Lb 有 n 个元素，每个都要在 La（最终最多 $m+n$ 个）中顺序查找，所以基本操作次数约为 $O(m \cdot n)$。这与存储方式有关：若 `GetElem` 是 $O(1)$（顺序表），总复杂度 $O(mn)$；若 `GetElem` 是 $O(n)$（链表），还要再乘上取元素的代价。这就是第 1 章说的“实现依赖于存储结构”。

2.3.2 例 2.2　有序表的归并（课件 2 P11，已按纠错①修订）

问题：La、Lb 中元素都按值**非递减**排列，把它们归并为新表 Lc，仍按值非递减排列。
思路：设 i、j 分别“指着”La、Lb 中当前还没放进 Lc 的最小元素 a、b。每次比较 a 与 b，把较小的放到 Lc 表尾，并让对应的指针后移；一个表放完后，把另一个表剩下的元素依次照搬。

$$c = \begin{cases} a, & a \le b \\ b, & a > b \end{cases} \qquad \text{（保留重复的经典规则）}$$

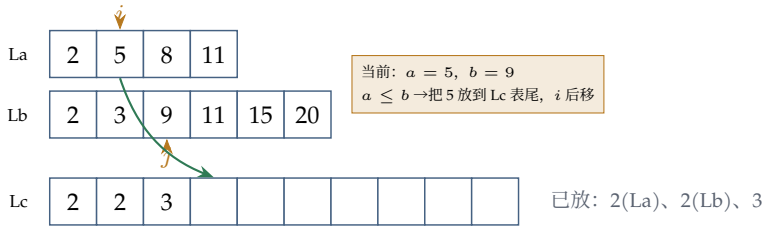


图 2.5: 有序归并的“双指针”：每一步只比较两个表当前的最小元素。

代码 2.3: 例 2.2 有序归并：allowDup 为 true 保留重复，为 false 相等时只放一个

```
template <class ElemType>
void MergeList(const SqList<ElemType> &La, const SqList<ElemType> &Lb,
               SqList<ElemType> &Lc, bool allowDup = true) {
    Lc.Clear();
    int i = 1, j = 1, k = 0;                 // i、j 指向 La、Lb 当前元素；k 为 Lc 当前长度
    ElemType a, b;
    while (i <= La.Length() && j <= Lb.Length()) {   // 两个表都还有元素
```

```
        La.GetElem(i, a);
        Lb.GetElem(j, b);
        if (a < b)       { Lc.Insert(++k, a); i++; }
        else if (b < a)  { Lc.Insert(++k, b); j++; }
        else {                              // a == b
            Lc.Insert(++k, a); i++;
            if (!allowDup) j++;             // 不要重复：Lb 中相等的也跳过
        }
    }
    while (i <= La.Length()) { La.GetElem(i++, a); Lc.Insert(++k, a); }   // La 剩余部分
    while (j <= Lb.Length()) { Lb.GetElem(j++, b); Lc.Insert(++k, b); }   // Lb 剩余部分
}
```

运行结果：La=(2,5,8,11)，Lb=(2,3,9,11,15,20) 时，

保留重复：Lc = (2, 2, 3, 5, 8, 9, 11, 11, 15, 20)；　不重复：Lc = (2, 3, 5, 8, 9, 11, 15, 20)，与课件示例一致。

为什么正确（循环不变式）：每次循环开始时，Lc 中已有的元素是 La、Lb 中**最小**的那 k 个且已有序，a、b 分别是两表剩余部分的最小值，所以 $\min(a,b)$ 就是剩余所有元素中最小的，放到 Lc 尾部后不变式仍成立。

复杂度：每次循环恰好放一个元素到 Lc（不重复版本相等时放一个、消耗两个），总次数不超过 $m+n$，若取元素和表尾插入都是 $O(1)$，则 $T=O(m+n)$。

2.3.3 例 2.3　由非纯集合构造纯集合（课件 2 P13 主观题）

问题：集合 B 中可能有重复元素，构造纯集合 A，使 A 只包含 B 中所有值各不相同的元素。

思路：与例 2.1 完全同构——把“La 已有的元素”换成“A 已有的元素”：A 从空表开始，依次取 B 的元素，A 中没有就插到 A 表尾。

代码 2.4: 例 2.3 去重：A 中保留 B 里每个值的第一次出现

```
template <class ElemType>
void Purify(const SqList<ElemType> &B, SqList<ElemType> &A) {
    ElemType e;
    A.Clear();                              // A 从空表开始
    for (int i = 1; i <= B.Length(); i++) {
        B.GetElem(i, e);
        if (A.LocateElem(e) == 0)           // A 中还没有 e
            A.Insert(A.Length() + 1, e);    // 插到表尾，保持 B 中的先后次序
    }
}
// B = (3,5,3,8,5,5,1)  →  A = (3,5,8,1)
```

本节小结：三个算法的共同“骨架”

①依次取一个表的元素（`GetElem`）；②在另一个表中判断/比较（`LocateElem` 或比较 a,b）；③按条件插到结果表的表尾（`Insert(Length()+1, e)`）。课件 2 P27 的“差集”、第二章作业的“交集”也是这个骨架，只是第②步的判断条件不同：差集是“在 Lb 中**不存在**才插入”，交集是“在 Lb 中**存在**才插入”。

学完这里，你应该能够回答

1. 写出线性表的定义，并解释“有限”“序列”“相同类型”三个词各起什么作用。
2. 为什么插入位置的合法范围是 $1\le i\le n+1$，而删除是 $1\le i\le n$?
3. `DestroyList` 与 `ClearList` 执行后，线性表分别处于什么状态?

4. 有序归并中，若 La 有 m 个元素、Lb 有 n 个元素，最多循环多少次？

本章回顾与知识地图

到这里，我们只在“逻辑层面”定义了线性表，并且已经能写出只调用基本操作的算法。但有一个问题始终悬着：**这些基本操作本身怎么实现？代价是多少？**例 2.1 的复杂度取决于 `GetElem` 和 `LocateElem` 的代价，而它们取决于存储方式。下一章先研究最直接的一种：顺序存储。

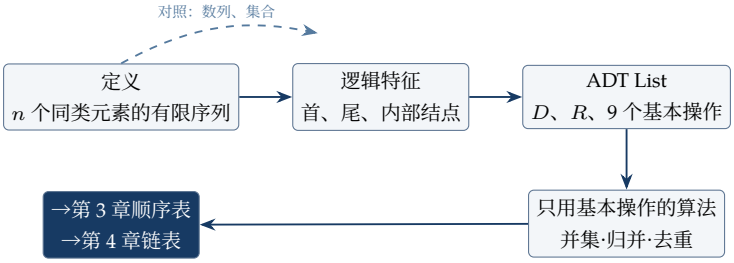


图 2.6: 第 2 章知识地图。

第 3 章　线性表的顺序存储：顺序表

上一章留下的问题：基本操作的代价取决于存储方式。**本章回答三个问题：**①怎样用一段连续内存存放线性表，第 i 个元素在哪里？②插入、删除为什么要移动元素，平均移动多少个？③在顺序表上怎样实现差集、多项式相加，代价多大？

本章知识路线：顺序存储→地址公式（随机存取）→ C++ 类模板→插入/删除与平均移动次数→五步口诀→应用（差集、多项式）→优缺点→引出链表。

3.1　顺序存储结构与地址公式

定义（顺序存储结构 / 顺序表，课件 2 P15）

线性表的**顺序存储结构**：把线性表的数据元素按**逻辑顺序**依次存放在一组**地址连续**的存储单元里。用这种方法存储的线性表简称**顺序表**。

特点：表中**逻辑上相邻**的数据元素，存储时在**物理位置上也一定相邻**（笔记 Memo 549：以物理位置相邻表示逻辑关系）。

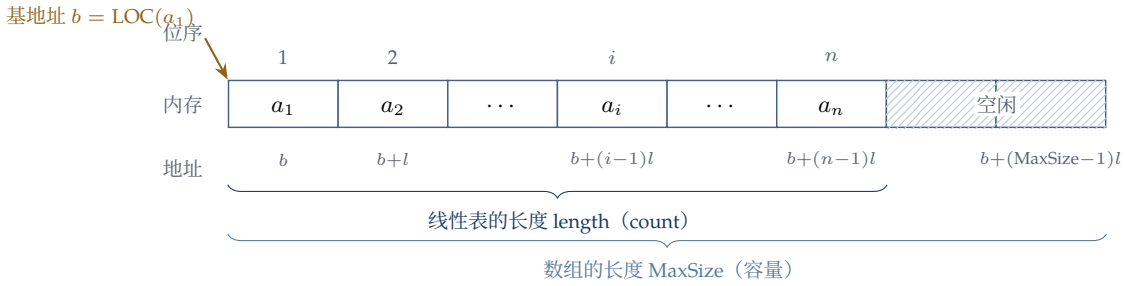


图 3.1: 顺序表的存储示意（课件 2 P17、笔记 Memo 549）。每个元素占 l 个存储单元。

设每个元素占 l 个存储单元，并以元素所占第一个单元的地址作为它的存储位置。因为相邻元素紧挨着存放，所以

地址公式（课件 2 P15–P16，笔记 Memo 549）

$$\mathrm{LOC}(a_{i+1}) = \mathrm{LOC}(a_i) + l, \qquad \boxed{\mathrm{LOC}(a_i) = \mathrm{LOC}(a_1) + (i-1)\times l} \quad (1 \le i \le n)$$

$$\mathrm{LOC}(a_i) \;=\; \mathrm{LOC}(a_1) \;+\; (i-1) \;\times\; l$$

第 i 个元素的首地址　基地址（第 1 个元素）　a_i 前面有 $i-1$ 个元素　每个元素占 l 个单元

图 3.2: 地址公式逐项含义：“起点 + 前面元素个数 × 每个元素的宽度”。

推导（由相邻关系累加，教学补全）

已知相邻关系 $\mathrm{LOC}(a_{k+1}) - \mathrm{LOC}(a_k) = l$ 对 $k = 1, 2, \ldots, i-1$ 都成立。把这 $i-1$ 个等式相加，左边中间项两两抵消（“裂项相消”）：

$$\sum_{k=1}^{i-1} \left[\mathrm{LOC}(a_{k+1}) - \mathrm{LOC}(a_k)\right] = \mathrm{LOC}(a_i) - \mathrm{LOC}(a_1) = (i-1)\,l,$$

移项即得 $\mathrm{LOC}(a_i) = \mathrm{LOC}(a_1) + (i-1)l$。■

意义：计算任一元素的地址只需一次乘法和一次加法，与表长 n 无关，所以存取任一元素的时间是 $O(1)$——这叫**随机存取**（笔记 Memo 549 红字“常用⇒ $T(n) = O(1)$”）。“随机”不是“碰运气”，而是“想访问哪个就直接访问哪个”。

例题 3.1　计算地址

题：顺序表第一个元素的地址是 2000，每个元素占 4 个字节，求第 5 个元素的地址。

解：$\mathrm{LOC}(a_5) = 2000 + (5-1)\times 4 = 2016$。　**检查：**$a_1, a_2, a_3, a_4$ 依次占 2000–2003、2004–2007、2008–2011、2012–2015，a_5 从 2016 开始。

本题想说明：公式里是 $(i-1)$ 不是 i——a_1 本身就在基地址上，前面没有元素。

这个公式在 C++ 里天天都在用。数组 `ElemType elems[...]` 中，`elems[k]` 的地址就是 `elems + k`，字节地址为“首地址 $+\,k\times$`sizeof(ElemType)`”。因为 C++ 下标从 0 开始，位序 i 对应下标 $k = i-1$，于是 `&elems[i-1]` 恰好就是 $\mathrm{LOC}(a_1) + (i-1)l$。

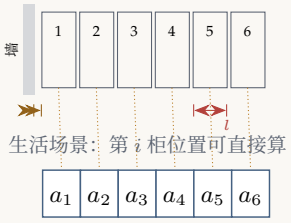
生活中的它　超市寄存柜

场景：超市一排寄存柜从左到右编号 1–20，每格宽 40 cm，1 号柜左边缘离墙 1 m。

对应：柜子编号 = 位序 i；格宽 = l；1 号柜位置 = 基地址；“第 i 号柜离墙多远” = $\mathrm{LOC}(a_i)$。

小估算：13 号柜左边缘离墙 $1 + (13-1)\times 0.4 = 5.8$ m，不用从 1 号数过去。

边界：柜子一排装满就不能再加柜子（容量固定）——这正是顺序表“表满”的问题。



对照：地址公式 ↔ 中学数学的等差数列通项公式（类比同构型）

一句话定位：元素地址 $\mathrm{LOC}(a_1), \mathrm{LOC}(a_2), \ldots$ 构成首项为基地址、公差为 l 的等差数列。

等差数列中的结论	迁移	理由或反例
通项 $a_n = a_1 + (n-1)d$，已知首项和公差可直接求任一项	✓	$\mathrm{LOC}(a_i) = \mathrm{LOC}(a_1) + (i-1)l$，这就是 $O(1)$ 随机存取
相邻两项之差恒为公差	✓	相邻元素地址差恒为 l
公差可以是负数、分数	✗	l 是每个元素占的单元数，必为正整数（C++ 中为 `sizeof(ElemType)`）
前 n 项和公式	✗	地址相加没有实际意义
二维“等差”：$\mathrm{LOC}(a_{ij}) = \mathrm{LOC}(a_{11}) + [(i-1)m + (j-1)]l$（按行存放，每行 m 个）	△	要多一个条件：先确定是按行还是按列存放（第 5 章数组） 拓展联想

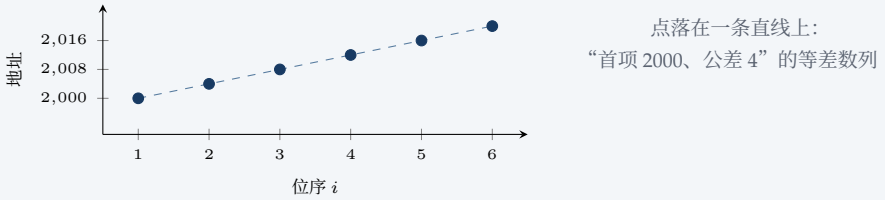


图 3.3: 地址随位序线性增长：斜率就是每个元素的宽度 l。

3.2　顺序表的 C++ 实现

课件 2 P18 给出两种描述顺序表存储空间的方式：

- **静态数组**：`ElemType a[100];`——容量在编译时确定，不能扩充；
- **动态申请**：`ElemType *elems = new ElemType[maxSize];`——容量在运行时确定，空间可以动态分配。

笔记 Memo 549 写道：线性表长度可变，但数组长度不能动态定义，所以要**另用一个变量表示表长**。教材的类模板正是这样设计的：

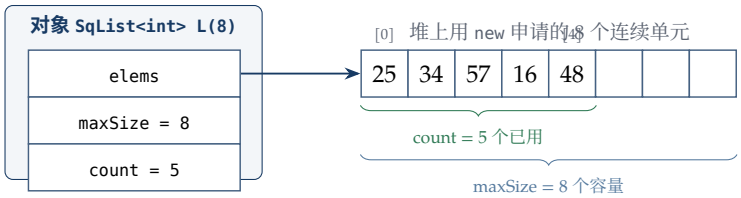


图 3.4: 一个顺序表对象的内存图：对象里只存首地址、容量、表长三样东西，元素在另一段连续空间里。

代码 3.1: 顺序表类模板（与课件 2 P19–P20、你的代码一致；去掉了 Delete 中的调试输出）

```
const int DEFAULT_SIZE = 100;

template <class ElemType>
class SqList {
protected:
    ElemType *elems;   // 元素存储空间的首地址
    int maxSize;       // 容量：最多能存多少个元素
    int count;         // 表长：当前实际有多少个元素
public:
    SqList(int size = DEFAULT_SIZE);                    // 构造：建一个空表（InitList）
    virtual ~SqList();                                  // 析构：释放空间（DestroyList）
    int  Length() const { return count; }               // 求表长
    bool Empty()  const { return count == 0; }          // 判空
    void Clear()        { count = 0; }                  // 清空：只把表长置 0，空间保留
    void Traverse(void (*visit)(const ElemType &)) const;  // 遍历
```

```
    bool GetElem(int position, ElemType &e) const;          // 取第 position 个
    bool SetElem(int position, const ElemType &e);          // 改第 position 个
    int  LocateElem(const ElemType &e) const;               // 按值查找，返回位序，找不到返回 0
    bool Insert(int position, const ElemType &e);           // 插入
    bool Delete(int position, ElemType &e);                 // 删除
    SqList(const SqList<ElemType> &source);                 // 复制构造函数
    SqList<ElemType> &operator=(const SqList<ElemType> &source);   // 赋值运算符
};

template <class ElemType>
SqList<ElemType>::SqList(int size) {
    elems = new ElemType[size];   // 申请 size 个元素的连续空间
    maxSize = size;
    count = 0;                    // 空表
}

template <class ElemType>
SqList<ElemType>::~SqList() { delete[] elems; }   // new[] 对应 delete[]

template <class ElemType>
bool SqList<ElemType>::GetElem(int position, ElemType &e) const {
    if (position < 1 || position > count) return false;   // 1 <= position <= Length()
    e = elems[position - 1];                               // 位序 position ↔ 下标 position-1, O(1)
    return true;
}

template <class ElemType>
int SqList<ElemType>::LocateElem(const ElemType &e) const {
    for (int i = 0; i < count; i++)
        if (elems[i] == e) return i + 1;   // 返回位序
    return 0;                              // 0 表示“没找到”
}
```

逐段说明。第 5–7 行三个数据成员对应图 3.4；`protected` 体现 ADT 的封装性。第 10、11 行的构造、析构函数就是 ADT 中的 `InitList`、`DestroyList`。第 14 行 `Clear` 只把 `count` 置 0，空间还在——正是笔记中“重置为空表”与“销毁”的区别。第 34–38 行 `GetElem` 先检查位置，再一次下标运算取值，$O(1)$。第 41–45 行 `LocateElem` 要逐个比较，最坏比较 n 次，$O(n)$。

`Traverse` 的函数指针参数（你的代码注释里已经写得很好，这里浓缩一下）：参数 `void (*visit)(const ElemType &)` 表示“一个接收 `const ElemType&`、无返回值的函数”。`Traverse` 负责逐个取元素，`visit` 决定对每个元素做什么；换一种处理方式只需换一个访问函数：

```
void Show(const int &x)   { cout << x << " "; }       // 打印
void ShowSq(const int &x) { cout << x * x << " "; }   // 打印平方
L.Traverse(Show);     // 10 20 30
L.Traverse(ShowSq);   // 100 400 900
```

修正你的两个代码片段（对应纠错⑤⑥）

你的笔记 Memo 549 用的是结构体写法 `struct SqList { ElemType *elem; int length; };`，并配了一个 `MAXSIZE` 常量。下面按这种写法给出修正后的初始化与建表，可以直接放进你的「线性表」文件夹替换原文件内容。

代码 3.2: 结构体写法的初始化与按输入建表（已修正）

```
#include <iostream>
#include <new>            // std::nothrow
using namespace std;
```

```cpp
const int MAXSIZE = 100;

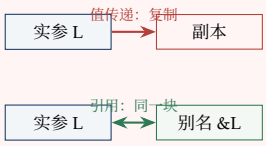
struct SqList {
    int *elem;            // 存储空间首地址
    int length;           // 当前长度
};

bool InitList(SqList &L) {                // 引用传参：要修改实参
    L.elem = new (nothrow) int[MAXSIZE];  // nothrow：失败时返回 nullptr 而不是抛异常
    if (L.elem == nullptr) return false;  // 这样判断才有意义
    L.length = 0;
    return true;
}

bool CreateList(SqList &L) {              // 依次读入整数，读到 -1 结束
    int x;
    while (cin >> x && x != -1) {         // ① 先读 ② 再判断是否为结束标志
        if (L.length >= MAXSIZE) {        // ③ 判满
            cout << "顺序表已满，无法插入新元素" << endl;
            return false;
        }
        L.elem[L.length] = x;             // ④ 存到下标 length（即第 length+1 个位置）
        L.length++;                       // ⑤ 表长加 1 —— 原代码漏了这一步
    }
    return true;
}
```

课件 2 P26 的三个常见错误

1. **混淆 `count` 与 `maxSize`**：前者是实际元素个数，后者是存储空间大小，即最多能存多少元素。
2. **顺序表对象作形参时用值传递**：`void f(SqList<int> L)` 会调用复制构造函数把整个表复制一遍，浪费空间和时间；且修改的是副本。应写 `SqList<int> &L`（要改）或 `const SqList<int> &L`（只读）。
3. **`Insert` 忘了检查位置合法性、忘了检查是否还有空间**——用下一节的“五步口诀”避免。



3.3　插入与删除：为什么要移动，平均移动多少

顺序表的优点是取元素快，代价出在插入和删除上。原因很直接：物理位置必须与逻辑次序一致，中间插进一个，后面的都要“让位”。

3.3.1　插入

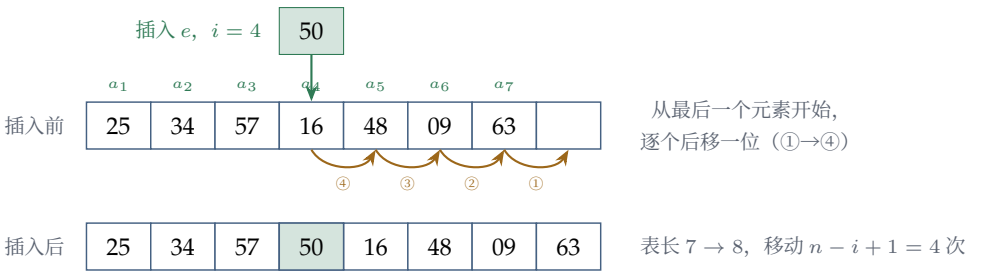


图 3.5: 在第 4 个位置插入 50（课件 2 P22）：$a_4 \sim a_7$ 从后往前依次后移，空出下标 3 放入新元素。

代码 3.3: 顺序表插入：在第 position 个位置之前插入 e

```
template <class ElemType>
bool SqList<ElemType>::Insert(int position, const ElemType &e) {
    if (count == maxSize) return false;                    // ① 空间是否支持：表满
    if (position < 1 || position > count + 1) return false; // ② 参数检查：1 <= position <= Length()+1
    for (int i = count; i >= position; i--)                // ③ 核心操作：从最后一个开始，逐个后移
        elems[i] = elems[i - 1];
    elems[position - 1] = e;                               //    放入新元素
    count++;                                               // ④ 对其他属性的影响：表长 +1
    return true;                                           // ⑤ 正确返回
}
```

第 5 行循环中 `i` 从 `count` 递减到 `position`：把下标 $i-1$ 的元素搬到下标 i。第一次搬的是最后一个元素（下标 `count-1` → `count`），这就是课件中“**从后往前**至 i 位置逐个元素后移一位”。为什么不能从前往后？看下图：

易错：后移必须从后往前

若从前往后搬，第一步就把 a_5 覆盖成 a_4 的值，原来的 a_5 永远丢失，后面全变成同一个值。

另外两个边界：$i=n+1$（插在表尾）时循环一次也不执行，移动 0 次；表满时必须先返回 `false`，否则 `elems[count]` 越界写内存。



3.3.2 平均移动次数的推导

定理 / 结论 3.1（插入的平均移动次数，课件 2 P22）

在长度为 n 的顺序表中插入一个元素，若在 $n+1$ 个合法位置上插入的概率相等，则平均移动元素次数为

$$E_{\text{ins}} = \frac{n}{2}, \qquad \text{时间复杂度} O(n).$$

证明

第 1 步，写出每个位置的移动次数。在第 i 个位置插入，要移动 $a_i, a_{i+1}, \dots, a_n$，共 $n-i+1$ 个。$i=1$ 时移动 n 个，$i=n+1$ 时移动 0 个。

第 2 步，写出概率。合法位置共 $n+1$ 个，等概率，每个位置 $p_i = \dfrac{1}{n+1}$。

第 3 步，求期望（按概率加权平均）：

$$\begin{aligned} E_{\text{ins}} &= \sum_{i=1}^{n+1} p_i\,(n-i+1) = \frac{1}{n+1}\sum_{i=1}^{n+1}(n-i+1) && (p_i \text{ 是常数，提出来}) \\ &= \frac{1}{n+1}\big(n+(n-1)+\cdots+1+0\big) = \frac{1}{n+1}\cdot\frac{n(n+1)}{2} = \frac{n}{2}. && (\text{等差数列求和}) \end{aligned}$$

第 4 步，得复杂度。基本语句（元素后移）平均执行 $\frac{n}{2}$ 次，最坏 n 次，故 $T(n)=O(n)$。■

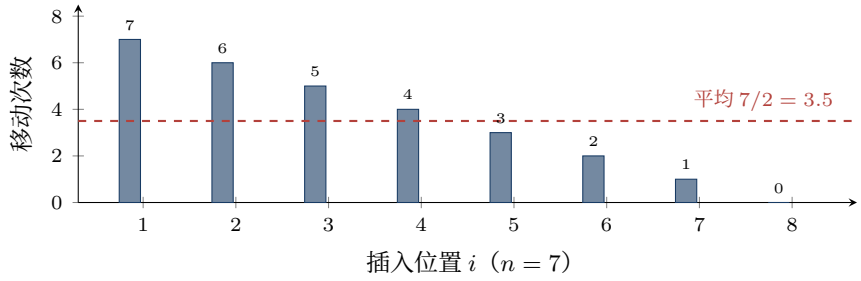


图 3.6: $n=7$ 时 8 个插入位置的移动次数：一个三角形，平均高度恰为 $n/2$。

3.3.3 删除

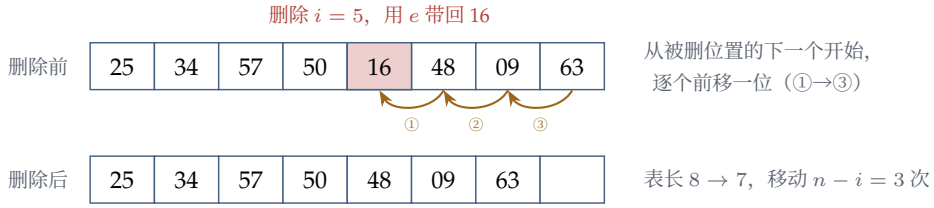


图 3.7: 删除第 5 个元素（课件 2 P24）：$a_6\sim a_8$ 从前往后依次前移。

代码 3.4: 顺序表删除：删除第 position 个元素并用 e 返回

```
template <class ElemType>
bool SqList<ElemType>::Delete(int position, ElemType &e) {
    if (position < 1 || position > count) return false;     // ① 参数检查：1 <= position <= Length()
    e = elems[position - 1];                                // ② 先把被删元素带出来
    for (int i = position; i < count; i++)                  // ③ 从被删位置的下一个开始，逐个前移
        elems[i - 1] = elems[i];
    count--;                                                // ④ 表长 -1
    return true;                                            // ⑤ 正确返回
}
```

定理 / 结论 3.2（删除的平均移动次数）

在 n 个合法位置上删除的概率相等时，平均移动次数 $E_{\text{del}}=\dfrac{n-1}{2}$，时间复杂度 $O(n)$。

证明

删除第 i 个元素要移动 $a_{i+1},\dots,a_n$ 共 $n-i$ 个；$p_i=\frac{1}{n}$。

$$E_{\text{del}}=\sum_{i=1}^{n}\frac{1}{n}(n-i)=\frac{1}{n}\big((n-1)+(n-2)+\cdots+0\big)=\frac{1}{n}\cdot\frac{(n-1)n}{2}=\frac{n-1}{2}.\qquad\blacksquare$$

比较：插入多一个合法位置（表尾之后），所以平均多移动 $\frac{1}{2}$ 次。两者的空间复杂度都是 $O(1)$（只用了循环变量 `i`，笔记 Memo 550）。

3.3.4 五步口诀：写出完整、健壮的操作

课件 2 P21 的“五步口诀”是写任何一个修改型操作的检查清单。对照上面的 `Insert`：



图 3.8: “五步口诀”与插入代码的对应（课本代码中①②顺序可互换，两者都要有）。

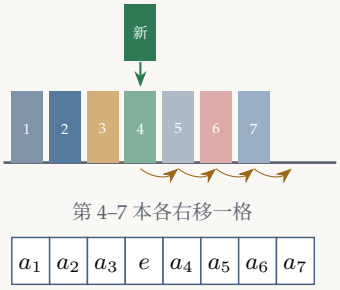
生活中的它　书架上按编号插一本书

场景：书架上 7 本书按编号紧挨着放，书架末尾还有空位。新到一本应放在第 4 位的书。

对应：书 = 元素；书架格子 = 连续存储单元；“紧挨着、不留空” = 物理相邻表示逻辑相邻。

小估算：要把第 4–7 本依次右挪一格，挪 4 本；如果新书恰好排最后，一本都不用挪；平均挪 $7/2 = 3.5$ 本。

边界：现实中可以先把书斜放着“凑合”，顺序表不允许留空洞——否则地址公式就失效了。



对照：平均移动次数 ↔ 概率论中离散型随机变量的数学期望（思想重现型）

一句话定位：“平均移动次数”就是随机变量 X=“移动次数”的数学期望 $E(X) = \sum x_i p_i$。

概率论中的结论	迁移	理由或反例
$E(X) = \sum x_i p_i$	✓	定理 3.1、3.2 的推导就是这个公式
等概率时期望 = 算术平均	✓	$p_i = \frac{1}{n+1}$，期望就是把各位置移动次数取平均
概率分布改变，期望随之改变	△	若插入总发生在表尾，期望为 0，此时“平均 $O(n)$”不再成立——结论依赖“等概率”这个条件
期望值一定能取到	✗	$n = 8$ 时平均插入移动 4 次可取到，但删除平均 3.5 次，任何一次删除都不会恰好移动 3.5 个

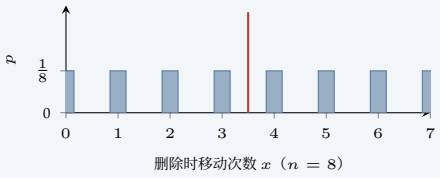


图 3.9: 删除移动次数服从 $\{0, \ldots, n-1\}$ 上的均匀分布，期望在两根柱子之间。

3.4 顺序表的应用：差集与稀疏多项式相加

3.4.1 差集（课件 2 P27–P29）

$L_c = L_a - L_b$：取出 L_a 的每个元素，在 L_b 中查找，未出现的插到 L_c 表尾。骨架与 2.3 节完全相同。

代码 3.5: 两个顺序表表示的集合求差集（课件 2 P27，已整理）

```
template <class ElemType>
void Difference(const SqList<ElemType> &La, const SqList<ElemType> &Lb, SqList<ElemType> &Lc) {
    ElemType aElem, bElem;
    Lc.Clear();
    for (int aPos = 1; aPos <= La.Length(); aPos++) {
        La.GetElem(aPos, aElem);                    // 取出 La 的一个元素
        bool isExist = false;                       // aElem 是否在 Lb 中出现
        for (int bPos = 1; bPos <= Lb.Length(); bPos++) {
            Lb.GetElem(bPos, bElem);
            if (aElem == bElem) { isExist = true; break; }   // 找到就不必再查
        }
        if (!isExist) Lc.Insert(Lc.Length() + 1, aElem);     // 只在 La 中出现：插入 Lc
    }
}
```

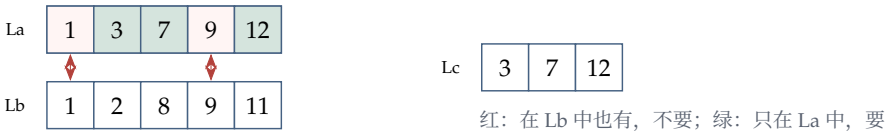


图 3.10: 课件 2 P29 的数据：La=(1,3,7,9,12)，Lb=(1,2,8,9,11)，差集 Lc=(3,7,12)（程序实测）。

复杂度：外层 m 次，内层最多 n 次，取元素与表尾插入都是 $O(1)$，所以 $T = O(m \cdot n)$；空间除 Lc 外 $O(1)$。**作业提示：**交集只需把第 12 行的条件 `!isExist` 改为 `isExist`，请自己写完整并测试。

3.4.2 稀疏多项式相加（笔记 Memo 543–544，你的代码「案例：多项式的合并.cpp」）

一元多项式 $P_n(x) = p_0 + p_1 x + \cdots + p_n x^n$ 可以只存系数：$P = (p_0, p_1, \ldots, p_n)$，系数下标就是指数。两个多项式相加，就是同下标系数相加：$R = (p_0 + q_0,\ p_1 + q_1, \ldots, p_m + q_m,\ p_{m+1}, \ldots, p_n)$。

但对**稀疏多项式** $S(x) = 1 + 3x^{10000} + 2x^{20000}$，只有 3 个非零项，却要开 20001 个单元，绝大多数存 0——**浪费空间**（笔记红字）。改进：只存非零项，每项存“(系数，指数)”二元组，按指数递增排列：

$$P_n(x) = p_1 x^{e_1} + p_2 x^{e_2} + \cdots + p_m x^{e_m} \quad \Longrightarrow \quad P = \big((p_1, e_1), (p_2, e_2), \ldots, (p_m, e_m)\big).$$



图 3.11: 两种表示。稀疏时 (a) 中红色的 0 是浪费；(b) 用顺序表存“(系数，指数)”。

相加的思路（笔记 Memo 544）：①建新表 C；②分别从头遍历比较 A、B 的每一项；③一个多项式遍历完，把另一个的剩余项复制到 C。这正是有序归并（例 2.2）——比较的是**指数**，指数相等时**系数相加**，和为 0 则不保留：

代码 3.6: 稀疏多项式相加（你的代码，三个多项式都按指数递增存放）

```
struct Term { double p; int q; };            // 一项：系数 p，指数 q

void AddPoly(const SqList<Term> &A, const SqList<Term> &B, SqList<Term> &C) {
    C.Clear();
    int i = 1, j = 1;
    Term a, b;
    while (i <= A.Length() && j <= B.Length()) {
        A.GetElem(i, a);
        B.GetElem(j, b);
        if (a.q < b.q)      { C.Insert(C.Length() + 1, a); ++i; }   // A 的这项指数小，先放
        else if (a.q > b.q) { C.Insert(C.Length() + 1, b); ++j; }   // B 的这项指数小，先放
        else {                                                      // 同类项
            Term t = {a.p + b.p, a.q};
            if (t.p != 0) C.Insert(C.Length() + 1, t);              // 系数为 0 的项不保留
            ++i; ++j;
        }
    }
    while (i <= A.Length()) { A.GetElem(i++, a); C.Insert(C.Length() + 1, a); }   // A 的剩余项
    while (j <= B.Length()) { B.GetElem(j++, b); C.Insert(C.Length() + 1, b); }   // B 的剩余项
}
```

例题 3.2　笔记 Memo 544 的数据，逐步执行

$A=((7,0),(3,1),(9,8),(5,17))$，$B=((8,1),(22,7),(-9,8))$。

步	a	b	比较指数	C 的变化
1	(7,0)	(8,1)	$0<1$	放入 (7,0)，i++
2	(3,1)	(8,1)	$1=1$	系数 $3+8=11$，放入 (11,1)，i++、j++
3	(9,8)	(22,7)	$8>7$	放入 (22,7)，j++
4	(9,8)	$(-9,8)$	$8=8$	系数 $9-9=0$，**不放**，i++、j++
5	(5,17)	—	B 已完	复制剩余项 (5,17)

结果 $C=((7,0),(11,1),(22,7),(5,17))$，即 $C(x)=7+11x+22x^7+5x^{17}$。程序实测输出 `(7,0) (11,1) (22,7) (5,17)`。

复杂度： A 有 m 项、B 有 n 项，每次循环至少消耗一项，$T=O(m+n)$。

本题想说明： 第 4 步说明“同类项相加为 0 要删掉”不是可有可无——否则 C 中会出现系数为 0 的项。

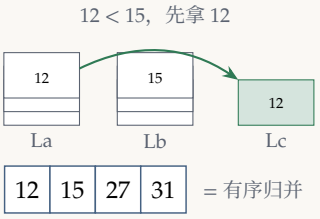
生活中的它　合并两摞按学号排好的试卷

场景： 两位助教各批完一个班的试卷，都已按学号从小到大叠好，老师要合成一摞，仍按学号排。

对应： 两摞试卷 = La、Lb；每摞最上面一张 = a、b；拿走学号小的那张放到新摞 = “取较小者插到 Lc 表尾”；一摞拿完，另一摞整摞放上去 = “复制剩余部分”。

小估算： 两摞各 30 张，最多比较 59 次就能合完，绝不需要两两比较 900 次——这就是 $O(m+n)$ 与 $O(mn)$ 的差别。

边界： 若两位助教各有一张重复的试卷（重修生），要事先规定留一张还是两张——对应纠错①中“相等时怎么办”。



对照：多项式相加 ↔ 线性代数中的向量加法（类比同构型）

一句话定位： 只存系数的表示 (a) 就是坐标向量，多项式相加 = 对应分量相加；只存非零项的表示 (b) 就是“稀疏向量”。

向量中的结论	迁移	理由或反例
向量加法：对应分量相加	✓	同次项系数相加
稀疏向量只存非零分量	✓	系数为 0 的项不保留（例 3.2 第 4 步）
对应分量直接按下标对齐	△	稀疏表示中第 k 个格子不再是 x^k 的系数，必须**按指数**对齐，而且要求两表都按指数递增排列，否则归并失效
两个向量逐分量相乘得到新向量	✗	多项式乘法不是逐项相乘，而是“每项乘每项再合并”（卷积），$(1+x)(1+x)=1+2x+x^2$



图 3.12: 稠密表示按下标对齐；稀疏表示必须按指数对齐（第 2 格：指数 8 对 7，不能直接相加）。

本章回顾：顺序表的优缺点，以及下一步

优点（笔记 Memo 550）
①存储密度大：不需要为表示逻辑关系另花空间；
②可以随机存取表中任一元素，$O(1)$。

缺点
①插入、删除需移动大量元素，平均约 $n/2$ 个，$O(n)$；
②预先分配空间：按最大需要分配，可能浪费；
③元素个数不能自由扩充（表满）。

图 3.13: 顺序表的优缺点。缺点全部来自“必须连续存放”这一条。

笔记 Memo 544 用红笔写了一句“顺序存储结构存在问题→链式存储”。如果不要求元素连续存放，插入删除时就不必移动元素，也不必预先分配一大块空间——代价是每个元素要多带一个“地址”，并且失去随机存取。下一章研究这种方法。

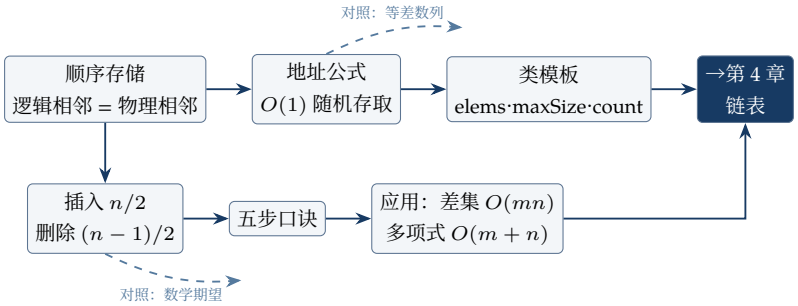


图 3.14: 第 3 章知识地图。

学完这里，你应该能够回答

1. 写出地址公式，并说明为什么顺序表能随机存取。
2. 推导插入、删除的平均移动次数；它们依赖什么假设？
3. `Clear()` 和析构函数有什么不同？`count` 和 `maxSize` 有什么不同？
4. 为什么插入时元素必须“从后往前”移动？
5. 稀疏多项式相加与有序归并有什么相同、有什么不同？

第 4 章　线性表的链式存储：链表

上一章留下的问题：顺序表插入删除要移动约一半元素，空间要预先分配。**本章回答三个问题：**①不要求连续存放时，怎样表示“后继”？头指针、头结点、首元结点各是什么？②单链表的查找、插入、删除、建立、逆置怎样一步步改指针，代价多少？③循环链表、双向链表分别解决了单链表的什么不便？

本章知识路线：结点 = 数据域 + 指针域→存储映像→头指针/头结点/首元结点→单链表类模板→查找→插入（先连后断）→删除→建表（头插/尾插）→逆置→循环链表（尾指针、合并、约瑟夫）→双向链表（对称性、四步插入）→效率比较。

4.1　链式存储结构与单链表

为什么需要它　顺序表插入、删除的时间代价是线性阶，而且会引起大量元素移动（课件 2 P32）。改进办法：**各元素的物理位置任意，不一定连续；每个元素放在一个独立的存储单元中，元素间的逻辑关系依靠单元中附加的指针来完成。**采用链式存储结构的线性表称为**链表**。

先看笔记 Memo 550–551 里的例子：线性表（赵，钱，孙，李，周，吴，郑，王）。顺序表把它们放在地

址 0031, 0033, ⋯, 0045 的连续单元里；链表则可以东一个西一个，每个单元除了名字，还记着“下一个人的地址”。

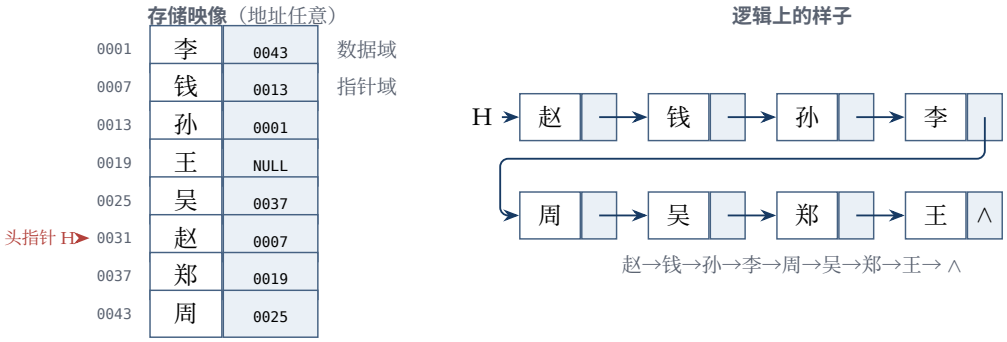


图 4.1: 链表的存储映像（笔记 Memo 551）。从头指针 H=0031 出发，顺着指针域就能按逻辑次序访问全部结点。

从 H=0031 找到“赵”，它的指针域是 0007，找到“钱”；钱指向 0013“孙”……直到“王”的指针域为 NULL。课件 2 P33 说：为了看清逻辑关系，以后链表都画成右边这种示意图。笔记中的红字概括了本质：**逻辑次序和物理次序不一定相同。**

定义（结点、链表与相关术语，课件 2 P34–P36，笔记 Memo 551–552）

- **结点**：数据元素的存储映像，由**数据域**（存元素值，`data`）和**指针域**（存后继的地址，`next`）组成。
- **链表**：n 个结点由指针链接成的一个序列。结点只有一个指针域的叫**单链表**；有两个指针域的叫**双链表**；首尾相接的叫**循环链表**。
- **头指针**：指向链表中第一个结点的指针（有头结点时指向头结点）。整个链表由头指针唯一确定。
- **头结点**：在首元结点之前附设的一个结点，它不是线性表的元素。
- **首元结点**：链表中存储第一个数据元素 a_1 的结点。

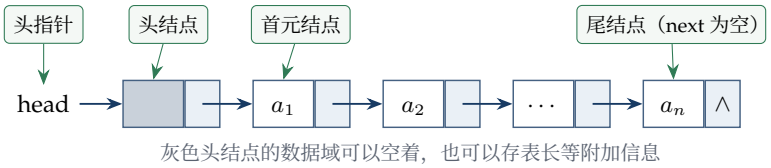


图 4.2: 带头结点的单链表各部分名称（课件 2 P35，笔记 Memo 551 图示）。

为什么要加头结点？

笔记 Memo 552 记了两点：①便于首元结点的处理；②便于空表和非空表的统一处理。用图和代码看具体差别：

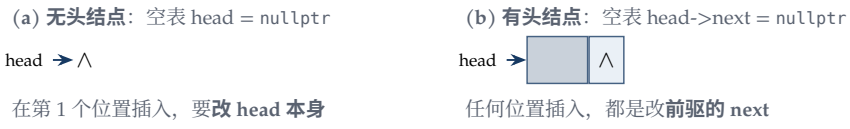


图 4.3: 空表的两种表示（笔记 Memo 552）。有头结点时，首元结点也有前驱（头结点），所以操作不必特殊处理。

代码 4.1: 无头结点：首位插入要单独处理

```
// head 是 Node<T>*，指向首元结点
if (position == 1) {              // 特殊情况
    head = new Node<T>(e, head);   // 改头指针
} else {
    Node<T> *p = 第 position-1 个结点;
    p->next = new Node<T>(e, p->next);
}
```

代码 4.2: 有头结点：所有位置一种写法

```
// head 指向头结点，头结点算第 0 个
Node<T> *p = GetElemPtr(position - 1);
p->next = new Node<T>(e, p->next);
// position == 1 时 p 就是头结点，
// 不需要任何特殊处理
```

单链表的特点（课件 2 P36，笔记 Memo 552）：结点在存储器中的位置任意；访问时只能通过头指针进入链表，顺着指针逐个找下去（**顺序存取法**），所以找第 i 个结点的时间是 $O(n)$——这是链表为“不必连续”付出的代价。

C++ 中的结点定义

笔记 Memo 552–553 的结构体写法与课件的类模板写法本质相同。下面两种都是合法的 C++：

代码 4.3: 结点的两种 C++ 写法（笔记中 `date` 已更正为 `data`）

```
// 写法一：笔记中的结构体 + 类型别名（C++11 起推荐用 using 代替 typedef）
template <class ElemType>
struct LNode {
    ElemType data;           // 数据域
    LNode *next;             // 指针域：指向后继结点
};
template <class ElemType>
using LinkList = LNode<ElemType> *;   // LinkList<int> L;  与  LNode<int> *L;  完全等价
// 习惯上：强调“整个链表”时写 LinkList L，强调“某个结点”时写 LNode *p

// 写法二：课件2 P38 的结点类模板（带构造函数，new 时可直接给出数据和指针）
template <class ElemType>
struct Node {
    ElemType data;
    Node<ElemType> *next;
    Node() : data(ElemType()), next(nullptr) {}                        // 建头结点：指针为空
    Node(const ElemType &e, Node<ElemType> *link = nullptr)             // 建数据结点
        : data(e), next(link) {}
};

// 笔记 Memo 553 的例子：存储学生学号、姓名、成绩的结点
struct Student {
    char num[8];             // 数据域
    char name[8];            // 数据域
    int score;               // 数据域
    Student *next;           // 指针域
};
```

C++ 中用 `nullptr` 表示空指针（C 语言和老教材写 `NULL`，含义相同，但 `nullptr` 类型更安全）。

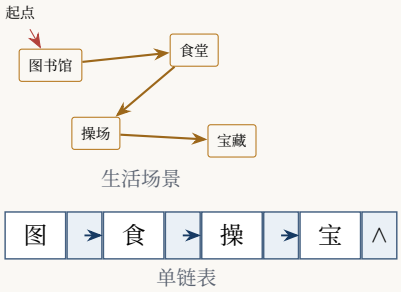
生活中的它　寻宝游戏的线索纸条

场景：班级活动玩寻宝：第一张纸条写着“去图书馆 3 楼”，那里的纸条写着“去食堂门口”……最后一张写“宝藏就在这里”。

对应：每张纸条 = 结点；纸条上的提示内容 = 数据域；“下一张在哪” = 指针域；主持人告诉你的第一个地点 = 头指针；“到此为止” = `nullptr`。

小判断：想直接去第 5 张纸条处可以吗？不行，必须从第 1 张开始依次走——这就是顺序存取，$O(n)$。

边界：纸条丢了一张，后面的全都找不到——链表中“断链”同样致命（4.2 节插入顺序的警告）。



对照：课件的单链表 ↔ C++ 标准库 `std::forward_list`（思想重现型，程序设计）

一句话定位：`std::forward_list` 就是标准库提供的单链表；它的 `before_begin()` 返回“首元结点之前的位置”，作用相当于头结点。

课件单链表的结论	迁移	理由或反例
插入/删除要先拿到**前驱**	✓	标准库只提供 `insert_after(pos, e)`、`erase_after(pos)`，都以前驱位置为参数
有头结点后首位插入不必特殊处理	✓	`insert_after(before_begin(), e)` 就是在首位插入
`Insert(position, e)` 按位序插入	✗	标准库不提供按位序插入；按位序找位置是 $O(n)$，需要自己先走过去
`Length()` 可以求表长	△	`forward_list` 故意不提供 `size()` 成员；需要时用 `std::distance(begin(), end())`，$O(n)$

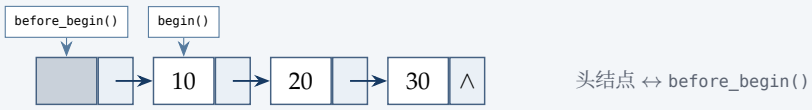


图 4.4: `before_begin()` 指向“首元之前”，与头结点的作用相同。

4.2　单链表基本操作的实现

课件 2 P39 的类模板与你的「单链表.cpp」相同：数据成员只有头指针 `head`，外加一个辅助函数 `GetElemPtr`，其余操作都建立在它之上。

代码 4.4: 简单单链表类模板的声明（带头结点）

```
template <class ElemType>
class SimpleLinkList {
protected:
    Node<ElemType> *head;                              // 头指针：指向头结点
    Node<ElemType> *GetElemPtr(int position) const;    // 辅助函数：返回第 position 个结点的指针
public:
    SimpleLinkList() { head = new Node<ElemType>; }    // 空表：只有一个头结点
    ~SimpleLinkList() { Clear(); delete head; }        // 先删数据结点，再删头结点
    int  Length() const;
    bool Empty() const { return head->next == nullptr; }   // 头结点后面没东西 = 空表
    void Clear();
    void Traverse(void (*visit)(const ElemType &)) const;
```

```
    bool GetElem(int position, ElemType &e) const;
    bool Insert(int position, const ElemType &e);
    bool Delete(int position, ElemType &e);
};
```

4.2.1 查找第 position 个结点：GetElemPtr

代码 4.5: 返回第 position 个结点的指针（头结点算第 0 个）

```
template <class ElemType>
Node<ElemType> *SimpleLinkList<ElemType>::GetElemPtr(int position) const {
    Node<ElemType> *temPtr = head;     // 从头结点出发
    int temPos = 0;                    // temPtr 所指结点的位置
    while (temPtr != nullptr && temPos < position) {
        temPtr = temPtr->next;         // 顺着 next 走一步
        temPos++;
    }
    if (temPtr != nullptr && temPos == position) return temPtr;   // 找到
    else return nullptr;                                           // position 越界或为负
}
```

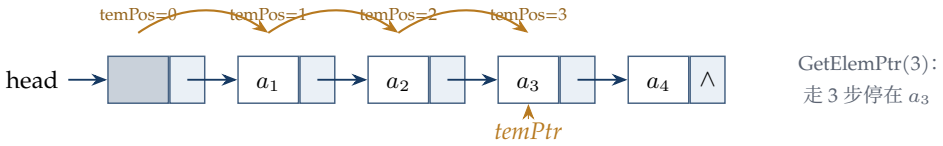


图 4.5: GetElemPtr(3) 的执行过程：工作指针从头结点出发，每走一步 temPos 加 1。

第 9 行为什么还要再判断一次？循环结束有两种可能：走到了表尾（temPtr == nullptr），或者走够了步数（temPos == position）。你在代码注释里还指出了第三种情况：若 position 为负数，循环一次也不执行，temPos=0 不等于 position，于是返回 nullptr——判断写成 temPos == position 正好把它挡住。

复杂度：最坏走 n 步，平均约 $n/2$ 步，$O(n)$。GetElem(i,e)、Length()、Traverse 也都要从头走一遍，都是 $O(n)$。

4.2.2 插入：先武装自己，再融入队伍

在第 i 个位置插入 e：先找到第 $i-1$ 个结点（前驱）p，然后改两个指针。课件 2 P43 总结为“**先武装自己，再融入队伍**”：

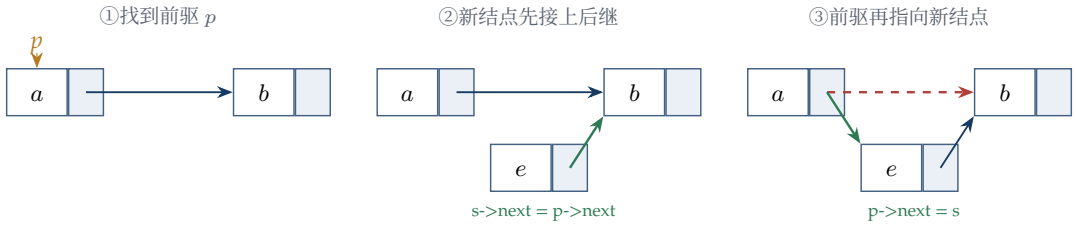


图 4.6: 单链表插入三步（课件 2 P42、P45）：绿色是新建的指针，红色虚线是被替换掉的旧指针。

代码 4.6: 单链表插入：在第 position 个位置插入 e

```
template <class ElemType>
bool SimpleLinkList<ElemType>::Insert(int position, const ElemType &e) {
    if (position < 1 || position > Length() + 1) return false;   // 1 <= position <= Length()+1
    Node<ElemType> *temPtr = GetElemPtr(position - 1);           // ① 找前驱（第 position-1 个）
    Node<ElemType> *newPtr = new Node<ElemType>(e, temPtr->next);// ② 新结点先接上后继
    temPtr->next = newPtr;                                       // ③ 前驱再指向新结点
    return true;
```

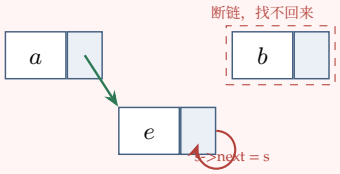
```
8 }
```

第 5 行一句话完成了课件的“四合一”：new 申请结点、写入 data、写入 next = p->next。拆开写就是课件 2 P44 的 tmp = new Node<T>; tmp->data = e; tmp->next = p->next; p->next = tmp;。

易错：两句的顺序不能反（你在代码注释里写了“顺序不能反了!!!!”）

若先执行 p->next = s;，原来的后继 b 的地址就丢了；再执行 s->next = p->next; 时，p->next 已经是 s，于是 s->next = s——新结点指向自己，b 及后面的整条链全部丢失（也无法释放，造成内存泄漏）。

记法：先改新结点（它的指针改了不影响任何人），再改链上的结点。



4.2.3 删除：摆龙门阵，找核心操作

删除第 i 个结点：同样先找前驱 p，让它“绕过”待删结点 q，再释放 q。课件 2 P47 的“删除总结”：①记住待删结点地址；②将值为 x 的结点旁路掉；③回收原来 x 结点占用的空间。

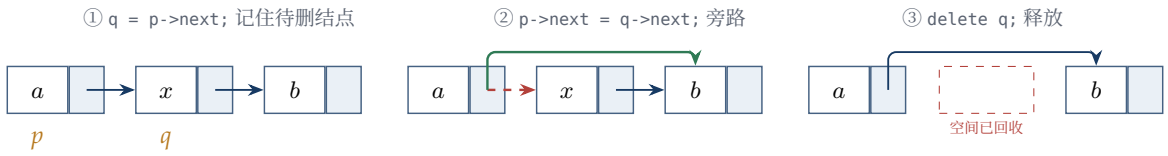


图 4.7: 单链表删除三步（课件 2 P46–P48）。

代码 4.7: 单链表删除：删除第 position 个结点，用 e 返回其值

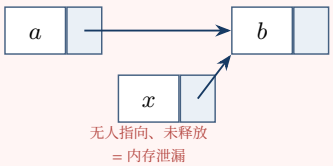
```
1  template <class ElemType>
2  bool SimpleLinkList<ElemType>::Delete(int position, ElemType &e) {
3      if (position < 1 || position > Length()) return false;   // 1 <= position <= Length()
4      Node<ElemType> *temPtr = GetElemPtr(position - 1);   // 前驱
5      Node<ElemType> *nextPtr = temPtr->next;              // ① 记住待删结点
6      temPtr->next = nextPtr->next;                        // ② 前驱“绕过”它
7      e = nextPtr->data;                                   //     带回被删的值
8      delete nextPtr;                                      // ③ 释放空间
9      return true;
10 }
```

易错提醒

错误 1：只写 p->next = p->next->next;，不保存也不释放被删结点——结点永远占着内存，这叫**内存泄漏**。

错误 2：先 delete q; 再 p->next = q->next;——q 已被释放，再读 q->next 是访问“悬空指针”，行为未定义。

错误 3：位置检查写成 position <= Length()+1——删除没有“表尾之后”这个位置（图 2.3）。



定理 / 结论（单链表插入、删除的时间复杂度，笔记 Memo 553 补充前提）

已知前驱结点指针 p 时，插入、删除只改常数个指针，与表长无关，$T = O(1)$（课件 2 P44、P47）。**只给位序** i 时，要先调用 GetElemPtr(i-1) 找前驱，$O(n)$；所以 Insert(i,e)、Delete(i,e) 整体是 $O(n)$，但**不移动任何元素**。

与顺序表比较：两者按位序插入都是 $O(n)$，但顺序表的 $O(n)$ 是**移动数据**，链表的 $O(n)$ 是**沿指针查找**。元素很大（比如每个元素是一整条学生记录）时，移动数据的代价远大于走指针。

4.2.4 建立单链表：头插法与尾插法

代码 4.8: 头插法与尾插法建表（教学补全，已编译验证）

```cpp
template <class ElemType>
void SimpleLinkList<ElemType>::CreateHead(const ElemType a[], int n) {   // 头插法
    Clear();
    for (int i = 0; i < n; i++)
        head->next = new Node<ElemType>(a[i], head->next);  // 新结点永远插在头结点之后
}

template <class ElemType>
void SimpleLinkList<ElemType>::CreateTail(const ElemType a[], int n) {   // 尾插法
    Clear();
    Node<ElemType> *tail = head;                            // tail 始终指向当前最后一个结点
    for (int i = 0; i < n; i++) {
        tail->next = new Node<ElemType>(a[i]);              // 新结点 next 默认为 nullptr
        tail = tail->next;                                  // tail 后移
    }
}
```

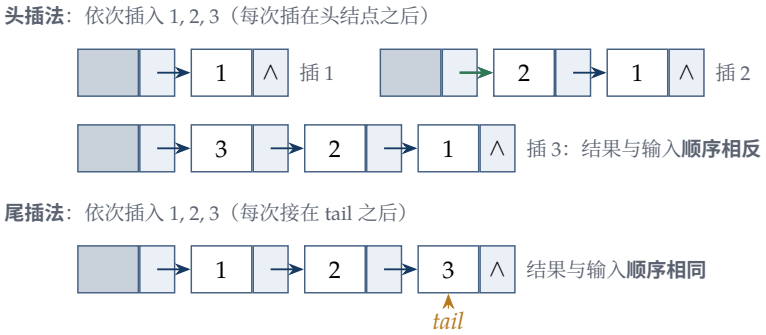


图 4.8: 两种建表方法。程序实测：头插 1,2,3,4 得 4 3 2 1；尾插得 1 2 3 4。两者都是 $O(n)$。

4.2.5 就地逆置：兄弟协同法 + 首席插入法（课件 2 P49–P51）

问题：把带头结点的单链表 (h,e,l,l,o) 逆置为 (o,l,l,e,h)，不新建结点（“就地”，$S(n)=O(1)$）。
思路：把头结点从链上“摘下”，变成一个空表；然后从原首元结点开始，把结点一个个用**头插法**插回去——头插法天然会把顺序倒过来（图 4.8）。摘下当前结点 p 之前，必须让“兄弟”q 先记住 p 的后继，否则后面的链就断了。

代码 4.9: 单链表就地逆置（课件 2 P51，改写为 C++ 成员函数）

```cpp
template <class ElemType>
void SimpleLinkList<ElemType>::Reverse() {
    Node<ElemType> *p = head->next, *q;   // p: 待处理的结点; q: p 的“兄弟”（后继）
    head->next = nullptr;                 // 头结点先“断开”，形成一个空表
    while (p != nullptr) {
        q = p->next;                      // ① 兄弟先记住后面的路
        p->next = head->next;             // ② p 插到头结点之后（首席插入）
        head->next = p;
        p = q;                            // ③ 处理下一个
    }
}
```

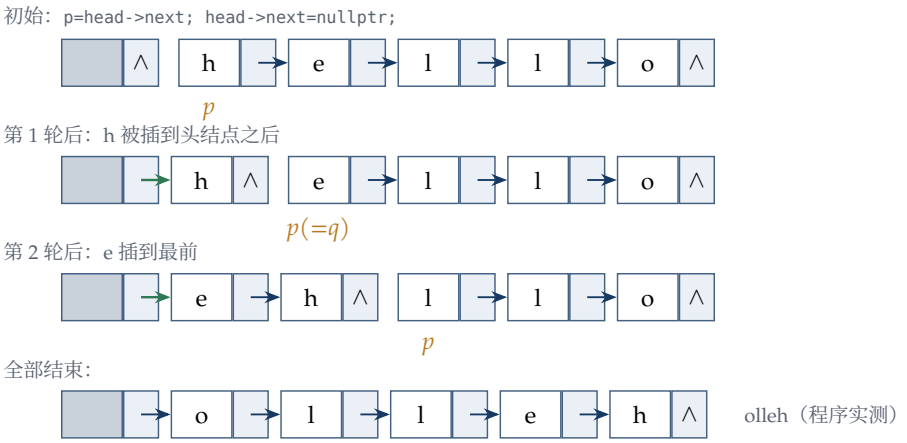
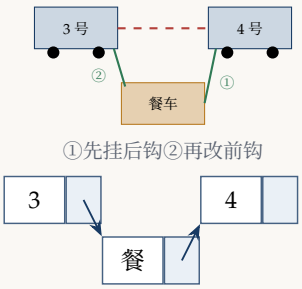


图 4.9: 逆置的“摆龙门阵”：左边是已逆置好的表，右边是还没处理的结点，每轮把 p 从右边搬到左边最前面。

复杂度：每个结点处理一次，$T(n)=O(n)$；只用了 `p`、`q` 两个指针，$S(n)=O(1)$。空表时 `p` 一开始就是 `nullptr`，循环不执行，安全。

生活中的它　火车车厢的挂接与摘除

场景：调车场要在 3 号与 4 号车厢之间加挂一节餐车。
对应：车厢 = 结点；车钩 = 指针；火车头 = 头结点。
小判断：正确做法是先把餐车的后钩挂上 4 号车厢（`s->next = p->next`），再把 3 号的车钩改挂餐车（`p->next = s`）。若先把 3 号摘开挂餐车，4 号以后的车厢就“溜走”了。摘车厢同理：先让 3 号挂上 5 号，再把 4 号拖走（`delete`）。整个过程其他车厢一节都不用挪——这就是链表插删不移动元素。
边界：真火车车厢有物理长度，找到第 100 节要走很远；链表找第 i 个结点也要 $O(n)$。



对照：头插法会逆序 ↔ 第 3 章的栈（后进先出）（思想重现型，同课程跨章）

一句话定位：头插法总在表头插入，读出时从表头读，最后插入的最先读到——这正是**栈**的“后进先出”（LIFO）。

头插法 / 逆置中的结论	迁移到栈	理由或反例
总在表头插入、从表头读出	✓	链栈：入栈 = 在表头插入，出栈 = 删除表头
输入 1,2,3,4 读出 4,3,2,1	✓	依次入栈再全部出栈，序列反转
插入删除 $O(1)$	✓	链栈的入栈出栈都是 $O(1)$
可以在任意位置插入	✗	栈只允许在栈顶一端操作，是“受限的线性表”

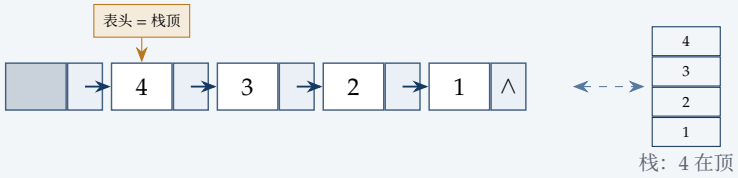


图 4.10: 头插法建成的链表 = 依次入栈后的链栈。

4.3　循环链表

单链表有一个不便：从中间某个结点出发，只能往后走，到表尾就“走投无路”，前面的结点再也访问不到。**循环链表**让末结点的 next 不再指向空，而是指回头结点（课件 2 P52，笔记 Memo 553–554）。

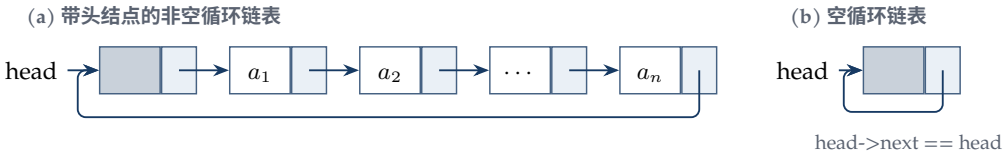


图 4.11: 带头结点的单循环链表。优点：从表中任一结点出发，都可以顺 next 访问到所有结点。

循环链表的操作与单链表几乎一样，差别只在“**怎样判断走完一圈**”。你在「循环列表.cpp」和「双向链表.cpp」末尾总结过，这里整理成表：

	单链表（带头结点）	循环链表（带头结点）
构造空表	`head->next = nullptr;`	`head->next = head;`
判空	`head->next == nullptr`	`head->next == head`
遍历结束条件	`p != nullptr`	`p != head`
`GetElemPtr` 出发点	从 `head`（第 0 个）出发	从 `head->next`（第 1 个）出发，第 0 个单独返回

表 4.1: 单链表与循环链表的代码差别（终止条件：判断 `p` 或 `p->next` 是否等于头指针）。

为什么循环链表的 `GetElemPtr` 不能像单链表那样从 `head` 出发？你的注释说得很准确：单链表“走到头”的标志是 `nullptr`，头结点不是 `nullptr`，所以可以从头结点出发；循环链表“走到头”的标志恰好就是**回到头结点**，起点和终点是同一个结点，若从 `head` 出发，循环条件 `temPtr != head` 一开始就不成立。所以先从 `head->next` 出发，把第 0 个（头结点）单独处理：

代码 4.10: 循环链表的 GetElemPtr（课件 2 P55，与单链表不同的部分）

```
template <class ElemType>
Node<ElemType> *SimpleCircLinkList<ElemType>::GetElemPtr(int position) const {
    if (position == 0) return head;              // 头结点的序号为 0，单独处理
    Node<ElemType> *temPtr = head->next;         // 从首元结点出发
    int temPos = 1;
    while (temPtr != head && temPos < position) {   // 单链表这里是 temPtr != nullptr
        temPtr = temPtr->next;
        temPos++;
    }
    if (temPtr != head && temPos == position) return temPtr;
    else return nullptr;
}
```

4.3.1　用尾指针表示单循环链表（笔记 Memo 554）

若只设头指针，找 a_1 是 $O(1)$，找 a_n 却要走一圈 $O(n)$——“不方便”。若改设**尾指针** R 指向表尾结点，则：a_n 就是 R；a_1 是 `R->next->next`（有头结点）或 `R->next`（无头结点）。两头都是 $O(1)$。笔记的红字也提到：为了循环方便，不带头结点的单循环链表居多（课件 2 P53）。

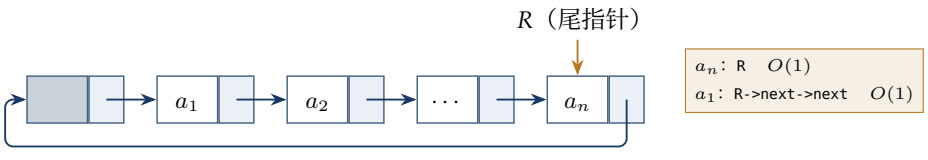


图 4.12: 只设尾指针的带头结点单循环链表：`R->next` 是头结点，再走一步就是首元结点。

例题 4.1　两个带尾指针的循环链表合并（笔记 Memo 554，已按纠错④补全）

问题：A、B 是两个带头结点、只设尾指针 `Ta`、`Tb` 的单循环链表，把 B 接到 A 后面合成一个循环链表。

思路（笔记四步）：① p 存 A 的表头结点；② B 的首元结点连到 A 的表尾；③释放 B 的表头结点；④修改指针，使 B 的表尾指回 A 的头结点。

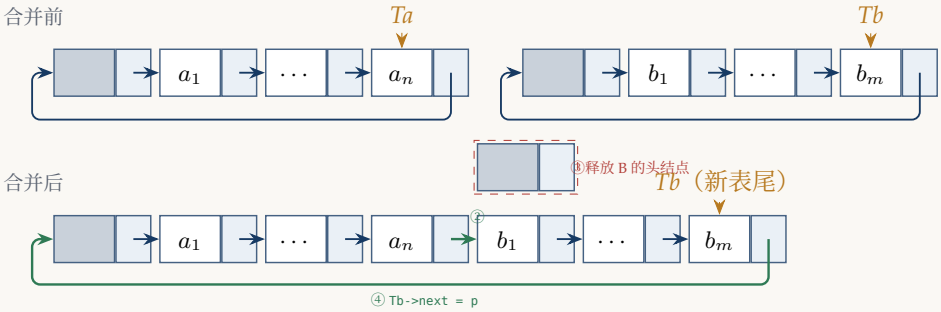


图 4.13: 带尾指针的循环链表合并：只改 2 个指针、释放 1 个结点，与表长无关。

```
Node<T> *p = Ta->next;          // ① p 记住 A 的头结点
Ta->next = Tb->next->next;      // ② A 的表尾接到 B 的首元结点
delete Tb->next;                // ③ 释放 B 的头结点
Tb->next = p;                   // ④ B 的表尾指回 A 的头结点，闭合成环（笔记漏了这一句）
// 合并后新表的尾指针是 Tb
```

复杂度：$O(1)$。若只有头指针，要先各走一圈找表尾，$O(n+m)$。程序实测：(1,2,3) 与 (4,5) 合并得 1 2 3 4 5。

本题想说明：②③的顺序也不能反——必须先用 `Tb->next->next` 取出 B 的首元结点地址，再释放 B 的头结点。

4.3.2　循环链表的应用：约瑟夫问题（课件 2 P57–P58）

n 个人围成一圈，从第 1 个人开始顺时针报数，报到 m 的人出列；再从下一个人开始报数，报到 m 的人出列……直到圈中只剩一人，此人为优胜者。“围成一圈”正是循环链表的天然模型。（这个问题以古罗马时期的历史学家约瑟夫斯命名，源于他记载的一段经历，后来演化为数学游戏。）



图 4.14: $n=8$，$m=3$ 的出列过程（与课件 2 P57 的图对应）：出列顺序 3, 6, 1, 5, 2, 8, 4，优胜者 7（程序实测）。

课件的写法用 `SimpleCircLinkList` 的 `Insert`、`Delete(pos)` 按位序操作，每次删除都要从头找位置。下面直接用结点指针在环上“走”，每报一次数只走一步，思路更直观：

代码 4.11: 约瑟夫问题：不带头结点的单循环链表（教学补全，已编译验证）

```
void Josephus(int n, int m) {
    // 1. 建立 1..n 的单循环链表（不带头结点），rear 指向最后一个人
    Node<int> *rear = new Node<int>(1);
    rear->next = rear;                                  // 只有一个人时自己指向自己
    for (int k = 2; k <= n; k++) {
        Node<int> *s = new Node<int>(k, rear->next);    // 新结点指向“第一个人”
        rear->next = s;                                 // 接在队尾
        rear = s;
    }
    // 2. 报数出列：pre 始终停在“当前报数人”的前一个
```

```
    Node<int> *pre = rear;
    for (int out = 1; out < n; out++) {                // 共出列 n-1 人
        for (int j = 1; j < m; j++) pre = pre->next;   // 走 m-1 步，pre 停在第 m 个人之前
        Node<int> *q = pre->next;                      // q 报到 m
        cout << q->data << " ";
        pre->next = q->next;                           // 删除 q（需要它的前驱，所以用 pre）
        delete q;
    }
    cout << "\n优胜者: " << pre->data << endl;
    delete pre;                                        // 只剩一个结点
}
```

为什么用 pre 而不是直接指着报数的人？单链表删除需要前驱。让 pre 停在“将要出列者”之前，删除就是 $O(1)$。**复杂度：**共出列 $n-1$ 人，每人走 $m-1$ 步，$T=O(nm)$。

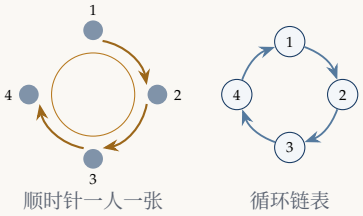
生活中的它　丢手绢与圆桌发牌

场景：小朋友围成一圈玩丢手绢；或者四个人围坐打牌，发牌员按顺时针一人一张，发完一轮接着从第一个人发。

对应：每个人 = 结点；“我左手边是谁” = next；转完一圈回到起点 = 尾结点指回头部；有人出局 = 删除结点。

小估算：54 张牌发给 4 个人，第 54 张发给谁？$54=4\times13+2$，转 13 圈后再发 2 张，发给第 2 个人。

边界：真实游戏里有人中途加入，循环链表同样可以插入；但“第 54 张给谁”用模运算一步算出，而链表要真的走 54 步（见下面的对照）。

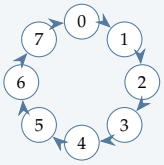


顺时针一人一张　　循环链表

对照：循环链表 ↔ 数论中的模运算（“时钟算术”）（类比同构型）

一句话定位：在 n 个结点的环上从位置 k 向后走一步，相当于 $k\mapsto(k \bmod n)+1$；环形缓冲区写指针 $w\leftarrow(w+1) \bmod N$ 是同一个想法用数组实现。

模运算中的结论	迁移	理由或反例
从任一剩余类出发，反复 $+1$ 能遍历全部 n 个类	✓	从任一结点出发可访问全部结点
走 k 步与走 $k \bmod n$ 步落在同一位置	✓	环上走一整圈回到原处
$k \bmod n$ 一步算出，$O(1)$	✗	链表上走 k 步就要执行 k 次 p=p->next，$O(k)$；链表不能随机跳转
模数 n 固定	△	约瑟夫问题中每出列一人，环长就减 1，若用数组 + 取模模拟，必须用**当前**人数取模



$w\leftarrow(w+1) \bmod 8$

环形缓冲区：数组 + 取模
循环链表：结点 + 尾指回头
同一思想，两种实现

图 4.15: “首尾相接”的两种实现。

4.4　双向链表与双向循环链表

> **为什么需要它**　单链表（含循环链表）只能沿 next 向后走。要找某结点的**前驱**，单链表必须从头找，$O(n)$；循环链表也要绕一圈。**双向链表**在每个结点里再增加一个指向直接前驱的指针域，链表中就有了两个方向不同的链（课件 2 P59，笔记 Memo 554–555）。

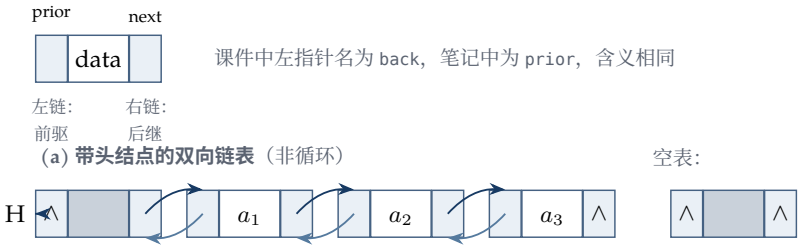


图 4.16: 双向链表的结点结构与非循环形式（笔记 Memo 555 图示）。

课件 2 P59 指出：双向链表通常采用**带头结点的循环链表**形式，即**双向循环链表**：①让头结点的前驱指针指向链表的最后一个结点；②让最后一个结点的后继指针指向头结点（笔记 Memo 555）。

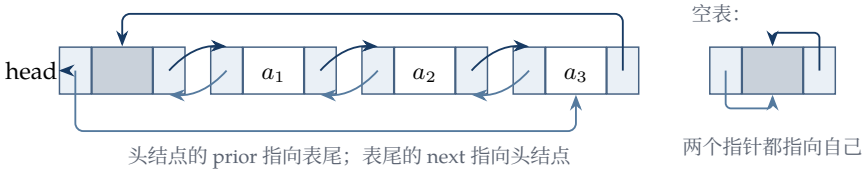


图 4.17: 带头结点的双向循环链表（课件 2 P60）。你的代码构造空表：`head->next = head; head->back = head;`。

定理 / 结论（双向链表的对称性，笔记 Memo 555）

对双向链表中任一结点 p（双向循环链表中任一结点，含头结点）：

$$p\text{->prior->next} = p = p\text{->next->prior}$$

即“我前驱的后继是我，我后继的前驱也是我”。

对称性是所有双向链表操作的检验标准：插入、删除之后，受影响的结点都必须重新满足这个等式。你的代码注释也提到：只要指针一直正确维护，`head->next == head` 与 `head->back == head` 一定同时成立或同时不成立。

4.4.1　插入：四个指针，顺序有讲究

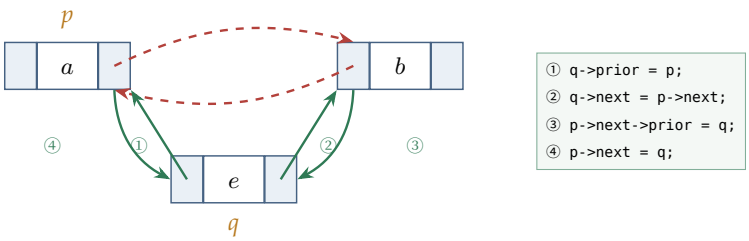


图 4.18: 在 p 之后插入 q（课件 2 P67 单选 6 的正确选项 C）：先“武装”q 的两只手（①②），再改 p 的后继的左手（③），最后改 p 的右手（④）。

代码 4.12: 双向循环链表：在 p 之后插入 q（教学补全，已编译验证）

```
template <class T>
void InsertAfter(DblNode<T> *p, DblNode<T> *q) {
    q->prior = p;              // ① q 的左手拉住 p
```

```
    q->next = p->next;          // ② q 的右手拉住 p 原来的后继
    p->next->prior = q;         // ③ 原后继的左手改拉 q       —— 必须在 ④ 之前!
    p->next = q;                // ④ 最后 p 的右手改拉 q
}
```

为什么③必须在④之前？ ③要通过 `p->next` 找到原后继 b。若先执行④，`p->next` 已经变成 q，③就成了 `q->prior = q`——b 的左指针没改，q 的左指针反被改错。课件的类模板 `Insert(position, e)` 先用 `nextPtr` 保存了后继，所以它那两句的顺序可以随意（见你的「双向链表.cpp」）。

4.4.2 删除：只需两句

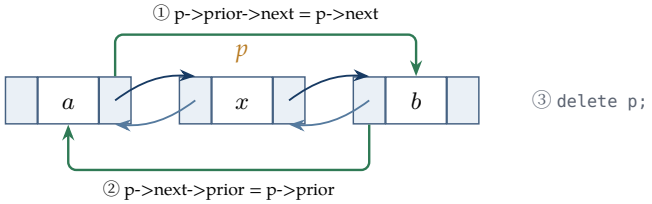


图 4.19: 删除 p 所指结点（课件 2 P66 单选 5 的正确选项 A）：前驱的右手越过 p，后继的左手越过 p。

代码 4.13: 双向循环链表：删除 p 所指结点

```
template <class T>
T DeleteNode(DblNode<T> *p) {
    p->prior->next = p->next;   // ① 前驱的右手越过 p
    p->next->prior = p->prior;  // ② 后继的左手越过 p（①② 顺序可交换：都没改 p 自己的指针）
    T e = p->data;
    delete p;                   // ③ 最后释放
    return e;
}
```

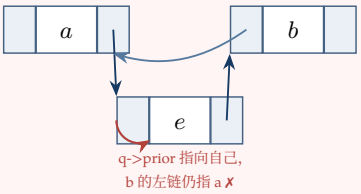
复杂度（澄清笔记 Memo 555）： `Length()`、`GetElem()` 等只涉及一个方向的指针，与单链表相同，$O(n)$。插入、删除要同时修改两个方向的指针，但修改本身只有常数步：已知 p 时是 $O(1)$；按位序操作时，$O(n)$ 花在找位置上。双向链表的真正好处是**已知 p 就能 $O(1)$ 删除 p 自己**（单链表必须知道前驱）。

易错提醒

错误 1： 插入时先写 `p->next = q;` 再写 `p->next->prior = q;`——第二句实际修改的是 `q->prior`（课件 P67 选项 D 的错误）。

错误 2： 只改了右链没改左链。程序正向遍历正常，反向遍历就乱了。用“对称性”检查：插入后 q、p、b 三个结点都要满足 `x->prior->next == x`。

错误 3： 在非循环双向链表的表尾插入时写 `p->next->prior = q;`，而 `p->next` 是 `nullptr`——空指针解引用。双向**循环**链表没有这个特殊情况，这也是课件推荐循环形式的原因。



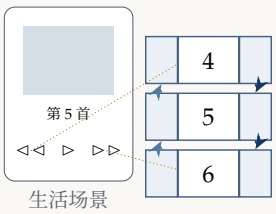
生活中的它　音乐播放器的“上一首 / 下一首”

场景：手机音乐 App 里，当前播放第 5 首，你可以点“上一首”或“下一首”，开启“列表循环”后，最后一首的下一首是第一首。

对应：每首歌 = 结点；“下一首” = `next`；“上一首” = `prior`；列表循环 = 双向**循环**链表。

小判断：从当前歌删掉这首（不喜欢）后，能直接播放前一首吗？双向链表能：`p->prior` 一步得到，删除也 $O(1)$；单链表要从头找前驱。

边界：真实播放器可能用数组 + 下标实现，这里只说明双向链接的逻辑。



对照：单链表 / 双向链表 ↔ 离散数学中的有向边 / 无向边（类比型）

一句话定位：单链表的每个 `next` 像一条**有向边**，只能从前往后走；双向链表相当于每对相邻结点之间有一条“可以双向走”的边——但在内存里它是用**两条**方向相反的指针实现的。

图论中的结论	迁移	理由或反例
有向路径只能沿箭头方向走	✓	单链表只能沿 next 走
无向边两端互为邻居（邻接关系对称）	✓	对称性 p->prior->next $= p$
一条无向边只算一条、存一次	△	双向链表每对相邻结点要存 **2** 个指针，修改时两个都要改，漏改一个就“不对称”
图中一个点可以有多个邻居	✗	链表中每个结点最多一个前驱、一个后继，仍是线性结构



图 4.20: “双向”在内存中是两条方向相反的指针。

4.5　补充：在链表中保存当前位置和元素个数（课件 2 P61–P63）

前面的链表每次 `GetElem(i)` 都从头结点开始走，`Length()` 也要走一遍。若应用程序常常连续访问相邻位置（比如依次取第 5、6、7 个元素），每次都从头走就很浪费。改进：在链表对象中保存**当前位置序号** `curPosition`、**指向当前位置的指针** `curPtr` 和**元素个数** `count`。

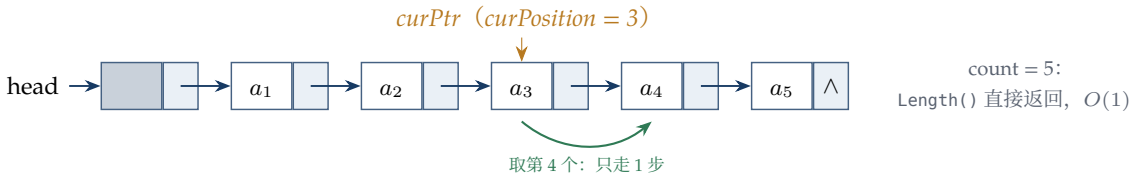


图 4.21: 保存当前位置后，向后访问可以从 `curPtr` 接着走，不必回到头结点。

课件的类模板 `LinkList` 把 `curPosition`、`curPtr` 声明为 `mutable`。原因：`GetElem(...) const` 是只读函数，按 `const` 规则不能修改成员；但“记住刚才访问到哪里”并不改变线性表的内容，`mutable` 允许在 `const` 成员函数中修改这类“缓存”成员。`count` 则在每次插入、删除时同步加减，`Length()` 变为 $O(1)$。

4.6　单链表、循环链表和双向链表的时间效率比较

笔记 Memo 555–556 比较了几种链表“找表头、找表尾”的代价，整理并配图如下（L 为头指针，R 为尾指针，均带头结点）：

链表形式	找首元结点	找表尾结点	找 p 的前驱
带头结点的单链表 L	`L->next`，$O(1)$	从 `L->next` 向后，$O(n)$	$O(n)$
带头结点、仅设头指针的单循环链表 L	`L->next`，$O(1)$	从 `L->next` 向后，$O(n)$	$O(n)$（绕一圈）
带头结点、仅设尾指针的单循环链表 R	`R->next->next`，$O(1)$	`R`，$O(1)$	$O(n)$
带头结点的双向循环链表 L	`L->next`，$O(1)$	`L->prior`，$O(1)$	`p->prior`，$O(1)$

表 4.2: 各种链表的时间效率（笔记 Memo 555–556，增加“找前驱”一列）。

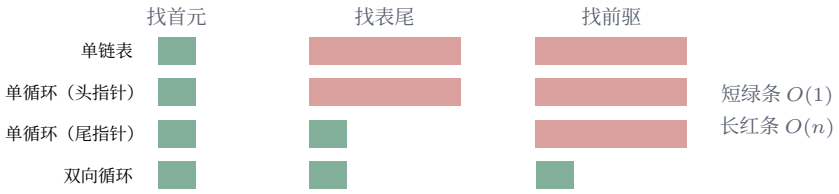


图 4.22: 表 4.2 的直观对照：从上到下，用越来越多的指针换来越来越多的 $O(1)$。

本章小结

①链表用指针表示逻辑关系，结点位置任意；头结点让空表与非空表、首位与其他位置统一处理。
②单链表所有操作都要先“走到”位置：查找 $O(n)$；已知前驱时插入删除 $O(1)$。插入“先连后断”，删除“先记住、再旁路、后释放”。
③头插法逆序、尾插法保序；逆置 = 摘下头结点 + 逐个头插。
④循环链表：终止条件改为 `p != head`；尾指针使两端都 $O(1)$，合并只需 $O(1)$。
⑤双向链表：对称性 p->prior->next $= p =$ p->next->prior；插入四步中“改后继的左指针”必须在“改 p 的右指针”之前。

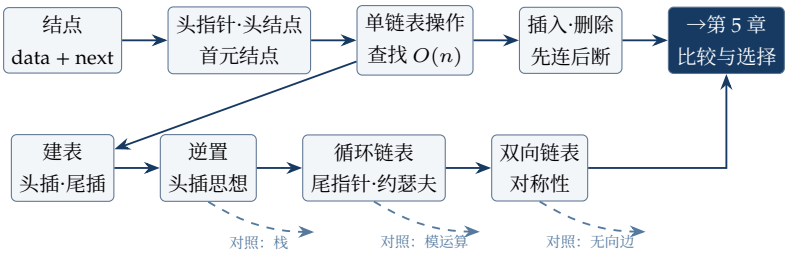


图 4.23: 第 4 章知识地图。

学完这里，你应该能够回答

1. 头指针、头结点、首元结点分别是什么？加头结点有什么好处？带头结点和不带头结点的空表分别怎样判断？
2. 在 `p` 之后插入 `s` 的两句为什么不能交换？删除时为什么要先保存 `q`？
3. 头插法、尾插法建成的表次序有什么不同？逆置为什么只需 $O(1)$ 额外空间？
4. 循环链表的遍历终止条件是什么？为什么只设尾指针更方便？
5. 写出双向链表的对称性等式，以及在 `p` 后插入 `q` 的四句，并说明哪两句不能交换。

第 5 章　顺序表与链表的比较

前两章分别解决了什么：顺序表——连续存放、随机存取；链表——指针连接、插删不移动。**本章回答：**两者在空间、时间上的差别从何而来？遇到具体问题选哪一个？

5.1　从两个维度比较

下表依据青岛大学《数据结构与算法基础》2.6 节截图整理，并补充了“为什么”一栏。

比较项目		顺序表	链表	根源
空间	存储空间	预先分配，会导致空间闲置或溢出	动态分配，不会出现存储空间闲置或溢出	是否要求一整块连续空间
	存储密度	不用为表示结点间的逻辑关系而增加额外的存储开销，存储密度等于 1	需要借助指针来体现元素间的逻辑关系，存储密度小于 1	关系靠位置还是靠指针
时间	存取元素	随机存取，按位置访问元素的时间复杂度为 $O(1)$	顺序存取，按位置访问元素的时间复杂度为 $O(n)$	地址能否直接算出
	插入、删除	平均移动约表中一半元素，时间复杂度为 $O(n)$	不需移动元素，确定插入、删除位置后，时间复杂度为 $O(1)$	逻辑次序是否绑定物理次序
适用情况		①表长变化不大，且能事先确定变化的范围；②很少进行插入或删除操作，经常按元素位置序号访问数据元素	①长度变化较大；②频繁进行插入或删除操作	

表 5.1: 顺序表与链表的比较。

你的笔记 Memo 556 写下了链式存储的优缺点：优点①结点空间可动态申请与释放；②插入、删除时不需移动数据元素。缺点①存储密度小，每个结点的指针域需额外占用存储空间；②是非随机存取结构，增加了算法的复杂度。

存储密度（教学补全）

$$\text{存储密度} = \frac{\text{结点中数据元素本身占用的空间}}{\text{结点占用的空间总量}}$$

算一算：在 64 位系统上用本讲义的 `Node<int>`：`int` 占 4 字节、指针占 8 字节，编译器为了对齐把结点补齐到 `sizeof(Node<int>)`$= 16$ 字节（用 g++ 实测）。存储密度 $= 4/16 = 0.25$。双向链表 `DblNode<int>` 是 24 字节，密度只有 $4/24 \approx 0.17$。若元素是一条 200 字节的学生记录，单链表结点约 208 字节，密度约 0.96——**元素越大，指针的额外开销越不明显**。顺序表的密度“等于 1”是只算已用部分；若容量 100 只存了 10 个，从空间利用率看只有 10%。



图 5.1: 存储密度的直观对照（`int` 元素，64 位系统）。

5.2　怎样选择

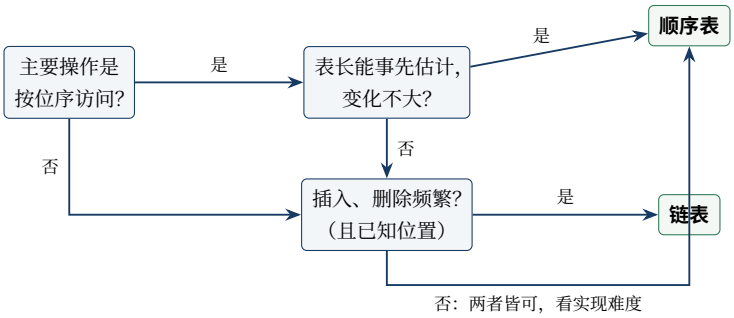


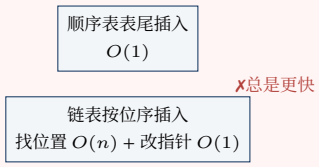
图 5.2: 选择存储结构的判定流程（依据表 5.1 的“适用情况”）。

例题 5.1　三个场景各选什么

(1) 学校全体学生的成绩表，学期中基本不变，经常按学号顺序号查第 k 个人——**顺序表**（随机存取 $O(1)$，表长稳定）。
(2) 文本编辑器中按行存放文章，用户随时在任意行之间插入、删除行，行数变化很大——**链表**（插删不移动、长度可变）。
(3) 约瑟夫问题——**循环链表**（反复删除、首尾相接）。
本题想说明：没有“更好”的结构，只有“更适合当前操作”的结构。这也是第 1 章“实现依赖于存储结构”的直接应用。

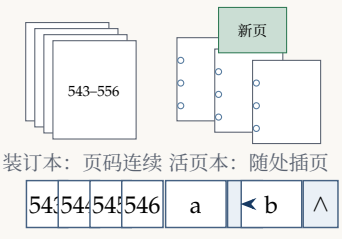
易错提醒

错误想法：“链表插入删除是 $O(1)$，所以链表总比顺序表快。”
错在哪：$O(1)$ 的前提是“已经确定插入、删除位置”（截图原文）。按位序插入时链表也要先 $O(n)$ 地找位置；而且顺序表在表尾插入、删除同样是 $O(1)$。比较要**在同样的前提下**进行。



生活中的它　装订笔记本 vs 活页本

场景：你的数据结构笔记写在装订本上（Memo 543–556 页码连续）；也可以用活页本，每页可拆。
对应：装订本 = 顺序表：知道页码直接翻到（随机存取），但想在 545 和 546 之间加一页，只能把后面全部重抄；页数到头就写不下了（表满）。活页本 = 链表：任意位置插一页，其他页不动；但若活页没印页码，只靠“这页后面接哪页”串起来，找第 10 页只能从头一页页数。
小判断：期末复习要反复按页码查，又不再增删——用装订本（顺序表）。
边界：活页本实际上有页码可直接翻，比纯链表方便；这里只取“插页容易”这一点类比。



对照：顺序表 / 链表 ↔ 计算机组成中的随机存取存储器 / 顺序存取存储器（同名异义型）

一句话定位：两门课都用“随机存取 / 顺序存取”这对词，含义一致：访问时间与位置无关，还是要从头依次经过。内存（RAM）是随机存取的；磁带是顺序存取的。

计算机组成中的结论	迁移	理由或反例
RAM 中任一地址的访问时间相同	✓	顺序表 `GetElem` 为 $O(1)$
磁带要绕到指定位置才能读写	✓	链表要从头指针走到第 i 个结点
链表存放在 RAM 里，所以链表也是随机存取	✗	存储**介质**可以随机访问，但链表不知道第 i 个结点的**地址**，只能沿指针找——“随机存取”说的是数据结构能否直接算出地址
顺序存取介质就不能插入	△	链表在已知位置插入是 $O(1)$；磁带中间插入数据却很困难——两者只在“访问方式”上类似



图 5.3: “随机存取”与“顺序存取”在两门课中的含义一致。

学完这里，你应该能够回答

1. 从“空间”“时间”两个维度说出顺序表与链表的差别，并说出差别的根源。
2. 计算存储密度：元素为 `double`（8 字节），64 位系统单链表结点 16 字节，密度是多少？
3. 什么前提下链表的插入删除才是 $O(1)$？

正式练习（38 题）

练习分三层：A 基础巩固 25 题　B 中等偏下提升 10 题　C 跨学科迁移 3 题。题目中的图只给出题目已知的信息；答案与带标注的解析图集中在后面。请先独立完成。

知识点—题目对应表

核心知识点	基础题	提升题	跨学科	涉及的对照
C1 数据结构与逻辑结构	8, 9	—	—	
C2 存储结构	4, 9	—	—	逻辑/物理地址
C3 / C7 线性表定义与 ADT 操作	1, 3, 13	—	—	数列 / 集合
C4 算法特性	14	—	—	算法与程序
C5 时间复杂度	7, 17, 20, 22	27	—	无穷大的阶
C6 空间复杂度	—	30	—	
C8 地址公式	2, 11	31	38	等差数列
C9 顺序表插入删除	3, 13, 16, 21	—	—	数学期望
C10 顺序表应用	—	33, 34	—	向量加法
C11 单链表结构	6, 12, 19	—	—	forward_list
C12 单链表操作	5, 18, 23, 24, 25	30	—	栈
C13 循环链表	15	28, 32	36	模运算
C14 双向链表	10	26, 35	—	无向边
C15 顺序表与链表比较	11	29	37	RAM 与磁带

A. 基础巩固训练（25 题）

A1　单项选择题（10 题）

1. 线性表 $L=(a_1,a_2,\ldots,a_n)$，下列说法正确的是（　）。



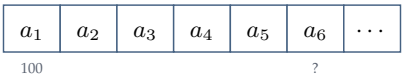
A. 每个元素都有一个直接前驱和一个直接后继

B. 线性表中至少有一个元素

C. 表中诸元素的排列必须由小到大或由大到小

D. 除第一个和最后一个元素外，其余每个元素都有且仅有一个直接前驱和直接后继

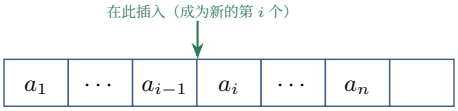
2. 顺序表第一个元素的存储地址是 100，每个元素占 4 个存储单元，则第 6 个元素的地址是（　）。



A. 120　　B. 124

C. 104　　D. 116

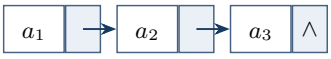
3. 在长度为 n 的顺序表的第 i 个位置（$1\le i\le n+1$）插入一个元素，需要向后移动的元素个数为（　）。



A. $n-i$　　B. $n-i+1$

C. $n-i-1$　　D. i

4. 线性表若采用链式存储结构，要求内存中可用存储单元的地址（　）。

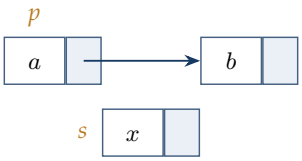


这些结点在内存中的地址关系?

A. 必须是连续的　　B. 部分地址必须是连续的

C. 一定是不连续的　　D. 连续或不连续都可以

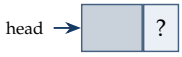
5. 在单链表中，要将 s 所指结点插入到 p 所指结点之后，其语句应为（　）。



A. `s->next=p+1; p->next=s;`　　B. `(*p).next=s; (*s).next=(*p).next;`

C. `s->next=p->next; p->next=s->next;`　　D. `s->next=p->next; p->next=s;`

6. 带头结点的单链表 head 为空表的判定条件是（　）。



A. `head==nullptr`　　B. `head->next==nullptr`

C. `head->next==head`　　D. `head!=nullptr`

7. 下面程序段的时间复杂度是（　）。

```
for (int i = 1; i <= n; i *= 2)
    s++;
```

循环：i 从 1 开始，每次 i*=2，条件 $i \le n$
循环体 s++

A. $O(n)$　　B. $O(\log_2 n)$

C. $O(n\log_2 n)$　　D. $O(n^2)$

8. 在数据结构中，从逻辑上可以把数据结构分成（　）。

A. 动态结构和静态结构　　B. 紧凑结构和非紧凑结构

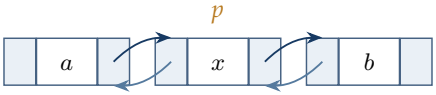
C. 线性结构和非线性结构　　D. 内部结构和外部结构

9. 与数据元素本身的形式、内容、相对位置、个数无关的是数据的（　）。

A. 存储结构　　B. 存储实现

C. 逻辑结构　　D. 运算实现

10. 在双向链表中，删除 p 所指的结点时须修改指针（　）。



A. `p->next->prior=p->prior; p->prior->next=p->next;`

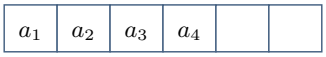
B. `p->next=p->next->next; p->next->prior=p;`

C. `p->prior->next=p; p->prior=p->prior->prior;`

D. `p->prior=p->next->next; p->next=p->prior->prior;`

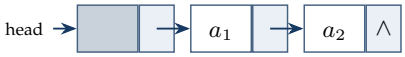
A2　多项选择题（5 题，每题有两个或两个以上正确选项）

11. 关于顺序表，下列说法正确的有（　）。



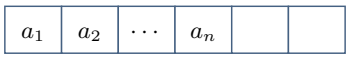
A. 逻辑上相邻的元素，物理位置也相邻
B. 可以随机存取表中任一元素
C. 插入、删除元素时不需要移动其他元素
D. 不计未使用的空间时，存储密度等于 1

12. 关于带头结点的单链表，下列说法正确的有（　）。



A. 头结点就是线性表的第一个元素
B. 头结点使空表与非空表的处理得到统一
C. 头结点的数据域可以不存信息，也可以存表长等附加信息
D. 带头结点的单链表为空的条件是 `head==nullptr`

13. 对一个长度为 n 且未满的顺序表，下列操作中时间复杂度为 $O(1)$ 的有（　）。

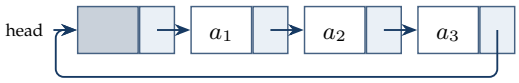


A. 取第 i 个元素
B. 在表尾插入一个元素
C. 在表头插入一个元素
D. 删除第 1 个元素

14. 下列属于算法五个特性的有（　）。

A. 有穷性
B. 确定性
C. 可读性
D. 可行性

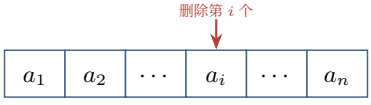
15. 设 `head` 是带头结点的单循环链表的头指针，`p` 是工作指针。下列说法正确的有（　）。



A. 空表的条件是 `head->next==head`
B. 遍历时循环结束条件是 `p!=nullptr`
C. 从表中任一结点出发，都能访问到所有结点
D. 最后一个结点的 `next` 为 `nullptr`

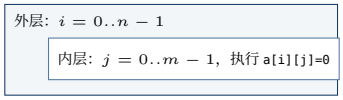
A3　填空题（5 题）

16. 在长度为 n 的顺序表中删除一个元素，若在各个合法位置删除的概率相等，则平均需要移动_____个元素。

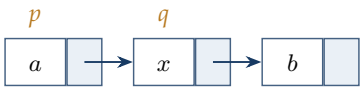


17. 下面程序段的时间复杂度为_____。

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        a[i][j] = 0;
```



18. p 指向单链表中某个结点（不是尾结点），删除 p 的后继结点的语句为：`q = p->next;  p->next =_____;  delete q;`



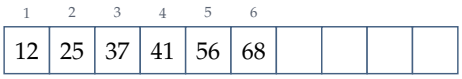
19. 带头结点的单链表如下图，执行 `p = head->next->next;` 后，`p->data` 的值为_____。



20. 若某算法的时间频度 $T(n) = 3n^2 + 5n\log_2 n + 100$，则其时间复杂度为_____。

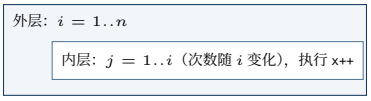
A4　解答题（5 题）

21. 顺序表 $L = (12, 25, 37, 41, 56, 68)$，容量 `maxSize`$= 10$。
(1) 在第 3 个位置插入 30，写出插入后的表，指出哪些元素后移、共移动几次；
(2) 在 (1) 的结果上删除第 5 个元素，写出删除后的表、被带回的值和移动次数。



22. 求下面程序段中语句 `x++` 的频度，并写出时间复杂度。

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        x++;
```



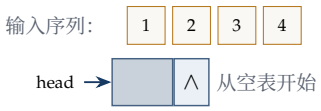
23. 带头结点的单链表 (a, b, c)，p 已指向结点 b。画出在 b 之后插入数据为 x 的新结点的每一步指针变化，并写出对应的 C++ 语句（结点类型用 4.1 节的 `Node<char>`）。



24. 为 4.2 节的 `SimpleLinkList` 写一个成员函数模板 `int CountValue(const ElemType &e) const`，统计带头结点单链表中数据域等于 e 的结点个数，并分析时间复杂度和空间复杂度。



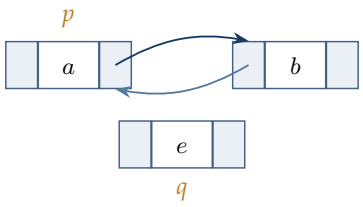
25. 依次输入 1, 2, 3, 4，用**头插法**建立带头结点的单链表，画出最终链表，并说明结点次序与输入次序的关系；若改用**尾插法**，结果如何？



B. 中等偏下提升训练（10 题）

B1　单项选择题（3 题）

26. 在双向循环链表中，在 p 所指的结点之后插入 q 所指的新结点，其修改指针的操作是（　）。



A. `p->next=q; q->prior=p; p->next->prior=q; q->next=q;`

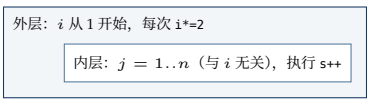
B. `p->next=q; p->next->prior=q; q->prior=p; q->next=p->next;`

C. `q->prior=p; q->next=p->next; p->next->prior=q; p->next=q;`

D. `q->prior=p; q->next=p->next; p->next=q; p->next->prior=q;`

27. 下面程序段的时间复杂度是（　）。

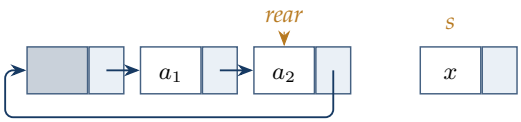
```
for (int i = 1; i <= n; i *= 2)
    for (int j = 1; j <= n; j++)
        s++;
```



A. $O(n)$　　B. $O(n\log_2 n)$

C. $O(n^2)$　　D. $O(\log_2 n)$

28. 设带头结点的单循环链表只设尾指针 `rear`（指向表尾结点）。要把 s 所指的新结点插到表尾，并使 `rear` 仍指向表尾，正确的语句是（　）。



A. `rear->next=s; s->next=rear->next; rear=s;`

B. `s->next=rear->next; rear->next=s; rear=s;`

C. `s->next=rear; rear->next=s; rear=s;`

D. `s->next=rear->next; rear=s; rear->next=s;`

B2　多项选择题（2 题）

29. 关于顺序表与链表的选择，下列说法正确的有（　）。

A. 经常按序号访问元素、很少插入删除时，宜采用顺序表

B. 表长变化较大且频繁插入删除时，宜采用链表

C. 链表用指针表示关系，所以存储密度大于 1

D. 若只在表尾插入且空间足够，顺序表的插入也是 $O(1)$

30. 对带头结点的单链表调用 4.2 节的 `Reverse()`（代码 4.9），下列说法正确的有（　）。



A. 时间复杂度为 $O(n)$

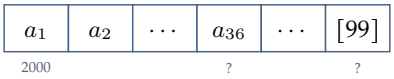
B. 额外空间复杂度为 $O(1)$

C. 若删去语句 `q=p->next;`，并把 `p=q;` 改为 `p=p->next;`，结果会出错

D. 对空表调用会因访问空指针而崩溃

B3　填空题（3 题）

31. 顺序表的 `elems` 首地址为 2000，每个元素占 8 个字节，`maxSize`$= 100$。则 a_{36} 的地址为_____，数组最后一个单元 `elems[99]` 的地址为_____。



32. 约瑟夫问题中 $n = 7$、$m = 3$（从 1 号开始报数）。前三个出列者依次是_____，优胜者是_____。



33. 稀疏多项式用“(系数，指数)”顺序表按指数递增存放：$A = ((5,0),(2,3),(-4,6))$，$B = ((1,1),(4,6),(3,9))$。用 3.4 节的 `AddPoly` 计算 $C = A + B$，则 C 共有_____ 项，$C =$_____。

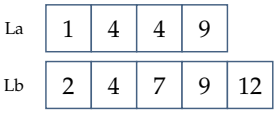
B4　综合大题（2 题）

34. 有序归并。La=(1,4,4,9)，Lb=(2,4,7,9,12)，均按值非递减。

(1) 按“$a \le b$ 时取 a”的保留重复规则（代码 2.3，`allowDup=true`），写出 Lc；

(2) 若改用 `allowDup=false`，Lc 是什么？结果中为什么仍有重复元素？怎样修改才能使 Lc 中元素互不相同？

(3) 设 La、Lb 的长度分别为 m、n，分析 (1) 的时间复杂度，并说明为什么不是 $O(mn)$。

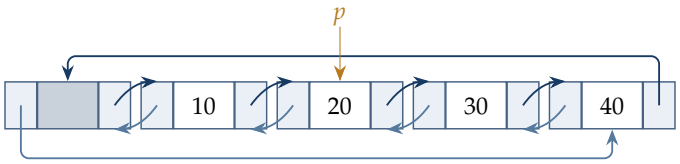


35. 双向循环链表的删除与“前插”。带头结点的双向循环链表 $L=(10,20,30,40)$，`p` 指向 20 所在结点。

(1) 写出删除 `p` 所指结点的 C++ 语句，并画出删除后的链表；

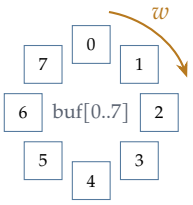
(2) 删除后，设 `r` 指向 30 所在结点，`s` 指向新结点 25。写出把 `s` **插在 `r` 之前**的语句（不得借助其他指针），并说明语句顺序上的注意点；

(3) 分析 (1)(2) 的时间复杂度；若改用单链表实现 (2)，还需要付出什么额外代价？



C. 跨学科迁移训练（3 题）

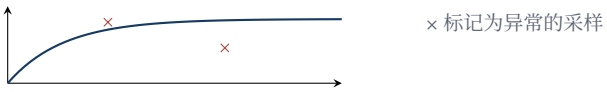
36. 跨学科 · 单选　**（信号采集）**音频采集芯片把采样值写入一个容量 $N=8$ 的**环形缓冲区**（一个长度为 8 的数组）。写指针 w 初值为 0，每写入一个采样就执行 `buf[w] = x; w = (w + 1) % N;`，写满后从头覆盖最旧的数据。连续写入 13 个采样后，w 的值是（　）。



A. 5　　B. 13

C. 4　　D. 6

37. 跨学科 · 多选　**（控制系统实验数据记录）**温度控制实验中，单片机以 100 Hz 采样，每个采样记录 16 字节。实验时长事先不确定；实验结束后要先删除若干被标记为“异常”的中间采样，再按采样序号 k 画出曲线。下列说法正确的有（　）。



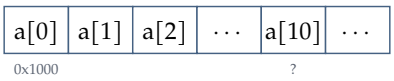
A. 若实验时长未知且需在中间删除多个采样，用链表存储更合适

B. 若主要操作是按序号 k 直接读取第 k 个采样，用顺序表更合适

C. 用顺序表存储时，第 k 个采样的首地址 = 基地址 $+(k-1)\times 16$

D. 链表结点在内存中不连续，因此无法按采样先后次序遍历全部数据

38. 跨学科 · 填空　**（计算机组成）**内存按字节编址，`int` 占 4 字节。C++ 数组 `int a[100];` 的首地址为十六进制 `0x1000`（十六进制中 `0x10` = 16，`0x28` = 40）。则 `a[10]` 的地址为_____（用十六进制表示），它是线性表意义下的第_____ 个元素。



全部答案速查

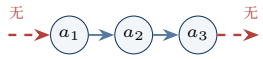
A1 基础单选	1. D　2. A　3. B　4. D　5. D　6. B　7. B　8. C　9. C　10. A
A2 基础多选	11. ABD　12. BC　13. AB　14. ABD　15. AC
A3 基础填空	16. $\dfrac{n-1}{2}$　17. $O(mn)$　18. `q->next`　19. 20　20. $O(n^2)$
A4 基础解答	21. (1) (12,25,30,37,41,56,68)，后移 37、41、56、68 共 4 次；(2) (12,25,30,37,56,68)，带回 41，移动 2 次 22. 频度 $\frac{n(n+1)}{2}$，$O(n^2)$　23. `s=new Node<char>('x'); s->next=p->next; p->next=s;`，结果 (a,b,x,c) 24. 遍历计数，时间 $O(n)$，空间 $O(1)$　25. 头插得 4,3,2,1（与输入相反）；尾插得 1,2,3,4
B1 提升单选	26. C　27. B　28. B
B2 提升多选	29. ABD　30. ABC
B3 提升填空	31. 2280；2792　32. 3, 6, 2；4　33. 4 项；$((5,0),(1,1),(2,3),(3,9))$
B4 综合大题	34. (1) (1,2,4,4,4,7,9,9,12)；(2) (1,2,4,4,4,7,9,12)，La 内部有重复；插入前与 Lc 表尾比较；(3) $O(m+n)$ 35. (1) `p->prior->next=p->next; p->next->prior=p->prior; delete p;`；(2) `s->prior=r->prior; s->next=r; r->prior->next=s; r->prior=s;`，第 3 句须在第 4 句前；(3) 均 $O(1)$，单链表需 $O(n)$ 找前驱
C 跨学科	36. A　37. ABC　38. `0x1028`；11

详细解析：A. 基础巩固

A1　单项选择题解析

第 1 题　答案：**D**　基础　考点：线性表的逻辑特征（C7）

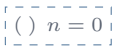
思路　逐条对照线性表的三条逻辑特征：首结点无前驱、尾结点无后继、内部结点恰有一个前驱一个后继；再注意 $n \geq 0$ 与“序列不等于有序”。



A　✗ a_1 无前驱，a_n 无后继



C　✗“序列”是位置先后，不要求数值有序



B　✗ 空表 $n=0$ 也是线性表



D　✓ 内部结点恰有一个前驱、一个后继

判断　A 把首尾也算进去了（左图红虚线：根本不存在）；B 忽略了空表；C 把“序列”误解成“有序”，$(3,1,4,1)$ 无序且重复，仍是线性表；D 正是第三条逻辑特征。

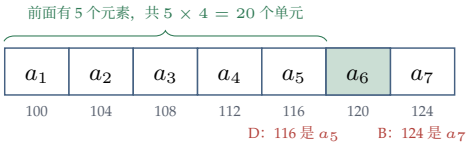
常见错误　选 A——只记住了“一个前驱一个后继”，忘了首尾例外。

一句话　**首无前驱、尾无后继、中间各一个；可以为空、可以无序。**

第 2 题　答案：**A**　基础　考点：地址公式（C8）

公式　$\mathrm{LOC}(a_i) = \mathrm{LOC}(a_1) + (i-1) \times l$，其中 $\mathrm{LOC}(a_1) = 100$，$l = 4$，$i = 6$。

计算　$\mathrm{LOC}(a_6) = 100 + (6-1) \times 4 = 120$。



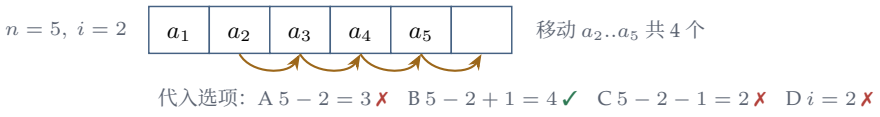
解析图：a_6 前面有 5 个元素，首地址 120。

错误选项　B 124 误用 $i \times l$（相当于 a_7）；C 104 只加了一个元素宽度（a_2）；D 116 是 a_5 的地址。

一句话　**乘的是“前面有几个”，即 $i-1$。**

第 3 题　答案：**B**　基础　考点：顺序表插入（C9）

思路　在第 i 个位置插入，原来的 $a_i, a_{i+1}, \ldots, a_n$ 都要后移一位，个数 $= n - i + 1$。



用 $n = 5$、$i = 2$ 代入检验四个选项。

边界检查　$i = 1$ 时移动 n 个，$i = n + 1$ 时移动 0 个，只有 $n - i + 1$ 同时满足。

一句话　**从 a_i 到 a_n 共 $n - i + 1$ 个。**

第 4 题　答案：**D**　基础　考点：链式存储（C2、C11）

思路　链式存储用指针表示后继，结点放在哪里都行：碰巧连续可以，不连续也可以。



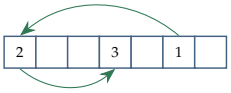
A　✗ 不要求连续



B　✗ 没有“部分必须连续”的要求



C　✗ 连续存放也合法（只要有指针）



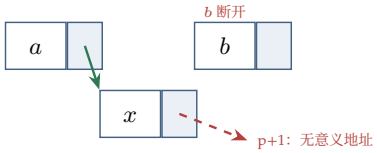
D　✓ 上面三种摆法都可以是链式存储

判断　四幅图中的数字表示 a_1, a_2, a_3 所在单元，绿色箭头是指针。A、B 把一种摆法说成“必须”；C 说“一定不连续”，但图 C 中结点恰好连续、靠指针相连，仍是合法的链式存储；只要每个结点带有指向后继的指针，任何摆法都是合法的链式存储，所以 D 正确（课件 2 P64）。

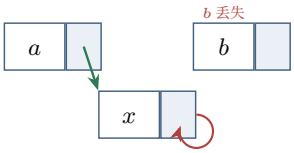
一句话　**链式存储“不要求”连续，而不是“要求不连续”。**

第 5 题　答案：**D**　基础　考点：单链表插入（C12）

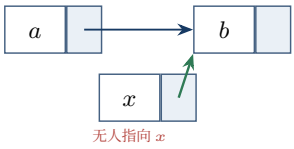
思路　“先武装自己，再融入队伍”：先让 s 指向 p 的后继，再让 p 指向 s。下面把每个选项执行后的状态画出来（a 为 p，b 为原后继）。



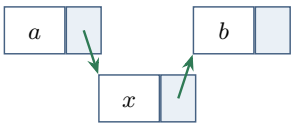
A　✗ p+1 不是后继结点的地址



B　✗ 顺序反了：s->next 变成 s



C　✗ p->next 又指回 b，s 没接进来

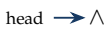


D　✓ $a \to x \to b$

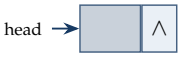
逐项　A：p+1 是指针算术，指向 p 所指结点之后的内存，而链表结点不连续，那里不是 b。B：(*p).next=s 先改了 p->next，第二句 (*s).next=(*p).next 得到 s->next=s。C：第一句正确，第二句 p->next=s->next 等于让 p->next 仍指向 b，新结点没接进链。D 正确。

一句话　**先改新结点，再改链上结点。**

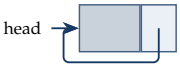
第 6 题　答案：**B**　基础　考点：头结点与空表（C11）



A　✗ 这是**无**头结点的空表



B　✓ 头结点之后为空



C　✗ 这是**循环**链表的空表

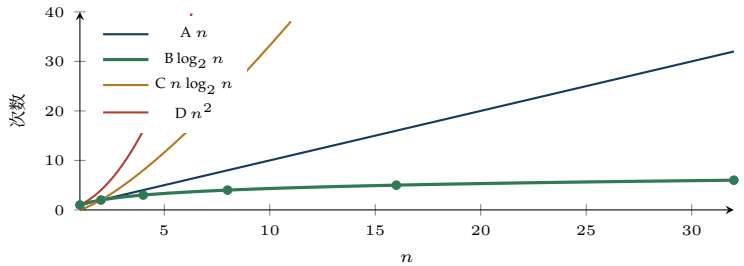


D　✗ 只说明头结点存在，非空表也成立

一句话　**带头结点：看头结点的 next；不带头结点：看 head 本身。**

第 7 题　答案：**B**　基础　考点：时间复杂度（C5）

推理　i 依次取 $1, 2, 4, \ldots, 2^{k-1}$，要求 $2^{k-1} \le n < 2^k$，循环体执行 $k = \lfloor \log_2 n \rfloor + 1$ 次（例 1.4），故 $O(\log_2 n)$。



四个选项的增长曲线（共同坐标）。绿点是程序实测次数：$n = 32$ 时只有 6 次，贴着 $\log_2 n$ 曲线。

错误选项　A 把“循环一直到 n”误当成执行 n 次；C、D 误把单层循环当成嵌套。

一句话　**循环变量成倍增长→对数阶。**

第 8 题　答案：C　基础　考点：逻辑结构的分类（C1）

思路　划分方法一：线性结构（线性表、栈、队列、串）与非线性结构（树、图）。A“动态/静态”、B“紧凑/非紧凑”、D“内部/外部”都不是逻辑结构的分类（它们描述的是存储分配方式或存储位置），这三个选项没有可画的逻辑关系，只为 C 画图：

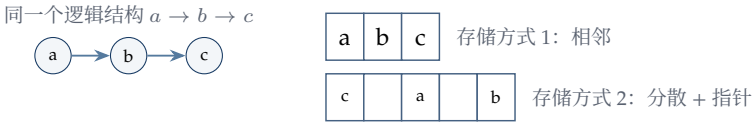


逻辑结构按“关系的对数”分为线性与非线性。

一句话　**逻辑结构只分线性与非线性（细分为集合、线性、树、图）。**

第 9 题　答案：C　基础　考点：逻辑结构的定义（C1、C2）

思路　逻辑结构只描述元素间的关系；存储结构要考虑元素在内存中的相对位置（A、B 错），运算实现依赖存储结构（D 错）。

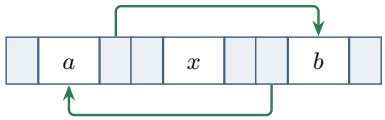


相对位置变了，存储结构不同，逻辑结构不变。

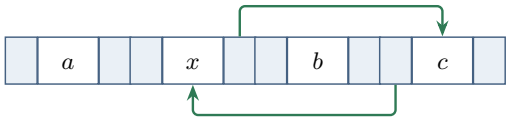
一句话　**“与形式、内容、相对位置、个数无关”= 逻辑结构。**

第 10 题　答案：A　基础　考点：双向链表删除（C14）

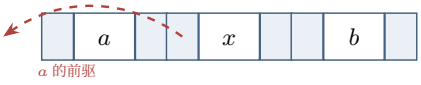
思路　删除 p：前驱的 `next` 越过 p，后继的 `prior` 越过 p。逐项画出执行后的状态（a 为前驱，x 为 `p`，b 为后继，c 为 b 的后继）：



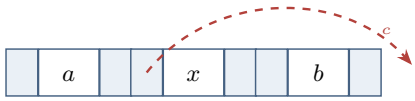
A　✓ $a \leftrightarrow b$，x 被摘出



B　✗ 删的是 p 的**后继** b



C　✗ 第一句无变化，第二句把 p 的左链改到 a 的前驱



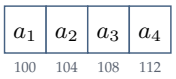
D　✗ p 没被删除，自身左指针还被改到 c

逐项　A：`p->next->prior=p->prior` 让 b 的左手拉 a；`p->prior->next=p->next` 让 a 的右手拉 b，正确（两句可交换）。B：`p->next` 被改成 c，再让 c 的左手拉 p——删掉的是 b。C：`p->prior->next=p` 本来就成立，什么都没变；第二句只改了 p 自己的左指针。D：第一句让 `p->prior` 指向 c；第二句 `p->next = p->prior->prior` 即 `c->prior`，仍是 b。a、b 的指针都没改，p 根本没被摘出，反而把自己的左指针改错了。

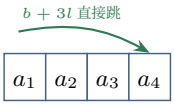
一句话　**删 p：改“前驱的右手”和“后继的左手”，p 自己的指针不用动。**

A2　多项选择题解析

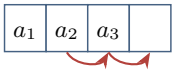
第 11 题　答案：ABD　基础　考点：顺序表的特点（C8、C9、C15）



A ✓ 逻辑相邻⇒物理相邻



B ✓ 地址可直接算出，$O(1)$



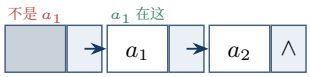
C ✗ 插入要后移、删除要前移



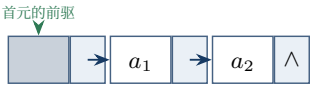
D ✓ 不需要额外存指针

一句话　顺序表：相邻、随机存取、密度大；代价是插删要移动。

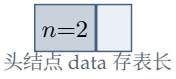
第 12 题　答案：BC　基础　考点：头结点（C11）



A ✗ 头结点不是元素，a_1 在首元结点



B ✓ 首元结点也有前驱，插删写法统一



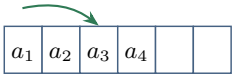
C ✓ 数据域可空或存表长



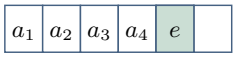
D ✗ 带头结点时 head 永远非空

一句话　头结点是“哨兵”，不是元素；空表判断看 `head->next`。

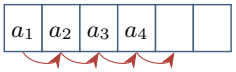
第 13 题　答案：AB　基础　考点：顺序表各操作的代价（C8、C9）



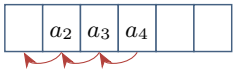
A ✓ 一次地址计算，$O(1)$



B ✓ 插在表尾，移动 0 个



C ✗ 插在表头，移动 n 个，$O(n)$



D ✗ 删第 1 个，移动 $n-1$ 个，$O(n)$

一句话　顺序表的代价取决于“要移动多少元素”：表尾 $O(1)$，表头 $O(n)$。

第 14 题　答案：ABD　基础　考点：算法的特性（C4）

思路　五特性：有穷性、确定性、可行性、输入、输出。可读性是**设计要求**（另外三项是正确性、健壮性、高效性）。四个选项都是概念名称，没有可单独画出的结构，用一张归类图说明：

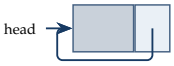
特性：**A 有穷**　**B 确定**　**D 可行**　输入　输出

设计要求：正确　**C 可读**　健壮　高效

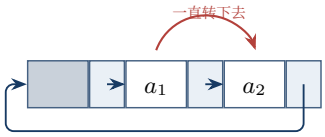
C 属于右边的“设计要求”。

一句话　特性管“是不是算法”，要求管“好不好”。

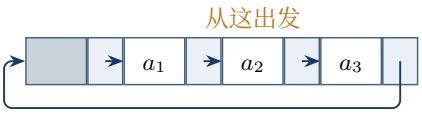
第 15 题　答案：**AC**　基础　考点：循环链表（C13）



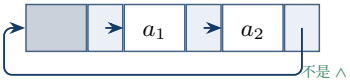
A　✓空表：头结点指向自己



B　✗p 永远不会变成 `nullptr`，死循环



C　✓从 a_2 出发也能到 a_1



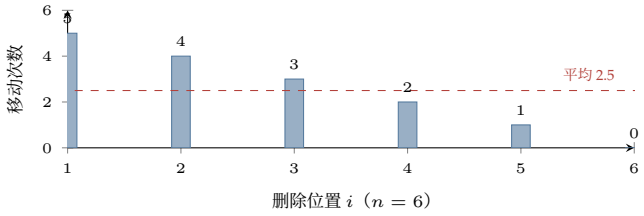
D　✗尾结点指回头结点

一句话　**循环链表没有 `nullptr`：判空看 `head->next==head`，遍历看 `p!=head`。**

A3　填空题解析

第 16 题　答案：$\dfrac{n-1}{2}$　基础　考点：删除的平均移动次数（C9）

推导　删除第 i 个要移动 $a_{i+1}..a_n$ 共 $n-i$ 个，$p_i=\frac{1}{n}$：$E=\frac{1}{n}\sum_{i=1}^{n}(n-i)=\frac{1}{n}\cdot\frac{n(n-1)}{2}=\frac{n-1}{2}$。



$n=6$：$(5+4+3+2+1+0)/6=2.5=\frac{6-1}{2}$。

易错　与插入的 $\frac{n}{2}$ 混淆。插入有 $n+1$ 个位置，删除只有 n 个。　**等价答案：**$(n-1)/2$。

第 17 题　答案：$O(mn)$　基础　考点：乘法规则（C5）

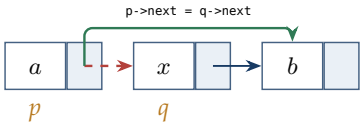
推理　内层次数 m 与外层的 i 无关，乘法规则直接适用：$\sum_{i=0}^{n-1}\sum_{j=0}^{m-1}1=nm$。



每格一次 `a[i][j]=0`。

等价答案　$O(m\times n)$、`O(m*n)`。**易错：**写成 $O(n^2)$——m 与 n 是两个独立的规模，不能合并。

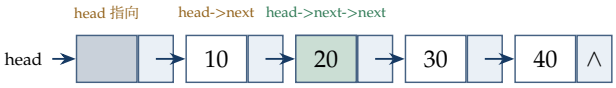
第 18 题　答案：**`q->next`**　基础　考点：单链表删除（C12）



让 p 越过 q 指向 q 的后继。

等价答案　`p->next->next`（此时 `p->next` 还是 `q`，两者等价）。**易错：**填 `q`（没变化）或 `nullptr`（把后面整条链丢掉）。

第 19 题　答案：**20**　基础　考点：头指针、头结点、首元结点（C11）

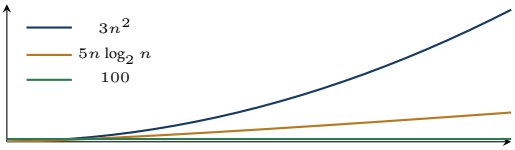


head 指头结点，走一步到 10，再走一步到 20。

易错　答 30——把 head 当成首元结点。有头结点时 `head->next` 才是 a_1。

第 20 题　答案：$O(n^2)$　基础　考点：只看最高阶（C5）

推理　比较三项的阶：$n^2 > n\log_2 n > 1$（图 1.12），保留 $3n^2$，去系数得 $O(n^2)$。$n=1000$ 时 $3n^2 = 3\times10^6$，$5n\log_2 n \approx 5\times10^4$，后者约是前者的 1.7%。



n 稍大，$3n^2$ 就远远压过另两项。

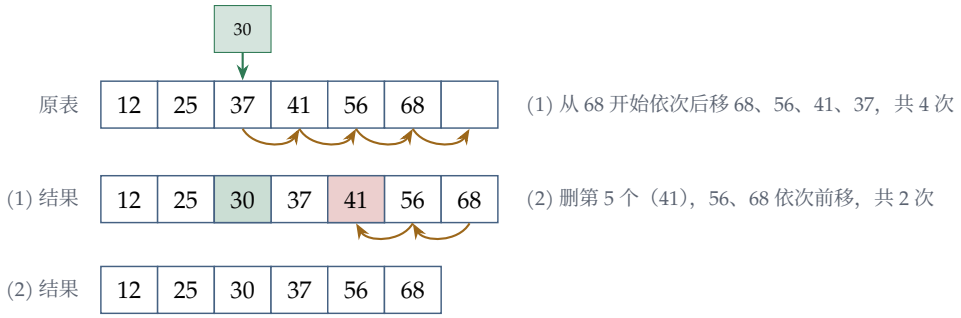
易错　写成 $O(n^2+n\log n)$——大 O 只保留最高阶。

A4　解答题解析

第 21 题　答案：**见下**　基础　考点：顺序表插入删除（C9）

题意分析　已知表长 $n=6$、容量 10（不满）。(1) 插入位置 $i=3$ 合法（$1\le3\le7$）；(2) 插入后表长 7，删除位置 $i=5$ 合法（$1\le5\le7$）。

使用公式　插入移动 $n-i+1$ 个，删除移动 $n-i$ 个。



两步操作的关键过程图（程序实测结果相同）。

结果　(1) $(12,25,30,37,41,56,68)$，后移 37、41、56、68，移动 $6-3+1=4$ 次；(2) $(12,25,30,37,56,68)$，带回 $e=41$，移动 $7-5=2$ 次。

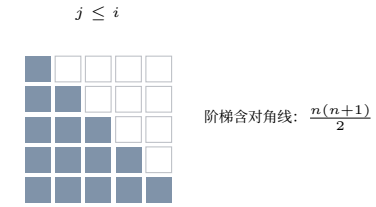
检查　表长 $6\to7\to6$；插入后 30 恰在第 3 位。

易错　(2) 中仍用原表长 6 计算（得 1 次）——删除时表长已经是 7。

同类题　先确认表长与位置合法，再用 $n-i+1$ / $n-i$，最后画图核对。

第 22 题　答案：$\frac{n(n+1)}{2}$，$O(n^2)$　基础　考点：求和计算频度（C5）

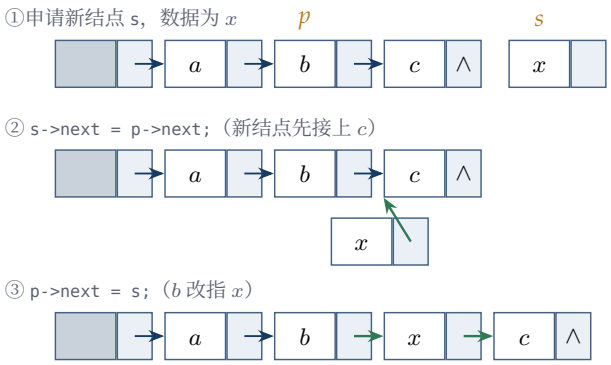
分步　外层 $i=1..n$，内层 $j=1..i$ 执行 i 次：$\sum_{i=1}^{n}\sum_{j=1}^{i}1=\sum_{i=1}^{n}i=\frac{n(n+1)}{2}=\frac{1}{2}n^2+\frac{1}{2}n$，最高阶 $\frac{1}{2}n^2$，故 $O(n^2)$。



$n=5$：$1+2+3+4+5=15=\frac{5\times6}{2}$。

检查　$n=10$ 时程序实测 55 次。**易错：**与例 1.3（`j<i`，$\frac{n(n-1)}{2}$）混淆——差别就在对角线上那 n 格。

第 23 题　答案：**见下**　基础　考点：单链表插入（C12）



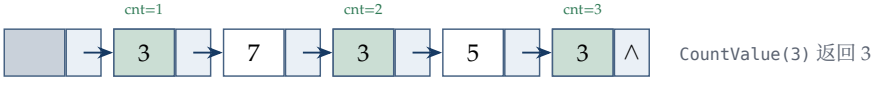
插入的三个状态。

```
Node<char> *s = new Node<char>('x');   // ① 申请结点，data='x', next=nullptr
s->next = p->next;                     // ② 先接后继 c
p->next = s;                           // ③ 再让 b 指向 s
// 合并写法：p->next = new Node<char>('x', p->next);
```

结果　(a,b,x,c)。**检查：**从头遍历 $a\to b\to x\to c\to\wedge$，无断链。**易错：**②③交换。

第 24 题　答案：**见下**　基础　考点：单链表遍历（C12）

```
template <class ElemType>
int SimpleLinkList<ElemType>::CountValue(const ElemType &e) const {
    int cnt = 0;
    for (Node<ElemType> *p = head->next; p != nullptr; p = p->next)   // 从首元结点走到表尾
        if (p->data == e) cnt++;
    return cnt;
}
```

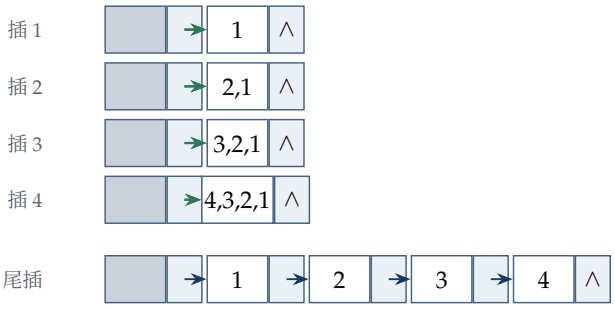


遍历过程中 `cnt` 的变化。

分析　每个结点访问一次，$T(n)=O(n)$；只用了 `cnt`、`p`，$S(n)=O(1)$。末尾 `const` 表示不修改链表。程序实测：(h,e,l,l,o) 中 `'l'` 的个数为 2。

易错　从 `head` 开始比较——会把头结点的数据域（无意义）也算进去。

第 25 题　答案：**见下**　基础　考点：头插法与尾插法（C12）



头插法每次插在头结点之后（绿线），最终 4,3,2,1；尾插法得 1,2,3,4。

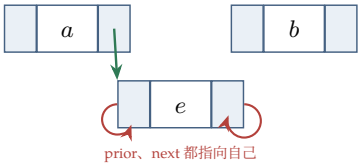
结论　头插法得到的结点次序与输入次序**相反**，因为后输入的元素总排在前面；尾插法**相同**。两者时间都是 $O(n)$（尾插法靠 `tail` 指针避免每次从头找表尾）。

同类题　要“保持输入顺序”用尾插法；要“逆序”或实现栈用头插法。

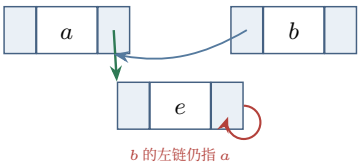
详细解析：B. 中等偏下提升

第 26 题　答案：**C**　提升　考点：双向循环链表插入（C14）

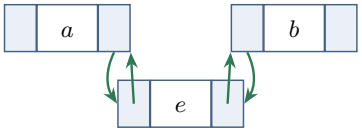
知识链　对称性→四个指针→顺序约束（“改后继的左指针”必须在“改 p 的右指针”之前）。逐项执行，画出结果（a 为 `p`，b 为原后继，e 为 `q`）：



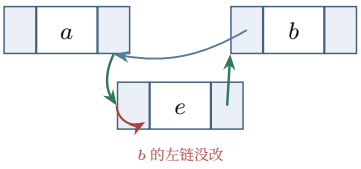
A　✗ `q->next=q`，b 被甩出



B　✗ `q->next` 最后被赋成 `q`



C　✓四个指针全部正确



D　✗第 4 句改的是 `q->prior`

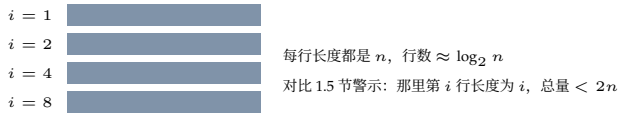
逐项　A：`p->next=q` 后，第 3 句 `p->next->prior=q` 实为 `q->prior=q`，第 4 句 `q->next=q`。B：前两句后 `q->prior=q`，第 3 句改回 `q->prior=p`，第 4 句 `q->next=p->next=q`；b 的左指针始终未改。C：①②武装 q，③通过 `p->next`（此时还是 b）改 b 的左指针，④最后改 `p->next`，正确。D：③④顺序颠倒，`p->next->prior=q` 变成 `q->prior=q`。

检验　C 执行后：`q->prior->next==q`（a 的右指 q）且 `q->next->prior==q`（b 的左指 q），对称性成立。

一句话　**先武装新结点，再从“远”到“近”修改：先改后继，最后改 p。**

第 27 题　答案：**B**　提升　考点：乘法规则与对数（C5）

推理　外层 $i = 1, 2, 4, \ldots$，执行 $\lfloor \log_2 n \rfloor + 1$ 次；内层 $j = 1..n$，每次都执行 n 次，**与 i 无关**，可用乘法规则：$T(n) = (\lfloor \log_2 n \rfloor + 1) \cdot n = O(n \log_2 n)$。



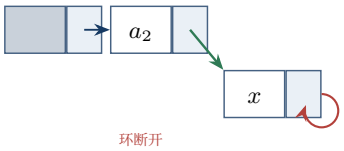
$n = 8$：4 行 × 8 列 = 32 次（程序实测 32，$n = 100$ 时实测 700）。

错误选项　A $O(n)$：把 1.5 节警示框“内层依赖外层、总量 $< 2n$”的结论硬套过来——那里内层是 `j<i`，本题是 `j<=n`，条件不同结论不同；C $O(n^2)$：误以为外层也执行 n 次；D $O(\log_2 n)$：只数了外层。

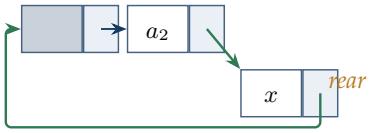
一句话　**先看内层次数与外层是否有关：无关才相乘。**

第 28 题　答案：B　提升　考点：尾指针循环链表（C13）

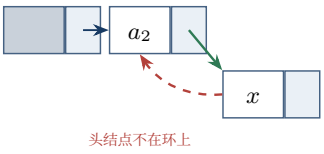
思路　表尾插入要让：新结点指向头结点（`rear->next`）→原表尾指向新结点→ `rear` 后移。取原表尾的 `next`（头结点地址）必须在修改它之前。



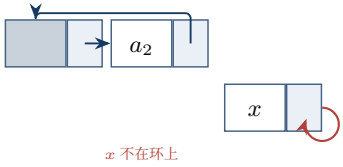
A　✗ `s->next=s`，头结点失联



B　✓ $a_2 \to x \to$ 头结点，`rear=s`



C　✗ x 指回原表尾，形成小环

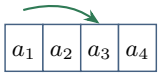


D　✗ 原表尾没接上，x 自环

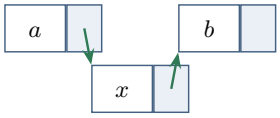
逐项　A：先 `rear->next=s`，再 `s->next=rear->next` 即 `s`。C：`s->next=rear` 指向原表尾而不是头结点。D：`rear=s` 之后的 `rear->next=s` 改的是 `s` 自己，原表尾仍指向头结点。B 与 4.3 节约瑟夫建环的写法一致。

一句话　**尾插三步：新结点接头结点→旧尾接新结点→尾指针后移。**

第 29 题　答案：ABD　提升　考点：顺序表与链表的选择（C15）



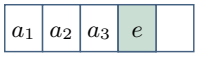
A　✓ 按序号访问 $O(1)$



B　✓ 插删不移动、长度可变



C　✗ 指针额外占空间，密度 < 1



D　✓ 表尾插入不移动元素

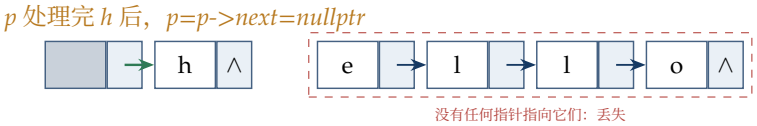
关键条件　D 的成立依赖“只在表尾”和“空间足够”两个条件；C 说反了——存储密度 = 数据所占 / 结点总占，链表的分母更大，所以小于 1。

一句话　**选择看主要操作：随机访问多选顺序表，插删多、长度变化大选链表。**

第 30 题　答案：ABC　提升　考点：逆置算法（C12、C6）

A、B　每个结点处理一次，$O(n)$；只用 `p`、`q`，$O(1)$。

C　去掉 `q` 后，`p->next` 在第②步已被改成 `head->next`（第一次是 `nullptr`），再执行 `p=p->next` 就走到了“已逆置部分”或空处，原链后面的结点全部丢失：



第一轮后循环就结束，只剩 h。

D　空表时 `p=head->next=nullptr`，`while(p!=nullptr)` 一次也不执行，函数正常返回（程序实测空表逆置后仍为空），所以 D 错。

一句话　**改 `p->next` 之前，必须先用“兄弟”`q` 记住后面的路。**

第 31 题　答案：2280；2792　提升　考点：地址公式（C8）

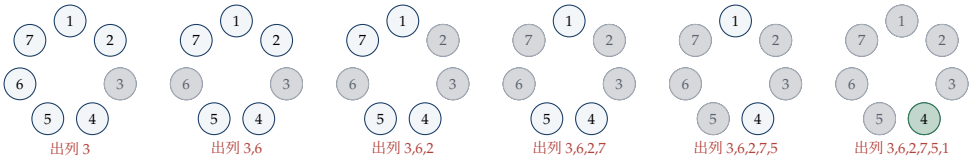
计算　$\mathrm{LOC}(a_{36}) = 2000 + (36-1)\times 8 = 2280$；`elems[99]` 是第 100 个单元：$2000 + 99\times 8 = 2792$。



位序 i 对应下标 $i-1$。

易错　第二空写成 $2000 + 100\times 8 = 2800$——那是数组之后的第一个字节，已越界。**最后一个单元的首地址**是 2792（它占 2792–2799）。

第 32 题　答案：3, 6, 2；4　提升　考点：约瑟夫问题（C13）



$n=7$、$m=3$：出列 3, 6, 2, 7, 5, 1，优胜者 4（程序实测）。

推理　1,2,**3** 出列；从 4 数：4,5,**6**；从 7 数：7,1,**2**；从 4 数：4,5,**7**；从 1 数：1,4,**5**；从 1 数：1,4,**1**；剩 4。

易错　出列后忘记“跳过已出列者”，或从出列者本人重新数起。

第 33 题　答案：4 项；$((5,0),(1,1),(2,3),(3,9))$　提升　考点：稀疏多项式相加（C10）

步	a	b	比较	C 的变化
1	(5,0)	(1,1)	$0<1$	放 (5,0)
2	(2,3)	(1,1)	$3>1$	放 (1,1)
3	(2,3)	(4,6)	$3<6$	放 (2,3)
4	$(-4,6)$	(4,6)	$6=6$	系数 0，**不放**
5	—	(3,9)	A 已完	复制 (3,9)

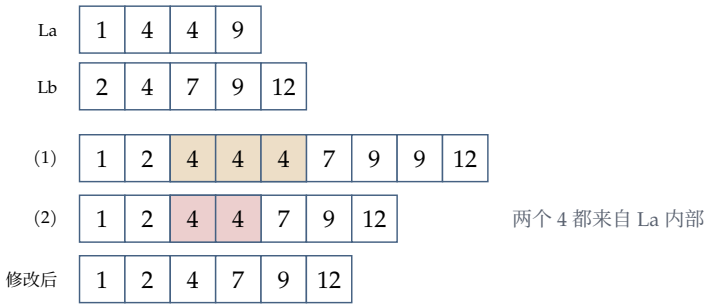
归并过程表：比较指数，相等则系数相加。

结果　$C(x) = 5 + x + 2x^3 + 3x^9$，共 4 项。**易错：**把 $(0,6)$ 也放进去写成 5 项。

第 34 题　答案：见下　提升　考点：有序归并（C10、C5）

1. 题目在问什么　同一组数据，用两种归并规则各算一次，并解释差别；最后分析复杂度。

2. 知识链　有序归并（双指针）→相等时的处理规则→循环不变式→复杂度。



三种结果（程序实测）。

3. 第一问　保留重复规则：$a \le b$ 取 a，否则取 b。依次放入 1(a)、2(b)、4(a)、4(a)、4(b)、7(b)、9(a)、9(b)，La 完，复制 12：**Lc=(1,2,4,4,4,7,9,9,12)**，共 $4+5=9$ 个，长度检查无误。

4. 第二问　`allowDup=false` 时相等则只放一个并同时后移 i、j：放 1、2；$a=4=b$ 放一个 4，i、j 后移；此时 $a=4$（La 的第二个 4）、$b=7$，$a<b$ 又放一个 4；之后放 7、9（相等只放一个）、12。**Lc=(1,2,4,4,7,9,12)**。仍有重复，是因为该规则只消除“两表之间”的相等元素，而 La **自身**就有两个 4。**修改方法：**每次准备放入元素 x 时，先与 Lc 的表尾元素比较，不相等才插入（Lc 有序，重复元素一定紧挨着）：

```
ElemType last;
if (Lc.Length() == 0 || (Lc.GetElem(Lc.Length(), last), last != x))
    Lc.Insert(Lc.Length() + 1, x);
```

修改后结果 (1,2,4,7,9,12)。

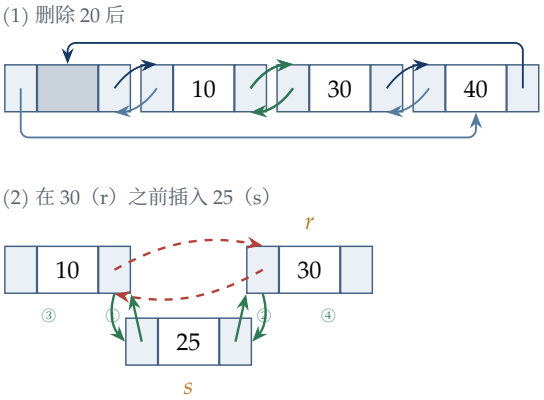
5. 第三问　每次循环恰好向 Lc 放入一个元素并后移一个指针，循环总次数 $=m+n$；取元素与表尾插入在顺序表上都是 $O(1)$，所以 $T=O(m+n)$。不是 $O(mn)$，因为利用了“两表都有序”：不需要拿 La 的每个元素去和 Lb 的所有元素比（那是并集、差集的做法）。

6. 最容易卡住　把纠错①中两种规则混用；或忘了 La 内部也可能重复。

7. 变式思考　若 La、Lb 用带头结点的单链表存放，能否不申请新结点、直接把两条链“拉链”成 Lc？此时空间复杂度是多少？（提示：只改指针，$O(1)$。）

第 35 题　答案：见下　提升　考点：双向循环链表（C14）

1. 题目在问什么　删除已知结点；在已知结点**之前**插入（正文讲的是“之后”，这里是迁移）；比较单、双链表的代价。



(1) 删除后 10 与 30 互指；(2) 前插四步，③通过 `r->prior` 找到 10，必须在④之前。

2. 第一问

```
p->prior->next = p->next;     // 10 的右手越过 20 指向 30
p->next->prior = p->prior;    // 30 的左手越过 20 指向 10
delete p;                     // 释放 20
```

结果 $L=(10,30,40)$。**检查对称性：**10 的 next 是 30，30 的 prior 是 10。

3. 第二问

```
s->prior = r->prior;          // ① s 的左手拉住 r 的前驱 (10)
s->next  = r;                 // ② s 的右手拉住 r
r->prior->next = s;           // ③ 10 的右手改拉 s   —— 必须在 ④ 之前
r->prior = s;                 // ④ 最后改 r 的左手
```

注意点：与“后插”镜像对称——③要借 `r->prior` 找到前驱 10，若先执行④，`r->prior` 已变成 `s`，③就成了 `s->next=s`。结果 $L=(10,25,30,40)$。

4. 第三问　(1)(2) 都只改常数个指针，$O(1)$。若用单链表实现 (2)，结点没有 `prior`，必须从头结点出发找到 `r` 的前驱，代价 $O(n)$——这就是双向链表“多存一个指针”换来的好处。

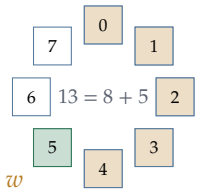
5. 最容易卡住　③④顺序；以及删除后忘记 `delete p`。

6. 变式思考　若只给出指向 10 的指针 `q`，要在 10 之后插入 25，四句应怎样写？（答案就是 4.4 节代码中的 `InsertAfter` 函数。）

详细解析：C. 跨学科迁移

第 36 题　答案：**A**　跨学科　考点：循环结构与模运算（C13）

推理　写入 13 个后 $w = 13 \bmod 8 = 5$：前 8 个把 w 从 0 推到 8，取模回到 0；再写 5 个，$w = 5$。



橙色：第 9–13 个采样覆盖了最旧数据

B 13：忘了取模（数组只有 0–7）

C 4：最后一个采样写在 4，w 已指向下一格

环形缓冲区的状态。

当前专题知识　循环链表“尾指回头”的思想（4.3 节对照：模运算）。

借用的外部情境　信号采集中的环形缓冲区。

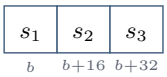
为什么不算超纲　题干给出了完整的更新语句和容量，只需按步骤取模，不需要任何信号处理知识。

迁移点　“走到末尾回到开头”在链表中靠 `rear->next=head` 实现，在数组中靠 $(w+1) \bmod N$ 实现。D 6 是把 13 次写入错数成 14 次。

第 37 题　答案：**ABC**　跨学科　考点：顺序表与链表的选择、地址公式（C15、C8）



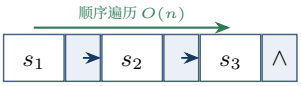
A ✓ 长度未知 + 中间删除



C ✓ 首地址 = 基址 + $(k-1) \times 16$



B ✓ 按序号直接读第 k 个



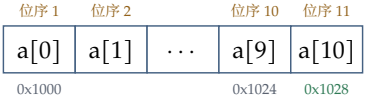
D ✗ 顺着指针就能按次序遍历

对照说明　D 把“物理不连续”误当成“逻辑无序”——链表的逻辑次序由指针保证（第 5 章对照：RAM 与磁带）。实际工程中常两者结合：采集时用链表或分块存，删除异常后再拷入顺序表画图。

当前专题知识　第 5 章的选择原则与地址公式。**借用情境：**控制实验的数据记录。**为什么不算超纲：**采样率、每条记录大小都已给出，判断只依赖“访问方式、插删频率、长度是否已知”。**迁移点：**“表长变化大、频繁插删→链表；按序号访问→顺序表”。

第 38 题　答案：**0x1028；11**　跨学科　考点：地址公式与位序/下标（C8）

计算　`a[10]` 前面有 `a[0]..a[9]` 共 10 个元素：地址 = `0x1000` + 10×4 = `0x1000` + 40 = `0x1000` + `0x28` = `0x1028`。下标 10 对应位序 11，所以是第 11 个元素。



下标 k ↔位序 $k+1$；地址 = 首地址 + $k \times 4$。

当前专题知识　$\mathrm{LOC}(a_i)=\mathrm{LOC}(a_1)+(i-1)l$。**借用情境：**计算机组成中的字节编址与十六进制。**为什么不算超纲：**题干给出了 0x28=40 的换算。**迁移点：**C++ 下标 k 就是公式中的 $i-1$，所以 `&a[k] == a + k`。

易错　写成 `0x1040`（把十进制 40 直接当十六进制）或 `0x102C`（用了位序 11 去乘）。

总结：高频易错点总表

	错误想法 / 写法	正确理解	位置
1	数据元素是最小单位	元素是基本单位，数据项是不可分割的最小单位	1.1
2	链式存储的地址一定不连续	不要求连续，碰巧连续也可以	1.2
3	可读性、健壮性是算法特性	它们是设计要求；特性是有穷、确定、可行、输入、输出	1.4
4	$O(f)$ 就是“与 f 同阶”	大 O 只是上界：$3n=O(n^2)$	1.5
5	内外层循环次数直接相乘	内层依赖外层时要求和	1.5
6	线性表元素必须有序、每个都有前驱后继	只要求位置上一对一；首无前驱、尾无后继；可为空	2.1
7	插入、删除合法位置相同	插入 $1\le i\le n+1$，删除 $1\le i\le n$	2.2
8	地址公式乘 i	乘 $i-1$；C++ 下标 $k=i-1$	3.1
9	顺序表插入时从前往后移	必须从后往前移，否则覆盖	3.3
10	插入平均移动 $n/2$，删除也是 $n/2$	删除是 $(n-1)/2$	3.3
11	顺序表作形参用值传递	用引用 `&L` 或 `const &L`	3.2
12	`p->next=s; s->next=p->next;`	先 `s->next=p->next;` 再 `p->next=s;`	4.2
13	删除时不保存、不释放结点	`q=p->next; p->next=q->next; delete q;`	4.2
14	带头结点空表判断 `head==nullptr`	`head->next==nullptr`（循环链表为 `==head`）	4.1
15	循环链表遍历用 `p!=nullptr`	用 `p!=head`	4.3
16	尾指针合并漏写 `Tb->next=p`	四步缺一不可，新表尾为 `Tb`	4.3
17	双向插入先改 `p->next` 再改 `p->next->prior`	改后继的 prior 必须在改 `p->next` 之前	4.4
18	“链表插删 $O(1)$，所以总比顺序表快”	$O(1)$ 前提是已知位置；按位序仍需 $O(n)$ 查找	第 5 章

基础解题步骤总结

五类题的通用步骤

①复杂度分析：找最内层循环体→判断内层是否依赖外层（无关相乘，有关求和）→写频度→保留最高阶、去系数。

②顺序表插入/删除：确认表长 n 与位置合法→插入从后往前移 $n-i+1$ 个 / 删除从前往后移 $n-i$ 个→表长 ±1 →画图核对。

③地址计算：$\mathrm{LOC}(a_i)=\mathrm{LOC}(a_1)+(i-1)l$；题目给的是下标就用 k 代替 $i-1$。

④链表指针题：先画出“操作前”的图→每执行一句就在图上改一根线→检查有没有断链、自环、悬空指针→双向链表再查对称性。

⑤写算法：用“五步口诀”：参数检查→空间检查→核心操作→对其他属性的影响→正确返回；链表操作额外检查空表与表尾两个边界。

相近概念总对比

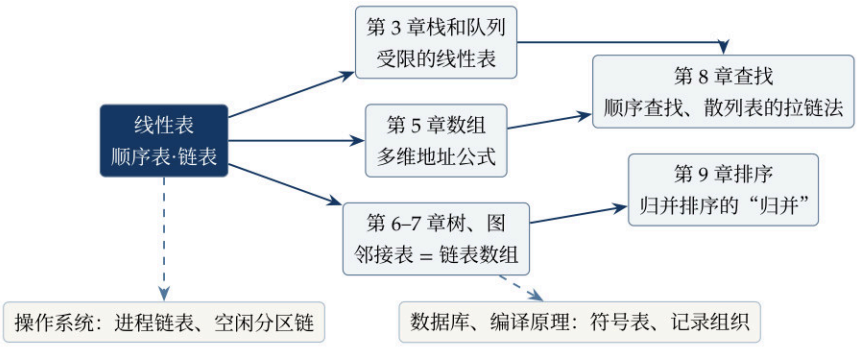
概念对	区别	判断方法
逻辑结构 / 存储结构	前者只管关系，与计算机无关；后者是关系在内存中的表示	换一种存法，变的是存储结构
数据元素 / 数据项	一行 / 一格	能否再分
算法特性 / 设计要求	是不是算法 / 好不好	缺了它还算不算算法
`count` / `maxSize`	实际元素个数 / 容量	插入后哪个变
`Clear` / 析构	表还在、为空 / 表不存在	之后还能不能用
头指针 / 头结点 / 首元结点	指针变量 / 附加的结点 / 存 a_1 的结点	是不是结点、存不存 a_1
头插法 / 尾插法	逆序 / 保序	新结点接在头结点后还是表尾后
单链表 / 循环链表	尾为 `nullptr` / 尾指回头	判空与遍历终止条件
单链表 / 双向链表	只能向后 / 可前可后	找前驱的代价 $O(n)$ / $O(1)$
随机存取 / 顺序存取	地址可算出 / 只能沿链走	访问第 i 个是 $O(1)$ 还是 $O(n)$

跨学科 / 相似知识总对比表

本专题知识	相似知识（课程）	类型	最重要的迁移结论
数据元素 / 数据项	记录 / 字段（数据库）	类比同构	✓整体—部分；✗数据库表的行无次序
逻辑 / 存储结构	逻辑 / 物理地址（操作系统）	同名异义	△顺序表只在虚拟地址中连续
ADT	向量空间（线性代数）	类比同构	✗不同实现代价不同
算法	程序（程序设计）	同名异义	✗程序可以不终止
大 O	无穷大的阶（高等数学）	同名异义	✗O 不等于同阶
空间复杂度	时间复杂度（本课程）	方法互补	✗空间复用不累加
线性表	数列 / 集合（高数、离散）	推广 / 异义	✗有限、有序、可重复
地址公式	等差数列通项（中学数学）	类比同构	✓首项 + $(i-1)$ 公差
平均移动次数	数学期望（概率论）	思想重现	△依赖等概率假设
多项式相加	向量加法（线性代数）	类比同构	✗多项式乘法不是逐项乘
单链表	`std::forward_list`（C++）	思想重现	✓插删需前驱
头插法	栈（第 3 章）	思想重现	✓后进先出
循环链表	模运算（数论）	类比同构	✗链表走 k 步要 $O(k)$

本专题知识	相似知识（课程）	类型	最重要的迁移结论
双向链表	无向边（离散数学）	类比	△两个指针都要维护
顺序表 / 链表	RAM / 磁带（计算机组成）	同名异义	✗链表在 RAM 中仍是顺序存取

后续课程连接



线性表在后续课程中的再现。实线：本课程；虚线：其他课程。

你接下来最先会用到的：第 3 章的链栈就是“只在表头插删的单链表”（4.2 节对照），循环队列就是“数组 + 取模”（4.3 节对照）；第 9 章归并排序的核心步骤就是例 2.2 的有序归并。

参考资料

原始教学依据

- 某高校相关学院　　《数据结构与算法》课件：第 1 章绪论（84 页）、第 2 章线性表（69 页），2026 年秋。
- 游洪跃，唐宁九. 数据结构与算法（C++ 版）[M]. 第 2 版.（课程指定教材，课件中的类模板与之一致）
- 本人课堂笔记 Memo 543–556（2026-10-16 至 10-18）；本人代码文件夹「线性表」中的 6 个.cpp 文件。
- 青岛大学数据科学与软件工程学院《数据结构与算法基础》视频 2.6 节“顺序表和链表的比较”截图。

核心定义与公式核验

- 严蔚敏，吴伟民. 数据结构（C 语言版）[M]. 北京：清华大学出版社.（线性表 ADT、平均移动次数、循环与双向链表的标准表述）
- Knuth D E. The Art of Computer Programming, Vol. 1: Fundamental Algorithms[M]. Addison-Wesley, 1968.（顺序分配与链接分配）
- Cormen T H, et al. Introduction to Algorithms[M]. 3rd ed. MIT Press, 2009.（渐近记号的定义）

代码与工程知识核验

- cppreference.com: `std::forward_list`（`before_begin`、`insert_after`，无 `size` 成员）；`std::nothrow`。

- The Linux Kernel documentation: “Linked Lists in Linux”（`include/linux/list.h`，循环双向链表）。
- 本讲义全部 C++ 代码均在 g++ 13（`-std=c++17 -Wall -fsanitize=address,undefined`）下编译运行，输出与正文一致。

学科史资料

- Wikipedia: “Linked list”条目 History 一节（IPL 1955–1956，LISP 1958–1960）。
- Knuth D E. Big Omicron and big Omega and big Theta[J]. SIGACT News, 1976, 8(2): 18–24.
- Liskov B, Zilles S. Programming with abstract data types[J]. ACM SIGPLAN Notices, 1974, 9(4): 50–59.
- Wirth N. Algorithms + Data Structures = Programs[M]. Prentice-Hall, 1976.